

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 1_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 13

Section 1 : MCQ

1. An algorithm must have a clear stopping point to ensure

Answer

Finiteness

Status : Correct

Marks : 1/1

2. Which of the following is true about algorithmic determinism?

Answer

Algorithms must produce the same output for the same input

Status : Correct

Marks : 1/1

3. An algorithm's effectiveness refers to

Answer

Each step contributes to solving the problem

Status : Correct

Marks : 1/1

4. Which notation is used to describe both the upper and lower bounds of an algorithm's efficiency?

Answer

Θ (Big theta)

Status : Correct

Marks : 1/1

5. What is the purpose of using asymptotic notations in the analysis of algorithm efficiency?

Answer

To take meaningful statements about an algorithm's efficiency based on its growth rate

Status : Correct

Marks : 1/1

6. What does the Big Theta notation (Θ) indicate about an algorithm's efficiency?

Answer

It describes the tightest possible bound on the algorithm's running time.

Status : Correct

Marks : 1/1

7. Which of the following statements is true regarding the relationship between Big Oh (O), Big Omega (Ω), and Big Theta (Θ) notations?

Answer

If $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$, then $f(n) = \Theta(g(n))$

Status : Correct

Marks : 1/1

8. What does "Big O" notation represent in relation to algorithms?

Answer

A method to analyze and express the efficiency of an algorithm

Status : Correct

Marks : 1/1

9. An algorithm is required to produce a definite and unambiguous outcome for every valid input. This characteristic is known as _____.

Answer

determinism

Status : Correct

Marks : 1/1

10. Which approach is used to prove the correctness of algorithms by analyzing a sequence of steps?

Answer

Mathematical induction

Status : Correct

Marks : 1/1

11. In the context of algorithm design, what does the term "in-place" refer to?

Answer

The algorithm modifies the input data directly without using additional memory

Status : Correct

Marks : 1/1

12. What is the primary purpose of analyzing an algorithm's efficiency?

Answer

To optimize the algorithm's time and space efficiency

Status : Correct

Marks : 1/1

13. In the context of algorithmic problem-solving, what does the "generality of an algorithm" refer to?

Answer

The efficiency of the algorithm in terms of time and space

Status : Wrong

Marks : 0/1

14. Which parameter is commonly used to measure an algorithm's input size?

Answer

Number of elements in the input array/list

Status : Wrong

Marks : 0/1

15. The concept of "Orders of Growth" in algorithm analysis refers to:

Answer

The function that describes the algorithm's efficiency

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 2_COD

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Given an integer n. The task is to find the first n Fibonacci Numbers.

Input Format

The input consists of a single integer n, representing how many Fibonacci terms to print.

Output Format

The output prints n space-separated Fibonacci numbers starting from 0.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

Output: 0 1 1

Answer

```
#include <stdio.h>
```

```
int main() {  
    int n;  
    scanf("%d", &n);  
  
    int a = 0, b = 1, next;  
  
    for (int i = 1; i <= n; i++) {  
        if (i == 1) {  
            printf("%d", a);  
        }  
        else if (i == 2) {  
            printf(" %d", b);  
        }  
        else {  
            next = a + b;  
            printf(" %d", next);  
            a = b;  
            b = next;  
        }  
    }  
  
    return 0;  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Given a string *s*, check if it is a palindrome using recursion. A palindrome is a word, phrase, or sequence that reads the same backward as forward.

Input Format

The input consists of a single string *s* containing no spaces.

Output Format

The output prints "The string is a palindrome" if the string is symmetric.

Otherwise, prints "The string is not a palindrome".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: abba

Output: The string is a palindrome

Answer

```
#include <stdio.h>
#include <string.h>

// Recursive function to check palindrome
int isPalindrome(char str[], int start, int end) {
    if (start >= end)
        return 1;

    if (str[start] != str[end])
        return 0;

    return isPalindrome(str, start + 1, end - 1);
}

int main() {
    char str[51];
    scanf("%s", str);

    int len = strlen(str);

    if (isPalindrome(str, 0, len - 1))
        printf("The string is a palindrome");
    else
        printf("The string is not a palindrome");
}
```

```
    return 0;  
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Shilpa is learning about the Fibonacci sequence for her computer science class. She wants to generate the first N terms of the Fibonacci series.

Can you help Shilpa by writing a program that takes a number N and prints the first N terms of the Fibonacci series using recursion?

Input Format

The input consists of an integer N, representing the number of terms in the Fibonacci sequence that need to be printed.

Output Format

The output prints the first N Fibonacci numbers, separated by spaces.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

Output: 0 1 1 2 3 5

Answer

```
import java.util.Scanner;
```

```
// You are using GCC
```

```
int fib(int n) {  
    if (n == 0)  
        return 0;  
    if (n == 1)  
        return 1;  
    return fib(n - 1) + fib(n - 2);  
}
```



```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
    int N = sc.nextInt();  
    for(int i = 0; i < N; i++) {  
        System.out.print(fib(i));  
        if(i != N - 1) System.out.print(" ");  
    }  
    System.out.println();  
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Ashish is preparing a mathematics assignment where he needs to calculate the factorial of a number. Since his teacher asked him to use recursion for this task, he plans to complete the method `int factorial` to solve it.

Help him to implement the task.

Input Format

The input consists of an integer num representing the number for which the factorial is to be calculated.

Output Format

The output prints the factorial value of num.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

Output: 6

Answer

```
#include <stdio.h>

int factorial(int num) {
    if (num == 0 || num == 1)
        return 1;
    return num * factorial(num - 1);
}

int main() {
    int num;
    scanf("%d", &num);
    printf("%d\n", factorial(num));
    return 0;
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 2_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Manoj loves puzzles and recently encountered an interesting mathematical challenge. He needs to find how many ways he can express a number x as the sum of distinct positive integers raised to the power of n . For example, for a given number x , he wants to find all possible ways to represent it as a sum of distinct powers of integers.

Help him implement this task.

Example 1

Input:

10

2

Output:

1

Explanation:

$x = 10$

$n = 2$

$10 = 12 + 32$, hence we have only 1 possibility.

Example 2

Input:

100

2

Output:

3

Explanation:

$x = 100$

$n = 2$

$100 = 102$ OR $62 + 82$ OR $12 + 32 + 42 + 52 + 72$, hence total 3 possibilities.

Function Specifications: `getAllWays(int, int, int)`

Input Format

The first line of input consists of an integer x , representing the target sum.

The second line of input consists of an integer n , representing the power to which the integers are raised.

Output Format

The first line of output prints: The total number of ways to express x as the sum of distinct powers.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 100

2

Output: 3

Answer

```
#include <stdio.h>
```

```
#include <math.h>
```

```
int getAllWays(int x, int n, int num) {  
    int power = pow(num, n);
```

```
    if (power == x)  
        return 1;
```

```
    if (power > x)  
        return 0;
```

```
    return getAllWays(x - power, n, num + 1) +  
        getAllWays(x, n, num + 1);  
}
```

```
int main() {  
    int x, n;
```

```
    scanf("%d", &x);  
    scanf("%d", &n);
```

```
    printf("%d", getAllWays(x, n, 1));
```

```
    return 0;  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Given an integer array of coins of size N representing different types of currency and an integer sum M. The task is to find the number of ways to make the sum by using different combinations from the coins array, using recursion.

Function Specifications: `int countWays(int [], int , int)`

Input Format

The first line of input consists of the number of coins N.

The second line consists of N integers denoting the coin denominations.

The third line consists of the integer sum M to be produced with those coins.

Output Format

The output displays the number of ways to make the sum by using different combinations from the coins array.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

1 2 3

4

Output: 4

Answer

```
#include <stdio.h>
```

```
int countWays(int coins[], int n, int sum) {
```

```
    if (sum == 0)
        return 1;
```

```

    if (sum < 0 || n == 0)
        return 0;

    return countWays(coins, n, sum - coins[n - 1]) +
        countWays(coins, n - 1, sum);
}

int main() {
    int n, sum;

    scanf("%d", &n);

    int coins[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &coins[i]);
    }

    scanf("%d", &sum);

    printf("%d", countWays(coins, n, sum));

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Meredith is a cashier at a local convenience store. To improve the efficiency of making change for customers, Meredith wants to develop a small program that calculates the number of different ways she can give change using a given set of coin denominations. This will help her quickly figure out the number of ways to make change for a particular amount of money using the available coins.

Meredith has a list of coin denominations and a specific amount of money for which she needs to provide change. She wants to write a program to determine the number of possible ways to achieve the given amount using the available denominations. This will help her understand all possible

combinations without having to manually calculate them each time.

Input Format

The first line contains an integer n, the number of different coin denominations.

The second line contains n space-separated integers representing the coin denominations.

The third line contains an integer m, the target amount of money for which Meredith needs to make change.

Output Format

The output displays the number of ways to make the sum by using different combinations from the coins array.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

1 2 3

4

Output: 4

Answer

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, m;
```

```
    scanf("%d", &n);
```

```
    int coins[n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &coins[i]);
```

```
    }
```

```
    scanf("%d", &m);
```



```

int dp[m + 1];

for (int i = 0; i <= m; i++) {
    dp[i] = 0;
}

dp[0] = 1;

for (int i = 0; i < n; i++) {
    for (int j = coins[i]; j <= m; j++) {
        dp[j] += dp[j - coins[i]];
    }
}

printf("%d", dp[m]);

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

In a small research lab, a programmer named Arin is analyzing binary data to detect system flags stored as set bits in an integer. One day Arin receives an integer and must determine how many bits are set to 1 in its binary representation.

Write a program to ensure that this task is done so Arin can continue validating the system's binary logs. The objective is to count all set bits in the given number using any valid approach.

Function specification: countSetBits(int n)

Input Format

The input consists of a single positive integer n.

Output Format

The output prints a single integer representing the count of set bits (1s) in the binary representation of n.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 21

Output: 3

Answer

```
#include <stdio.h>
```

```
int countSetBits(int n) {  
    int count = 0;
```

```
    while (n > 0) {  
        count += (n & 1);  
        n = n >> 1;  
    }
```

```
    return count;  
}
```

```
int main() {  
    int n;  
    scanf("%d", &n);
```

```
    printf("%d", countSetBits(n));
```

```
    return 0;  
}
```

Status : Correct

Marks : 10/10

5. Problem Statement

Raju is a computer graphics designer. He works around matrices with

ease. His friend challenged him to rotate an $M \times M$ matrix clockwise with an angle of 90 degrees.

However, Raju was given a constraint that he must not use any additional 2D matrix for doing the above-mentioned task. The rotation must be done in place, which means Raju has to modify the given input matrix directly.

Example: 1

Input:

[[1,2,3],[4,5,6],[7,8,9]]

Output:

[[7,4,1],[8,5,2],[9,6,3]]

Example: 2

Input:

matrix = [[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]]

Output:

[[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]]

Input Format

The first line of the input is a single integer value for the size of the row and column, M .

The next M lines of the input are the matrix elements, each containing M integers representing the elements of the matrix.

Output Format

The output displays the rotated matrix, with each row printed on a new line.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 3

1 2 3

4 5 6

7 8 9

Output: 7 4 1

8 5 2

9 6 3

Answer

```
#include <stdio.h>
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int matrix[n][n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < n; j++) {
```

```
            scanf("%d", &matrix[i][j]);
```

```
        }
```

```
    }
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = i + 1; j < n; j++) {
```

```
            int temp = matrix[i][j];
```

```
            matrix[i][j] = matrix[j][i];
```

```
            matrix[j][i] = temp;
```

```
        }
```

```
    }
```

```
    for (int i = 0; i < n; i++) {
```

```
        int left = 0, right = n - 1;
```

```
        while (left < right) {
```

```
            int temp = matrix[i][left];
```

```
            matrix[i][left] = matrix[i][right];
```

```
matrix[i][right] = temp;
left++;
right--;
    }
}
```

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        printf("%d ", matrix[i][j]);
    }
    printf("\n");
}
return 0;
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 2_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 14

Section 1 : MCQ

1. What is a non-recursive algorithm?

Answer

An algorithm that does not use recursion

Status : Correct

Marks : 1/1

2. Which of the following is an example of a non-recursive sorting algorithm?

Answer

Bubble Sort

Status : Correct

Marks : 1/1

3. The time complexity of non-recursive algorithms is often expressed using:

Answer

Big O notation

Status : Wrong

Marks : 0/1

4. Which non-recursive algorithm is commonly used for searching in a sorted array by repeatedly dividing the search interval in half?

Answer

Binary Search

Status : Correct

Marks : 1/1

5. In non-recursive algorithms, iteration is commonly achieved through:

Answer

Loops

Status : Correct

Marks : 1/1

6. What is the output of the code?

```
#include <stdio.h>
```

```
int main() {  
    int num = 2;  
    do {  
        printf("%d ", num);  
        num *= 2;  
    } while (num <= 8);  
    return 0;  
}
```

Answer

2 4 8

Status : Correct

Marks : 1/1

7. What is the output for the code?

```
#include <stdio.h>
```

```
void nonRecursiveAlgorithm(int n) {  
    for (int i = 1; i <= n; i *= 2) {  
        printf("%d ", i);  
    }  
}
```

```
int main() {  
    nonRecursiveAlgorithm(8);  
    return 0;  
}
```

Answer

1 2 4 8

Status : Correct

Marks : 1/1

8. What is the output for the code?

```
#include <stdio.h>
```

```
void nonRecursiveFunction1(int n) {  
    while (n > 0) {  
        printf("%d ", n);  
        n /= 2;  
    }  
}
```

```
int main() {  
    int number = 80;  
    nonRecursiveFunction1(number);  
    return 0;  
}
```



```
}
```

Answer

80 40 20 10 5 2 1

Status : Correct

Marks : 1/1

9. What will be the output for the following recursive code?

```
#include <stdio.h>
```

```
int power(int base, int exponent) {  
    if (exponent == 0)  
        return 1;  
    else  
        return base * power(base, exponent - 1);  
}
```

```
int main() {  
    int result = power(3, 2);  
    printf("%d", result);  
    return 0;  
}
```

Answer

9

Status : Correct

Marks : 1/1

10. What will be the output for the following recursive code?

```
#include <stdio.h>
```

```
int num(int n, int rev) {  
    if (n == 0) {  
        return rev;  
    } else {  
        int digit = n % 10;  
        rev = rev * 10 + digit;  
    }  
}
```

```
        return num(n / 10, rev);  
    }  
}
```

```
int main() {  
    int d = 12345;  
    int rev = num(d, 0);  
    printf("%d", rev);  
    return 0;  
}
```

Answer

54321

Status : Correct

Marks : 1/1

11. What will be the output for the following recursive code?

```
#include <stdio.h>
```

```
int function(int number) {  
    if (number <= 0) {  
        return 1;  
    } else {  
        return number * function(number - 1);  
    }  
}
```

```
int main() {  
    int result;  
    result = function(4);  
    printf("%d", result);  
  
    return 0;  
}
```

Answer

24

Status : Correct

Marks : 1/1

12. What will be the output for the following recursive code?

```
#include <iostream>
using namespace std;
```

```
void print(int n) {
    if (n == 0)
        return;
    print(n / 2);
    cout << n % 2;
}
```

```
int main() {
    int num = 12;
    print(num);
    return 0;
}
```

Answer

1100

Status : Correct

Marks : 1/1

13. What will be the output for the following recursive code?

```
#include <stdio.h>
```

```
int function(int a, int b) {
    if (b == 1)
        return a;
    else
        return a + function(a, b - 1);
}
```

```
int main() {
    int result = function(2, 3);
    printf("%d", result);
    return 0;
}
```

Answer

6

Status : Correct

Marks : 1/1

14. What will be the output for the following recursive code?

```
#include <stdio.h>
```

```
int function(int a, int b) {  
    if (b == 0)  
        return 0;  
    if (b == 1)  
        return a;  
    return a + function(a, b - 1);  
}
```

```
int main() {  
    int result = function(3, 8);  
    printf("%d", result);  
    return 0;  
}
```

Answer

24

Status : Correct

Marks : 1/1

15. What will be the output for the following recursive code?

```
#include <iostream>  
using namespace std;
```

```
int recursive(int n) {  
    if (n <= 0)  
        return 1;  
    else  
        return n + recursive(n - 1);  
}
```

```
}
```

```
int main() {  
    int result = recursive(5);  
    cout << result;  
    return 0;  
}
```

Answer

16

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 3_COD

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

You are given a set of N points on a 2D plane. Your task is to find the distance between the closest pair of points.

Write a program that takes as input the number of points N, followed by the x and y coordinates of each point. The program should output the distance between the closest pair of points, rounded to two decimal places.

Input Format

The first line of input consists of an integer N, representing the number of points.

The following N lines represent the coordinate points x and y, separated by space.

Output Format

The output prints a single floating-point number representing the distance between the closest pair of points, rounded to two decimal places.

Sample Test Case

Input: 3

2 3

12 30

40 50

Output: 28.79

Answer

```
#include <stdio.h>
```

```
#include <math.h>
```

```
int main() {
```

```
    int N;
```

```
    scanf("%d", &N);
```

```
    int x[25], y[25];
```

```
    for (int i = 0; i < N; i++) {
```

```
        scanf("%d %d", &x[i], &y[i]);
```

```
    }
```

```
    double minDist = 1e9;
```

```
    for (int i = 0; i < N; i++) {
```

```
        for (int j = i + 1; j < N; j++) {
```

```
            double dx = x[i] - x[j];
```

```
            double dy = y[i] - y[j];
```

```
            double dist = sqrt(dx * dx + dy * dy);
```

```
            if (dist < minDist) {
```

```
                minDist = dist;
```

```
            }
```

```
        }
```

```
    }
```

```
    printf("%.2lf", minDist);
```

```
    return 0;  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Ishita is working on a mapping application that helps locate the nearest landmarks on a 2D map. Each landmark is represented by its (x, y) coordinate. Her task is to find out the shortest distance between any two landmarks from the given list.

Help Ishita write a program that calculates the distance between the closest pair of points (landmarks) on the map.

Input Format

The first line of input contains an integer N representing the number of landmarks.

The next N lines contain two space-separated integers, representing the x and y coordinates of each landmark.

Output Format

The output prints a single floating-point number representing the distance between any two landmarks rounded to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

5 7

12 32

21 23

43 65

Output: 12.73

Answer


```

#include <stdio.h>
#include <math.h>

int main() {
    int N;
    scanf("%d", &N);

    int x[25], y[25];

    for (int i = 0; i < N; i++) {
        scanf("%d %d", &x[i], &y[i]);
    }

    double minDist = 100000.0;

    for (int i = 0; i < N; i++) {
        for (int j = i + 1; j < N; j++) {
            double dx = x[i] - x[j];
            double dy = y[i] - y[j];
            double dist = sqrt(dx * dx + dy * dy);

            if (dist < minDist) {
                minDist = dist;
            }
        }
    }

    printf("%.2lf", minDist);

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Ragu, an aspiring computer scientist, is currently learning about the concept of finding the closest pair of points in a plane. He wants to test his

skills by solving a programming challenge related to this concept.

Given a set of points in a 2D plane, Ragu needs to find the closest pair of points and calculate their Euclidean distance.

Input Format

The first line of input consists of an integer N , representing the number of points in the plane.

The following N lines, each consisting of two space-separated real numbers x and y , represent the coordinates of a point.

Output Format

The first line of output prints the coordinates of the two points forming the closest pair in the below format:

"Closest pair of points: (x_1 , y_1) and (x_2 , y_2)"

The second line prints "Distance: <distance>" representing the Euclidean distance between the closest pair of points, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

0 3

1 1

10 18

4 9

Output: Closest pair of points: (0, 3) and (1, 1)

Distance: 2.24

Answer

```
#include <stdio.h>
```

```
#include <math.h>
```

```
int main() {
```

```
    int N;
```

```

scanf("%d", &N);

double x[10], y[10];

for (int i = 0; i < N; i++) {
    scanf("%lf %lf", &x[i], &y[i]);
}

double minDist = 100000.0;
int p1 = 0, p2 = 1;

for (int i = 0; i < N; i++) {
    for (int j = i + 1; j < N; j++) {
        double dx = x[i] - x[j];
        double dy = y[i] - y[j];
        double dist = sqrt(dx * dx + dy * dy);

        if (dist < minDist) {
            minDist = dist;
            p1 = i;
            p2 = j;
        }
    }
}

printf("Closest pair of points: (%.0lf, %.0lf) and (%.0lf, %.0lf)\n",
       x[p1], y[p1], x[p2], y[p2]);

printf("Distance: %.2lf", minDist);

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Ragu, a passionate computer science student, is exploring geometry problems to sharpen his algorithmic thinking. Today, he challenges himself to determine the two closest points among a set of 2D coordinates.

Ragu must find the closest pair of points from the given set and compute the Euclidean distance between them with high precision. He also wants the output to display which two points are the closest, formatted clearly.

Input Format

The first line of input contains an integer N representing the number of points in the 2D plane.

The next N lines each contain two space-separated real numbers x and y representing the coordinates of each point.

Output Format

The first line of output prints the coordinates of the two points forming the closest pair in the below format:

"Closest pair of points: (x_1 , y_1) and (x_2 , y_2)"

The second line prints "Distance: <distance>" representing the Euclidean distance between the closest pair of points, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

1 2

3 4

1 0

6 8

2 3

Output: Closest pair of points: (1, 2) and (2, 3)

Distance: 1.41

Answer

```
#include <stdio.h>
```

```
#include <math.h>
```

```
int main() {
```

```

int N;
scanf("%d", &N);

double x[10], y[10];

for (int i = 0; i < N; i++) {
    scanf("%lf %lf", &x[i], &y[i]);
}

double minDist = 100000.0;
int p1 = 0, p2 = 1;

for (int i = 0; i < N; i++) {
    for (int j = i + 1; j < N; j++) {
        double dx = x[i] - x[j];
        double dy = y[i] - y[j];
        double dist = sqrt(dx * dx + dy * dy);

        if (dist < minDist) {
            minDist = dist;
            p1 = i;
            p2 = j;
        }
    }
}

printf("Closest pair of points: (%.0lf, %.0lf) and (%.0lf, %.0lf)\n",
       x[p1], y[p1], x[p2], y[p2]);

printf("Distance: %.2lf", minDist);

return 0;
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 3_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 21

Section 1 : Coding

1. Problem Statement

Adhi is learning about computational geometry and has come across the concept of a Convex Hull.

A Convex Hull is a convex polygon that encompasses a set of points in a two-dimensional plane, such that no point lies within the polygon's interior.

Adhi is particularly interested in determining whether a given point lies inside or outside the convex hull formed by a set of points.

Help Adhi by writing a program that checks whether a given point lies inside or outside the convex hull formed by a set of points.

Example

$N = 7$,

Points: $\{(1, 1), (2, 1), (3, 1), (4, 1), (4, 2), (4, 3), (4, 4)\}$,

Query: $X = 3, Y = 2$

Below is the image of the plotting of the given points:

The query $(3, 2)$ lies inside the Convex Hull.

Input Format

The first line of input consists of an integer N , representing the number of points in the set.

The following N lines, each containing two integers x and y , represent the coordinates of the points in the set.

The last line consists of two integers p_x and p_y , representing the coordinates of the test point.

Output Format

The output prints whether the given test points lie inside or outside the Convex Hull.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7

1 1

2 1

3 1

4 1

4 2

4 3

4 4

3 2

Output: The point (3, 2) lies inside the Convex Hull.

Answer

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int orientation(int x1, int y1, int x2, int y2, int x3, int y3) {  
    int val = (y2 - y1) * (x3 - x2) - (x2 - x1) * (y3 - y2);  
    if (val == 0) return 0;  
    return (val > 0) ? 1 : 2;  
}
```

```
int convexHull(int x[], int y[], int n, int hullX[], int hullY[]) {  
    if (n < 3) return 0;
```

```
    int left = 0;  
    for (int i = 1; i < n; i++)  
        if (x[i] < x[left])  
            left = i;
```

```
    int p = left, q;  
    int hullSize = 0;
```

```
    do {  
        hullX[hullSize] = x[p];  
        hullY[hullSize] = y[p];  
        hullSize++;
```

```
        q = (p + 1) % n;
```

```
        for (int i = 0; i < n; i++) {  
            if (orientation(x[p], y[p], x[i], y[i], x[q], y[q]) == 2)  
                q = i;  
        }
```

```
        p = q;
```

```
    } while (p != left);
```

```
    return hullSize;
```



```
}
```

```
int isInside(int hx[], int hy[], int m, int px, int py) {  
    int prev = 0;
```

```
    for (int i = 0; i < m; i++) {  
        int next = (i + 1) % m;
```

```
        int val = (hy[next] - hy[i]) * (px - hx[next]) -  
                  (hx[next] - hx[i]) * (py - hy[next]);
```

```
        if (val == 0) continue;
```

```
        if (prev == 0)  
            prev = val;
```

```
        else if ((prev > 0 && val < 0) || (prev < 0 && val > 0))  
            return 0; // outside
```

```
    }
```

```
    return 1;
```

```
}
```

```
int main() {
```

```
    int N;  
    scanf("%d", &N);
```

```
    int x[MAX], y[MAX];
```

```
    for (int i = 0; i < N; i++) {  
        scanf("%d %d", &x[i], &y[i]);
```

```
    }
```

```
    int px, py;  
    scanf("%d %d", &px, &py);
```

```
    int hullX[MAX], hullY[MAX];  
    int hullSize = convexHull(x, y, N, hullX, hullY);
```

```
    if (isInside(hullX, hullY, hullSize, px, py))  
        printf("The point (%d, %d) lies inside the Convex Hull.", px, py);  
    else
```

```
printf("The point (%d, %d) lies outside the Convex Hull.", px, py);  
return 0;  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement:

Olivia, a city planner, is designing a recreational area surrounded by specific landmarks. To ensure balance in the design, Olivia needs to calculate the weighted centroid of the boundary formed by these landmarks using the convex hull.

Your task is to compute the convex hull and determine the weighted centroid of its vertices.

Note:

MAXN (100000): The maximum capacity for both the points and hull arrays, allowing storage of up to 100,000 points each. $1e-9$ is the threshold for floating-point comparisons.

Example:

Input:

6

0 0

4 0

4 3

0 3

2 1

3 2

Output:

Weighted Centroid: 2.00 1.50

Explanation:

Input Points: (0, 0), (2, 0), (2, 2), (0, 2), (1, 1), (3, 1).

Pivot Selection: The point with the lowest y-coordinate (and leftmost in case of a tie) is selected as the pivot: (0, 0).

Sorting by Polar Angle: Points are sorted by the polar angle relative to the pivot: (0, 0), (2, 0), (3, 1), (2, 2), (0, 2).

Convex Hull Construction: Using counterclockwise turns, the points forming the convex hull are: (0, 0), (2, 0), (3, 1), (2, 2), (0, 2).

Input Format

The first line contains an integer N, representing the number of points.

The next N lines each contain two floating-point numbers x and y, representing the coordinates of the points.

Output Format

The output prints the coordinates of the weighted centroid of the convex hull, formatted to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

0 0

2 0

2 2

0 2

1 1

Output: Weighted Centroid: 1.00 1.00

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <math.h>
```

```
#define MAXN 100000
```

```
#define EPS 1e-9
```

```
typedef struct {  
    double x, y;  
} Point;
```

```
Point points[MAXN], hull[MAXN];  
Point pivot;
```

```
void swap(Point *a, Point *b) {  
    Point temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

```
double distSq(Point a, Point b) {  
    return (a.x - b.x)*(a.x - b.x) + (a.y - b.y)*(a.y - b.y);  
}
```

```
int orientation(Point a, Point b, Point c) {  
    double val = (b.y - a.y)*(c.x - b.x) - (b.x - a.x)*(c.y - b.y);  
    if (fabs(val) < EPS) return 0;  
    return (val > 0) ? 1 : 2;  
}
```

```
int compare(const void *vp1, const void *vp2) {  
    Point *p1 = (Point *)vp1;  
    Point *p2 = (Point *)vp2;
```

```
    int o = orientation(pivot, *p1, *p2);  
    if (o == 0)  
        return (distSq(pivot, *p1) < distSq(pivot, *p2)) ? -1 : 1;
```

```
    return (o == 2) ? -1 : 1;  
}
```

```
int convexHull(int n) {
```

```
int min = 0;
for (int i = 1; i < n; i++) {
    if (points[i].y < points[min].y ||
        (fabs(points[i].y - points[min].y) < EPS && points[i].x < points[min].x))
        min = i;
}
```

```
swap(&points[0], &points[min]);
pivot = points[0];
```

```
qsort(points + 1, n - 1, sizeof(Point), compare);
```

```
int m = 0;
hull[m++] = points[0];
hull[m++] = points[1];
```

```
for (int i = 2; i < n; i++) {
    while (m > 1 && orientation(hull[m-2], hull[m-1], points[i]) != 2)
        m--;
    hull[m++] = points[i];
}
```

```
return m;
```

```
}
```

```
int main() {
    int N;
    scanf("%d", &N);
```

```
    for (int i = 0; i < N; i++) {
        scanf("%lf %lf", &points[i].x, &points[i].y);
    }
```

```
    int hullSize = convexHull(N);
```

```
    double sumX = 0, sumY = 0;
    for (int i = 0; i < hullSize; i++) {
        sumX += hull[i].x;
        sumY += hull[i].y;
```

```

    }
    double cx = sumX / hullSize;
    double cy = sumY / hullSize;

    printf("Weighted Centroid: %.2lf %.2lf", cx, cy);

    return 0;
}

```

Status : Partially correct

Marks : 6/10

3. Problem Statement:

Gwen, a city planner, is designing a new park and needs to calculate the perimeter of the park's boundary. She has the coordinates of key boundary points and wants to find the minimum perimeter that encloses all these points.

Using the convex hull, Gwen needs help in computing both the boundary and its perimeter.

Note:

points[100000]: An array that can store up to 100,000 input points.
 hull[100000]: An array that can store up to 100,000 points on the convex hull.
 1e-9: A threshold for floating-point comparisons to handle precision issues.

Example

Input:

6

0.0 0.0

3.0 0.0

3.0 2.0

1.0 2.0

0.0 2.0

2.0 1.0

Output:

Convex Hull:

0 0

3 0

3 2

0 2

Perimeter: 10.00

Explanation:

Input Points: (0.0, 0.0), (3.0, 0.0), (3.0, 2.0), (1.0, 2.0), (0.0, 2.0), (2.0, 1.0).

Convex Hull Construction:

Pivot: Start with the lowest point, (0, 0). Sorted by Polar Angle: The points are sorted, and counterclockwise turns are included in the hull. Convex Hull Points: (0, 0), (3, 0), (3, 2), (0, 2). Perimeter Calculation:

Distances between consecutive hull points: (0, 0)-(3, 0): 3.00, (3, 0)-(3, 2): 2.00, (3, 2)-(0, 2): 3.00, (0, 2)-(0, 0): 2.00. Total Perimeter: 10.00.

Input Format

The first line contains an integer N, representing the number of points.

The next N lines each contain two space separated decimal values x and y, representing the coordinates of the points.

Output Format

The first line prints "Convex Hull:" followed by the coordinates of the points on the convex hull, each on a new line, in counterclockwise order.

The last line prints the "Perimeter: ", followed by the total perimeter of the convex hull, formatted to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

0.0 0.0

3.0 0.0

3.0 2.0

1.0 2.0

0.0 2.0

2.0 1.0

Output: Convex Hull:

0 0

3 0

3 2

0 2

Perimeter: 10.00

Answer

```
#include <stdio.h>
```

```
#include <math.h>
```

```
#define MAX 100
```

```
#define EPS 1e-9
```

```
typedef struct {  
    double x, y;  
} Point;
```

```
Point points[MAX], hull[MAX];
```

```
int orientation(Point a, Point b, Point c) {  
    double val = (b.y - a.y) * (c.x - b.x) -  
                (b.x - a.x) * (c.y - b.y);
```

```
    if (fabs(val) < EPS) return 0;
```

```
    return (val > 0) ? 1 : 2;
```

```
}
```

```
double distance(Point a, Point b) {  
    return sqrt((a.x - b.x)*(a.x - b.x) +
```



```
        (a.y - b.y)*(a.y - b.y));  
    }
```

```
int convexHull(int n) {  
    if (n < 3) return 0;  
  
    int left = 0;  
    for (int i = 1; i < n; i++) {  
        if (points[i].x < points[left].x)  
            left = i;  
    }
```

```
    int p = left, q;  
    int hullSize = 0;
```

```
    do {  
        hull[hullSize++] = points[p];  
        q = (p + 1) % n;  
  
        for (int i = 0; i < n; i++) {  
            if (orientation(points[p], points[i], points[q]) == 2)  
                q = i;  
        }
```

```
        p = q;
```

```
    } while (p != left);
```

```
    return hullSize;
```

```
}
```

```
int main() {  
    int N;  
    scanf("%d", &N);
```

```
    for (int i = 0; i < N; i++) {  
        scanf("%lf %lf", &points[i].x, &points[i].y);  
    }
```

```
    int hullSize = convexHull(N);
```

```

printf("convex Hull:\n");
for (int i = 0; i < hullSize; i++) {
    printf("%.0lf %.0lf\n", hull[i].x, hull[i].y);
}

```

```

double perimeter = 0.0;
for (int i = 0; i < hullSize; i++) {
    int next = (i + 1) % hullSize;
    perimeter += distance(hull[i], hull[next]);
}

```

```

printf("Perimeter: %.2lf", perimeter);

```

```

return 0;
}

```

Status : Partially correct

Marks : 5/10

4. Problem Statement:

Maria, a wildlife researcher, is studying the distribution of endangered species in a forest. She has mapped several points that represent observation locations inside the forest.

Maria wants to identify the K points that are the farthest from the forest's outer boundary, represented by a convex hull formed by a set of points. These points represent key observations, and she needs to rank them by their distance to the outer boundary of the forest.

Help Maria compute the convex hull and find the K farthest points from the forest boundary.

Note:

MAXN (100000): Maximum number of points for points and hull arrays. 1e-9: Threshold for floating-point equality checks to handle precision errors. 1e-12: Threshold to detect degenerate (zero-length) segments. 1e30: Initial large value for minimum distance calculations.

Example:

Input:

7 3

0 0

4 0

4 4

0 4

2 2

1 2

3 2

Output:

2 2

3 2

1 2

Explanation:

Convex Hull Points: Using Graham's Scan, the convex hull is: (0, 0), (4, 0), (4, 4), (0, 4).

Distance to Hull: Distances of all points to the convex hull boundary are computed.

Top 3 Farthest Points: Sorted by descending distance, the top 3 points are (2, 2), (3, 2), (1, 2).

Input Format

The first line contains two integers N and K, representing the number of points and the number of farthest points to find, respectively.

The next N lines each contain two integers x and y, representing the coordinates of the points.

Output Format

The output prints K lines, each containing two integers x and y, representing the coordinates of the K farthest points from the convex hull boundary. Print the points in descending order of their distances to the boundary.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7 3

0 0

4 0

4 4

0 4

2 2

1 2

3 2

Output: 2 2

3 2

1 2

Answer

```
#include <stdio.h>
```

```
#include <math.h>
```

```
#define MAX 20
```

```
#define EPS 1e-9
```

```
typedef struct {  
    int x, y;  
} Point;
```

```
typedef struct {  
    Point p;  
    double dist;  
} Result;
```

```
Point points[MAX], hull[MAX];
```

```
Result res[MAX];
```

```
int orientation(Point a, Point b, Point c) {
```

```

    int val = (b.y - a.y)*(c.x - b.x) - (b.x - a.x)*(c.y - b.y);
    if (val == 0) return 0;
    return (val > 0) ? 1 : 2;
}

```

```

double distance(Point a, Point b) {
    return sqrt((a.x - b.x)*(a.x - b.x) +
        (a.y - b.y)*(a.y - b.y));
}

```

```

double pointToLine(Point A, Point B, Point P) {
    double A1 = B.y - A.y;
    double B1 = A.x - B.x;
    double C1 = B.x*A.y - A.x*B.y;

    return fabs(A1*P.x + B1*P.y + C1) /
        sqrt(A1*A1 + B1*B1);
}

```

```

int convexHull(int n) {
    if (n < 3) return 0;

    int left = 0;
    for (int i = 1; i < n; i++)
        if (points[i].x < points[left].x)
            left = i;

    int p = left, q;
    int size = 0;

    do {
        hull[size++] = points[p];
        q = (p + 1) % n;

        for (int i = 0; i < n; i++) {
            if (orientation(points[p], points[i], points[q]) == 2)
                q = i;
        }
    }
}

```

```

        p = q;
    } while (p != left);

    return size;
}

```

```

int isHullPoint(Point p, int hSize) {
    for (int i = 0; i < hSize; i++) {
        if (hull[i].x == p.x && hull[i].y == p.y)
            return 1;
    }
    return 0;
}

```

```

int main() {
    int N, K;
    scanf("%d %d", &N, &K);

    for (int i = 0; i < N; i++) {
        scanf("%d %d", &points[i].x, &points[i].y);
    }
}

```

```

int hSize = convexHull(N);

```

```

int count = 0;

```

```

for (int i = 0; i < N; i++) {
    if (isHullPoint(points[i], hSize)) continue;
}

```

```

double minDist = 1e30;

```

```

for (int j = 0; j < hSize; j++) {
    int next = (j + 1) % hSize;
    double d = pointToLine(hull[j], hull[next], points[i]);
    if (d < minDist)
        minDist = d;
}

```

```

res[count].p = points[i];

```

```

        res[count].dist = minDist;
        count++;
    }

    for (int i = 0; i < count - 1; i++) {
        for (int j = i + 1; j < count; j++) {
            if (res[i].dist < res[j].dist) {
                Result temp = res[i];
                res[i] = res[j];
                res[j] = temp;
            }
        }
    }

    for (int i = 0; i < K && i < count; i++) {
        printf("%d %d\n", res[i].p.x, res[i].p.y);
    }

    return 0;
}

```

Status : Wrong

Marks : 0/10

5. Problem Statement:

Henry, a wildlife researcher, is studying the distribution of endangered species in a forest. He has mapped several points that represent observation locations inside the forest.

Henry wants to identify the K points that are the farthest from the forest's outer boundary, represented by a convex hull formed by a set of points. These points represent key observations, and he needs to rank them by their distance to the outer boundary of the forest.

Help Henry compute the convex hull and find the K farthest points from the forest boundary.

Note:

MAXN (100000): Maximum number of points for points and hull arrays. $1e-9$: Threshold for floating-point equality checks to handle precision errors. $1e-12$: Threshold to detect degenerate (zero-length) segments. $1e30$: Initial large value for minimum distance calculations.

Example:

Input:

7 3

0 0

4 0

4 4

0 4

2 2

1 2

3 2

Output:

2 2

3 2

1 2

Explanation:

Convex Hull Points: Using Graham's Scan, the convex hull is: (0, 0), (4, 0), (4, 4), (0, 4).

Distance to Hull: Distances of all points to the convex hull boundary are computed.

Top 3 Farthest Points: Sorted by descending distance, the top 3 points are (2, 2), (3, 2), (1, 2).

Input Format

The first line contains two integers N and K, representing the number of points and the number of farthest points to find, respectively.

The next N lines each contain two integers x and y, representing the coordinates of the points.

Output Format

The output prints K lines, each containing two integers x and y, representing the coordinates of the K farthest points from the convex hull boundary. Print the points in descending order of their distances to the boundary.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7 3

0 0

4 0

4 4

0 4

2 2

1 2

3 2

Output: 2 2

3 2

1 2

Answer

```
#include <stdio.h>
```

```
#include <math.h>
```

```
#define MAX 20
```

```
#define INF 1e30
```

```
#define EPS 1e-9
```

```
typedef struct {  
    int x, y;  
} Point;
```

```
typedef struct {  
    Point p;  
    double dist;
```

```
    int index;  
} Result;
```

```
Point points[MAX], hull[MAX];  
Result res[MAX];
```

```
int orientation(Point a, Point b, Point c) {  
    int val = (b.y - a.y)*(c.x - b.x) - (b.x - a.x)*(c.y - b.y);  
    if (val == 0) return 0;  
    return (val > 0) ? 1 : 2;  
}
```

```
double dist(Point a, Point b) {  
    return sqrt((a.x - b.x)*(a.x - b.x) +  
                (a.y - b.y)*(a.y - b.y));  
}
```

```
double pointToSegment(Point A, Point B, Point P) {  
    double dx = B.x - A.x;  
    double dy = B.y - A.y;  
  
    if (fabs(dx) < EPS && fabs(dy) < EPS)  
        return dist(A, P);  
  
    double t = ((P.x - A.x)*dx + (P.y - A.y)*dy) / (dx*dx + dy*dy);  
  
    if (t < 0.0)  
        return dist(P, A);  
    else if (t > 1.0)  
        return dist(P, B);  
  
    double projX = A.x + t * dx;  
    double projY = A.y + t * dy;  
  
    return sqrt((P.x - projX)*(P.x - projX) +  
                (P.y - projY)*(P.y - projY));  
}
```

```
int convexHull(int n) {
```

```

    if (n < 3) return 0;

    int left = 0;
    for (int i = 1; i < n; i++)
        if (points[i].x < points[left].x)
            left = i;

    int p = left, q, size = 0;

    do {
        hull[size++] = points[p];
        q = (p + 1) % n;

        for (int i = 0; i < n; i++) {
            if (orientation(points[p], points[i], points[q]) == 2)
                q = i;
        }

        p = q;

    } while (p != left);

    return size;
}

```

```

int isHull(Point p, int hSize) {
    for (int i = 0; i < hSize; i++)
        if (hull[i].x == p.x && hull[i].y == p.y)
            return 1;
    return 0;
}

```

```

int main() {
    int N, K;
    scanf("%d %d", &N, &K);

    for (int i = 0; i < N; i++)
        scanf("%d %d", &points[i].x, &points[i].y);

    int hSize = convexHull(N);
}

```

```
int count = 0;
```

```
for (int i = 0; i < N; i++) {  
    if (isHull(points[i], hSize)) continue;
```

```
    double minDist = INF;
```

```
    for (int j = 0; j < hSize; j++) {  
        int next = (j + 1) % hSize;  
        double d = pointToSegment(hull[j], hull[next], points[i]);  
        if (d < minDist)  
            minDist = d;  
    }
```

```
    res[count].p = points[i];  
    res[count].dist = minDist;  
    res[count].index = i;  
    count++;  
}
```

```
for (int i = 0; i < count - 1; i++) {  
    for (int j = i + 1; j < count; j++) {  
        if (res[i].dist < res[j].dist ||  
            (fabs(res[i].dist - res[j].dist) < EPS &&  
             res[i].index > res[j].index)) {  
            Result temp = res[i];  
            res[i] = res[j];  
            res[j] = temp;  
        }  
    }  
}
```

```
for (int i = 0; i < K && i < count; i++) {  
    printf("%d %d\n", res[i].p.x, res[i].p.y);  
}
```

```
return 0;
```

```
}
```

Status : Wrong

Marks : 0/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 3_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 13

Section 1 : MCQ

1. What is the primary characteristic of the Brute Force approach to solving the Closest-Pair problem?

Answer

Computing the distance between every pair of points and selecting the minimum

Status : Correct

Marks : 1/1

2. In the brute force convex hull algorithm (Jarvis March/Gift Wrapping), what is the next point selected after identifying a point on the hull?

Answer

The point that creates the smallest counterclockwise turn relative to the current point

Status : Correct

Marks : 1/1

3. In C, if you have a structure `Point { int x; int y; }`, what is the correct formula to calculate the squared distance between two points `p1` and `p2` to avoid unnecessary floating-point operations?

Answer

`(p1.x - p2.x)*(p1.x - p2.x) + (p1.y - p2.y)*(p1.y - p2.y)`

Status : Correct

Marks : 1/1

4. Given an array of `n` points in a 2D plane, what is the exact time complexity of the Brute Force Closest-Pair algorithm?

Answer

$O(n^2)$

Status : Correct

Marks : 1/1

5. Which geometric test is essential for determining if a point lies to the left or right of a directed line in the Convex Hull brute force algorithm?

Answer

The Orientation test (Cross Product)

Status : Correct

Marks : 1/1

6. If two points are exactly the same in the input set for the Closest Pair problem, what will the brute force algorithm correctly return?

Answer

An error

Status : Wrong

Marks : 0/1

7. In the brute force algorithm to check if a set of points is in a convex

position, the sign of the cross product indicates:

Answer

The direction of the turn (Left or Right)

Status : Correct

Marks : 1/1

8. Given 5 points in C where all x-coordinates are identical, what is the result of the brute force closest pair algorithm?

Answer

The distance between the two points with the closest y-coordinates

Status : Correct

Marks : 1/1

9. What is the worst-case time complexity of the Brute Force algorithm for checking if given points form a convex polygon?

Answer

$O(n^2)$

Status : Correct

Marks : 1/1

10. In a C program solving the Closest Pair problem via brute force, which nested loop structure correctly avoids redundant calculations (i.e., doesn't calculate distance between A,B and B,A)?

Answer

`for(i=0; i`

Status : Wrong

Marks : 0/1

11. When implementing the Brute Force Closest Pair algorithm in C, what is the minimal number of distance calculations required if there are exactly 3 points?

Answer

3

Status : Correct

Marks : 1/1

12. In the brute force method for finding the convex hull (checking all edges), an edge (i, j) is part of the convex hull if:

Answer

All other points lie on one side of the line through i and j

Status : Correct

Marks : 1/1

13. Which of the following best describes "collinearity" in the context of the Convex Hull problem when using brute force?

Answer

Points lying exactly on the hull edge

Status : Correct

Marks : 1/1

14. What is a significant disadvantage of the Brute Force Convex Hull algorithm compared to Divide and Conquer algorithms?

Answer

It is $O(n^2)$ in the worst case and $O(n)$ in the best case (for Jarvis March)

Status : Correct

Marks : 1/1

15. Which of the following correctly represents the Euclidean distance between two points (x1,y1) and (x2,y2) in C?

Answer

``hypot(x2 - x1, y2 - y1)``

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 3_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 48.5

Section 1 : Coding

1. Problem Statement:

Javier, a landscape architect, is designing a new botanical garden and needs to determine the outer boundary that will enclose all the selected plant species.

He has gathered coordinates of key points where the plants are located and needs to calculate the convex hull to create a protective garden boundary. To determine this boundary, Javier decides to use Graham's Scan algorithm.

Help Javier compute the convex hull by implementing the algorithm to design the garden's boundary.

Input Format

The first line of input consists of an integer n , representing the number of plant species locations.

The next n lines contain two integers x and y , representing the coordinates of each location.

Output Format

The output prints "Convex Hull:" followed by the coordinates of the points forming the convex hull in counterclockwise order, with each point on a new line.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

0 0

0 1

1 0

1 1

Output: Convex Hull:

(0, 0)

(1, 0)

(1, 1)

(0, 1)

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define MAX 20
```

```
typedef struct {
```

```
    int x, y;
```

```
} Point;
```

```
Point points[MAX], hull[MAX];
```

```
Point pivot;
```

```
void swap(Point *a, Point *b) {
```

```
    Point temp = *a;
    *a = *b;
    *b = temp;
}
```

```
int orientation(Point a, Point b, Point c) {
    int val = (b.y - a.y)*(c.x - b.x) - (b.x - a.x)*(c.y - b.y);
    if (val == 0) return 0;
    return (val > 0) ? 1 : 2;
}
```

```
int distSq(Point a, Point b) {
    return (a.x - b.x)*(a.x - b.x) +
           (a.y - b.y)*(a.y - b.y);
}
```

```
int compare(const void *vp1, const void *vp2) {
    Point *p1 = (Point *)vp1;
    Point *p2 = (Point *)vp2;

    int o = orientation(pivot, *p1, *p2);

    if (o == 0)
        return distSq(pivot, *p1) - distSq(pivot, *p2);

    return (o == 2) ? -1 : 1;
}
```

```
int convexHull(int n) {
    if (n < 3) return n;
```

```
    int min = 0;
    for (int i = 1; i < n; i++) {
        if (points[i].y < points[min].y ||
            (points[i].y == points[min].y && points[i].x < points[min].x))
            min = i;
    }
```

```
    swap(&points[0], &points[min]);
```

```

    pivot = points[0];

    qsort(points + 1, n - 1, sizeof(Point), compare);

    int m = 0;
    hull[m++] = points[0];
    hull[m++] = points[1];

    for (int i = 2; i < n; i++) {
        while (m > 1 && orientation(hull[m-2], hull[m-1], points[i]) != 2)
            m--;
        hull[m++] = points[i];
    }

    return m;
}

int main() {
    int n;
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d %d", &points[i].x, &points[i].y);
    }

    int size = convexHull(n);

    printf("Convex Hull:\n");
    for (int i = 0; i < size; i++) {
        printf("(%d, %d)\n", hull[i].x, hull[i].y);
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement:

Eva, a botanist, is surveying a nature reserve to protect rare plant species. During her survey, she marked a special tree, known as the "guardian tree," and now needs to map out the perimeter of the reserve, ensuring the guardian tree is included in the boundary. To do this, Eva needs to compute the convex hull of all the points, including the coordinates of the guardian tree.

Your task is to compute the convex hull, ensuring that the special guardian tree is included in the boundary.

Example

Input:

5

0.0 0.0

2.0 0.0

2.0 2.0

0.0 2.0

1.0 1.0

3.0 1.0

Output:

0 0

2 0

3 1

2 2

0 2

Explanation:

Input Points: (0.0, 0.0), (2.0, 0.0), (2.0, 2.0), (0.0, 2.0), (1.0, 1.0), (3.0, 1.0).

Pivot Selection: The point with the lowest y-coordinate (and leftmost in case of a tie) is selected as the pivot: (0, 0).

Sorting by Polar Angle: Points are sorted by the polar angle relative to the pivot: (0, 0), (2, 0), (3, 1), (2, 2), (0, 2).

Convex Hull Construction: Using counterclockwise turns, the points forming the convex hull are: (0, 0), (2, 0), (3, 1), (2, 2), (0, 2).

Input Format

The first line contains an integer N, representing the number of trees originally surveyed.

The next N lines each contain two double values x and y, representing the coordinates of the surveyed trees.

The last line contains two double values x and y, representing the coordinates of the guardian tree.

Output Format

The output prints the coordinates of the points forming the convex hull in counterclockwise order, ensuring the special guardian tree is included. Each coordinate should be printed on a new line, with the values of x and y separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

0.0 0.0

2.0 0.0

2.0 2.0

0.0 2.0

1.0 1.0

3.0 1.0

Output: 0 0

2 0

3 1

2 2

0 2

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
```

```
#define MAX 25
```

```
typedef struct {
    double x, y;
} Point;
```

```
Point points[MAX], hull[MAX], pivot;
```

```
void swap(Point *a, Point *b) {
    Point temp = *a;
    *a = *b;
    *b = temp;
}
```

```
int orientation(Point p, Point q, Point r) {
    double val = (q.y - p.y)*(r.x - q.x) - (q.x - p.x)*(r.y - q.y);
    if (fabs(val) < 1e-9) return 0;
    return (val > 0) ? 1 : 2;
}
```

```
double distSq(Point a, Point b) {
    return (a.x - b.x)*(a.x - b.x) + (a.y - b.y)*(a.y - b.y);
}
```

```
int compare(const void *vp1, const void *vp2) {
    Point *p1 = (Point *)vp1;
    Point *p2 = (Point *)vp2;
    int o = orientation(pivot, *p1, *p2);
    if (o == 0)
        return (distSq(pivot, *p1) - distSq(pivot, *p2) < 0) ? -1 : 1;
    return (o == 2) ? -1 : 1;
}
```

```
int convexHull(int n) {
```



```

int min = 0;
for (int i = 1; i < n; i++) {
    if (points[i].y < points[min].y ||
        (fabs(points[i].y - points[min].y) < 1e-9 && points[i].x < points[min].x))
        min = i;
}
swap(&points[0], &points[min]);
pivot = points[0];

```

```

qsort(points + 1, n - 1, sizeof(Point), compare);

```

```

int m = 0;
hull[m++] = points[0];
hull[m++] = points[1];

for (int i = 2; i < n; i++) {
    while (m > 1 && orientation(hull[m-2], hull[m-1], points[i]) != 2)
        m--;
    hull[m++] = points[i];
}

return m;
}

```

```

int main() {
    int n;
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        scanf("%lf %lf", &points[i].x, &points[i].y);

```

```

    Point guardian;
    scanf("%lf %lf", &guardian.x, &guardian.y);
    points[n++] = guardian;

```

```

    int hullSize = convexHull(n);

```

```

    for (int i = 0; i < hullSize; i++)
        printf("%.0lf %.0lf\n", hull[i].x, hull[i].y);

```

```
    return 0;  
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Nathan is learning about computational geometry and has come across the concept of a Convex Hull.

A Convex Hull is a convex polygon that encompasses a set of points in a two-dimensional plane, such that no point lies within the polygon's interior.

Nathan is particularly interested in determining whether a given point lies inside or outside the convex hull formed by a set of points.

Help Nathan by writing a program that checks whether a given point lies inside or outside the convex hull formed by a set of points.

Example

N = 7,

Points: {(1, 1), (2, 1), (3, 1), (4, 1), (4, 2), (4, 3), (4, 4)},

Query: X = 3, Y = 2

Below is the image of the plotting of the given points:

The query (3, 2) lies inside the Convex Hull.

Input Format

The first line of input consists of an integer N, representing the number of points in the set.

The following N lines, each containing two integers x and y, represent the coordinates of the points in the set.

The last line consists of two integers px and py, representing the coordinates of the test point.

Output Format

The output prints whether the given test points lie inside or outside the Convex Hull.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7

1 1

2 1

3 1

4 1

4 2

4 3

4 4

3 2

Output: The point (3, 2) lies inside the Convex Hull.

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <math.h>
```

```
#define MAX 10
```

```
typedef struct {
```

```
    int x, y;
```

```
} Point;
```

```
Point points[MAX], hull[MAX], pivot;
```

```
void swap(Point *a, Point *b) {
```

```
    Point temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
int orientation(Point p, Point q, Point r) {  
    int val = (q.y - p.y)*(r.x - q.x) - (q.x - p.x)*(r.y - q.y);  
    if (val == 0) return 0;  
    return (val > 0) ? 1 : 2;  
}
```

```
int distSq(Point a, Point b) {  
    return (a.x - b.x)*(a.x - b.x) + (a.y - b.y)*(a.y - b.y);  
}
```

```
int compare(const void *vp1, const void *vp2) {  
    Point *p1 = (Point*)vp1;  
    Point *p2 = (Point*)vp2;  
    int o = orientation(pivot, *p1, *p2);  
    if (o == 0)  
        return distSq(pivot, *p1) - distSq(pivot, *p2);  
    return (o == 2) ? -1 : 1;  
}
```

```
int convexHull(int n) {  
    int min = 0;  
    for (int i = 1; i < n; i++) {  
        if (points[i].y < points[min].y ||  
            (points[i].y == points[min].y && points[i].x < points[min].x))  
            min = i;  
    }  
    swap(&points[0], &points[min]);  
    pivot = points[0];
```

```
    qsort(points + 1, n - 1, sizeof(Point), compare);
```

```
    int m = 0;  
    hull[m++] = points[0];  
    hull[m++] = points[1];
```

```
    for (int i = 2; i < n; i++) {  
        while (m > 1 && orientation(hull[m-2], hull[m-1], points[i]) != 2)  
            m--;  
        hull[m++] = points[i];  
    }
```

```
    return m;
}
```

```
int isInside(int n, Point p) {
    int left = 0, right = 0;
    for (int i = 0; i < n; i++) {
        Point a = hull[i];
        Point b = hull[(i+1)%n];
        int o = orientation(a, b, p);
        if (o == 1) left = 1;
        if (o == 2) right = 1;
        if (left && right) return 0;
    }
    return 1;
}
```

```
int main() {
    int n;
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        scanf("%d %d", &points[i].x, &points[i].y);

    int px, py;
    scanf("%d %d", &px, &py);
    Point query = {px, py};

    int hullSize = convexHull(n);

    if (isInside(hullSize, query))
        printf("The point (%d, %d) lies inside the Convex Hull.\n", px, py);
    else
        printf("The point (%d, %d) lies outside the Convex Hull.\n", px, py);

    return 0;
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Ragu, an aspiring computer scientist, is currently learning about the concept of finding the closest pair of points in a plane. He wants to test his skills by solving a programming challenge related to this concept.

Given a set of points in a 2D plane, Ragu needs to find the closest pair of points and calculate their Euclidean distance.

Input Format

The first line of input consists of an integer N , representing the number of points in the plane.

The following N lines, each consisting of two space-separated real numbers x and y , represent the coordinates of a point.

Output Format

The first line of output prints the coordinates of the two points forming the closest pair in the below format:

"Closest pair of points: (x_1 , y_1) and (x_2 , y_2)"

The second line prints "Distance: <distance>" representing the Euclidean distance between the closest pair of points, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

0 3

1 1

10 18

4 9

Output: Closest pair of points: (0, 3) and (1, 1)

Distance: 2.24

Answer

```
#include <stdio.h>
```

```
#include <math.h>
```

```
typedef struct {  
    double x, y;  
} Point;
```

```
double distance(Point a, Point b) {  
    return sqrt((a.x - b.x)*(a.x - b.x) + (a.y - b.y)*(a.y - b.y));  
}
```

```
int main() {  
    int N;  
    scanf("%d", &N);
```

```
    Point points[N];  
    for(int i = 0; i < N; i++) {  
        scanf("%lf %lf", &points[i].x, &points[i].y);  
    }
```

```
    double minDist = 1e9;  
    Point p1, p2;
```

```
    for(int i = 0; i < N; i++) {  
        for(int j = i+1; j < N; j++) {  
            double d = distance(points[i], points[j]);  
            if(d < minDist) {  
                minDist = d;  
                p1 = points[i];  
                p2 = points[j];  
            }  
        }  
    }
```

```
    printf("Closest pair of points: (%.0lf, %.0lf) and (%.0lf, %.0lf)\n", p1.x, p1.y, p2.x,  
p2.y);  
    printf("Distance: %.2lf\n", minDist);
```

```
    return 0;  
}
```

Status : Correct

Marks : 10/10

5. Problem Statement:

Lena, a geologist, is studying the boundaries of a newly discovered forest reserve. To define the outer edges of the reserve, Lena needs to identify the convex hull from a set of points marking different trees within the area.

Using Graham's Scan algorithm, help Lena compute the convex hull to outline the forest reserve's perimeter.

Example1

Input:

6

0 0

1 1

2 2

4 4

0 4

4 0

Output:

Convex Hull:

(0, 0)

(4, 0)

(4, 4)

(0, 4)

Explanation:

Pivot Selection: The bottom-most point is (0, 0).

Sorting by Polar Angle: Points sorted as (0, 0), (4, 0), (4, 4), (0, 4), (1, 1), (2, 2).

Convex Hull Construction: Start with (0, 0), and include points forming

counterclockwise turns.

The final hull is: (0, 0), (4, 0), (4, 4), (0, 4).

Example2

Input:

3

0 0

0 3

3 0

Output:

Convex Hull:

(0, 0)

(3, 0)

(0, 3)

Explanation:

Pivot Selection: The bottom-most point is (0, 0).

Sorting by Polar Angle: Points sorted as (0, 0), (3, 0), (0, 3).

Convex Hull Construction: Start with (0, 0), and include points forming counterclockwise turns.

The final hull is: (0, 0), (3, 0), (0, 3).

Input Format

The first line of input contains an integer n , representing the number of trees in the forest reserve.

The next n lines contain two integers x and y , representing the coordinates of each tree.

Output Format

The output prints "Convex Hull:" followed by the coordinates of the trees forming

the convex hull in counterclockwise order in new line.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

0 0

1 1

2 2

4 4

0 4

4 0

Output: Convex Hull:

(0, 0)

(4, 0)

(4, 4)

(0, 4)

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct {
```

```
    int x, y;
```

```
} Point;
```

```
Point pivot;
```

```
int orientation(Point p, Point q, Point r) {
```

```
    int val = (q.y - p.y)*(r.x - q.x) - (q.x - p.x)*(r.y - q.y);
```

```
    if(val == 0) return 0;
```

```
    return (val > 0)? 1: 2;
```

```
}
```

```
void swap(Point *a, Point *b) {
```

```
    Point t = *a;
```

```
    *a = *b;
```

```
    *b = t;
```

```
}
```

```
int distSq(Point p1, Point p2) {  
    return (p1.x - p2.x)*(p1.x - p2.x) + (p1.y - p2.y)*(p1.y - p2.y);  
}
```

```
int compare(const void *vp1, const void *vp2) {  
    Point *p1 = (Point *)vp1;  
    Point *p2 = (Point *)vp2;  
  
    int o = orientation(pivot, *p1, *p2);  
    if(o == 0)  
        return (distSq(pivot, *p2) >= distSq(pivot, *p1))? -1 : 1;  
    return (o == 2)? -1 : 1;  
}
```

```
void convexHull(Point points[], int n) {
```

```
    int ymin = points[0].y, min = 0;  
    for(int i = 1; i < n; i++) {  
        int y = points[i].y;  
        if(y < ymin || (y == ymin && points[i].x < points[min].x)) {  
            ymin = points[i].y;  
            min = i;  
        }  
    }  
    swap(&points[0], &points[min]);  
    pivot = points[0];
```

```
    qsort(&points[1], n-1, sizeof(Point), compare);
```

```
    int m = 1;  
    for(int i = 1; i < n; i++) {  
        while(i < n-1 && orientation(pivot, points[i], points[i+1]) == 0)  
            i++;  
        points[m] = points[i];  
        m++;  
    }
```

```

    }

    if(m < 3) {
        printf("Convex Hull:\n");
        for(int i = 0; i < m; i++)
            printf("(%d, %d)\n", points[i].x, points[i].y);
        return;
    }

    Point stack[m];
    int top = 2;
    stack[0] = points[0];
    stack[1] = points[1];
    stack[2] = points[2];

    for(int i = 3; i < m; i++) {
        while(top >= 1 && orientation(stack[top-1], stack[top], points[i]) != 2)
            top--;
        stack[++top] = points[i];
    }

    printf("Convex Hull:\n");
    for(int i = 0; i <= top; i++)
        printf("(%d, %d)\n", stack[i].x, stack[i].y);
}

int main() {
    int n;
    scanf("%d", &n);
    Point points[n];
    for(int i = 0; i < n; i++) {
        scanf("%d %d", &points[i].x, &points[i].y);
    }

    convexHull(points, n);
    return 0;
}

```

Status : Partially correct

Marks : 8.5/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 4_COD

Attempt : 1

Total Mark : 40

Marks Obtained : 24

Section 1 : Coding

1. Problem Statement

You are a delivery manager who needs to plan the delivery route for a fleet of trucks to a set of distribution centers in a city. Each distribution center is at a different location, and the cost of traveling between centers is known.

Your goal is to find the shortest route that allows a truck to visit each distribution center exactly once and return to the starting point, using the Traveling Salesman Problem (TSP). Write a program to calculate the minimum cost of this route and output the optimal path.

Note: Solve using Branch and Bound approach.

Input Format

The first line contains an integer n, the number of distribution centers.

The next n lines contain n space-separated integers each, representing the adjacency matrix.

The ith integer on the jth line represents the cost of traveling from distribution center i to distribution center j.

Output Format

The first line prints:

Minimum Cost is: X

The second line prints:

Optimal Path: followed by the sequence of visited centers ending at the start node

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 3 9 5

3 0 6 4

9 6 0 7

5 4 7 0

Output: Minimum Cost is: 21

Optimal Path: 0 1 2 3 0

Answer

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
int n;
```

```
int adj[15][15];
int final_path[15];
int visited[15];
int final_res = INT_MAX;
```

```
// Copy temporary path to final path
```

```
void copyToFinal(int curr_path[]) {
    for (int i = 0; i < n; i++)
        final_path[i] = curr_path[i];
    final_path[n] = curr_path[0];
}
```

```
// Find minimum edge cost
```

```
int firstMin(int i) {
    int min = INT_MAX;
    for (int k = 0; k < n; k++) {
        if (adj[i][k] && adj[i][k] < min && i != k)
            min = adj[i][k];
    }
    return min;
}
```

```
// Find second minimum edge cost
```

```
int secondMin(int i) {
    int first = INT_MAX, second = INT_MAX;
    for (int j = 0; j < n; j++) {
        if (i == j) continue;
        if (adj[i][j] <= first) {
            second = first;
            first = adj[i][j];
        } else if (adj[i][j] <= second && adj[i][j] != first) {
            second = adj[i][j];
        }
    }
    return second;
}
```

```
// Recursive function for branch and bound
```

```
void TSPRec(int curr_bound, int curr_weight, int level, int curr_path[]) {
    if (level == n) {
        if (adj[curr_path[level - 1]][curr_path[0]] != 0) {
            int curr_res = curr_weight + adj[curr_path[level - 1]][curr_path[0]];

```

```

        if (curr_res < final_res) {
            copyToFinal(curr_path);
            final_res = curr_res;
        }
    }
    return;
}

for (int i = 0; i < n; i++) {
    if (adj[curr_path[level - 1]][i] != 0 && visited[i] == 0) {
        int temp = curr_bound;
        curr_weight += adj[curr_path[level - 1]][i];

        if (level == 1)
            curr_bound -= ((firstMin(curr_path[level - 1]) + firstMin(i)) / 2);
        else
            curr_bound -= ((secondMin(curr_path[level - 1]) + firstMin(i)) / 2);

        if (curr_bound + curr_weight < final_res) {
            curr_path[level] = i;
            visited[i] = 1;
            TSPRec(curr_bound, curr_weight, level + 1, curr_path);
        }

        curr_weight -= adj[curr_path[level - 1]][i];
        curr_bound = temp;

        for (int j = 0; j < n; j++)
            visited[j] = 0;
        for (int j = 0; j <= level - 1; j++)
            visited[curr_path[j]] = 1;
    }
}
}

```

// Solve TSP using Branch and Bound

```

void TSP() {
    int curr_path[n + 1];
    int curr_bound = 0;

    for (int i = 0; i < n; i++)
        curr_bound += (firstMin(i) + secondMin(i));
}

```



```

curr_bound = (curr_bound & 1) ? curr_bound / 2 + 1 : curr_bound / 2;

visited[0] = 1;
curr_path[0] = 0;

TSPRec(curr_bound, 0, 1, curr_path);
}

int main() {
    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &adj[i][j]);

    TSP();

    printf("Minimum Cost is: %d\n", final_res);
    printf("Optimal Path: ");
    for (int i = 0; i <= n; i++)
        printf("%d ", final_path[i]);
    printf("\n");

    return 0;
}

```

Status : Partially correct

Marks : 2/10

2. Problem Statement

You are an art collector who wants to visit a set of art galleries in a city. Each gallery is at a different location, and the cost of traveling between galleries is known.

Your goal is to find the shortest route that allows you to visit each gallery exactly once and return to your starting point, using the traveling salesman problem (TSP). Write a program to calculate the minimum cost of this route.

Note: Solve it using Branch and Bound technique.

Input Format

The first line contains an integer n, the number of galleries.

The next n lines contain n integers each, representing the adjacency matrix.

The ith integer on the jth line represents the cost of traveling from gallery i to gallery j.

Output Format

The output displays "Minimum Cost is: " followed by the minimum cost of the route that visits each gallery exactly once and returns to the starting gallery.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Output: Minimum Cost is: 80

Answer

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
int n;
```

```
int adj[15][15];
```

```
int visited[15];
```

```
int final_res = INT_MAX;
```

```
// Function to find the minimum edge cost
```

```
int firstMin(int i) {
```

```
    int min = INT_MAX;
```

```
    for (int k = 0; k < n; k++) {
```

```
        if (adj[i][k] && adj[i][k] < min && i != k)
```

```
            min = adj[i][k];
```

```
    }
```

```

    return min;
}

// Function to find the second minimum edge cost
int secondMin(int i) {
    int first = INT_MAX, second = INT_MAX;
    for (int j = 0; j < n; j++) {
        if (i == j) continue;
        if (adj[i][j] <= first) {
            second = first;
            first = adj[i][j];
        } else if (adj[i][j] <= second && adj[i][j] != first) {
            second = adj[i][j];
        }
    }
    return second;
}

```

```

// Recursive function for branch and bound
void TSPRec(int curr_bound, int curr_weight, int level, int curr_path[]) {
    if (level == n) {
        if (adj[curr_path[level - 1]][curr_path[0]] != 0) {
            int curr_res = curr_weight + adj[curr_path[level - 1]][curr_path[0]];
            if (curr_res < final_res) {
                final_res = curr_res;
            }
        }
    }
    return;
}

```

```

for (int i = 0; i < n; i++) {
    if (adj[curr_path[level - 1]][i] != 0 && visited[i] == 0) {
        int temp = curr_bound;
        curr_weight += adj[curr_path[level - 1]][i];

        if (level == 1)
            curr_bound -= ((firstMin(curr_path[level - 1]) + firstMin(i)) / 2);
        else
            curr_bound -= ((secondMin(curr_path[level - 1]) + firstMin(i)) / 2);

        if (curr_bound + curr_weight < final_res) {
            curr_path[level] = i;

```

```

        visited[i] = 1;
        TSPRec(curr_bound, curr_weight, level + 1, curr_path);
    }

    curr_weight -= adj[curr_path[level - 1]][i];
    curr_bound = temp;

    for (int j = 0; j < n; j++)
        visited[j] = 0;
    for (int j = 0; j <= level - 1; j++)
        visited[curr_path[j]] = 1;
    }
}

// Function to solve TSP using Branch and Bound
void TSP() {
    int curr_path[n + 1];
    int curr_bound = 0;

    for (int i = 0; i < n; i++)
        curr_bound += (firstMin(i) + secondMin(i));

    curr_bound = (curr_bound & 1) ? curr_bound / 2 + 1 : curr_bound / 2;

    visited[0] = 1;
    curr_path[0] = 0;

    TSPRec(curr_bound, 0, 1, curr_path);
}

int main() {
    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &adj[i][j]);

    TSP();

    printf("Minimum Cost is: %d\n", final_res);

    return 0;
}

```

}

Status : Partially correct

Marks : 2/10

3. Problem Statement

Sarah, a young entrepreneur starting her e-commerce business, has a limited budget for storing products in her warehouse. Each product has a specific weight and value, and Sarah aims to maximize the total value of the products in her warehouse without exceeding the weight capacity.

To achieve this, Sarah decided to apply the dynamic programming technique, which helps explore potential product combinations and avoid unnecessary calculations.

Input Format

The first line contains a single integer N , representing the number of products.

The next N lines each contain two integers: $\text{weight}[i]$ and $\text{value}[i]$, where $\text{weight}[i]$ is the weight of the product and $\text{value}[i]$ is the value (profit) of the product.

The next line contains a single integer W , representing the weight capacity of the warehouse.

Output Format

The output prints a single integer, representing the maximum total profit.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 5

2 40

3 50

1 100

5 95

3 30

10

Output: 245

Answer

```
#include <stdio.h>
```

```
// Function to return maximum of two values
```

```
int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```

```
int main() {  
    int N, W;  
    scanf("%d", &N);
```

```
    int weight[N], value[N];  
    for (int i = 0; i < N; i++) {  
        scanf("%d %d", &weight[i], &value[i]);  
    }
```

```
    scanf("%d", &W);
```

```
    // DP table: dp[i][w] = max profit using first i items with capacity w  
    int dp[N + 1][W + 1];
```

```
    for (int i = 0; i <= N; i++) {  
        for (int w = 0; w <= W; w++) {  
            if (i == 0 || w == 0) {  
                dp[i][w] = 0;  
            } else if (weight[i - 1] <= w) {  
                dp[i][w] = max(value[i - 1] + dp[i - 1][w - weight[i - 1]], dp[i - 1][w]);  
            } else {  
                dp[i][w] = dp[i - 1][w];  
            }  
        }  
    }  
}
```

```
printf("%d\n", dp[N][W]);  
return 0;
```

```
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

You are a savvy thief preparing to rob a jewelry store. You have a knapsack with a limited capacity and a list of items, each with a certain weight and a corresponding profit.

Your objective is to maximize the total profit you can obtain by selecting items to place in your knapsack while staying within its weight capacity. Write a program to help you calculate the maximum profit you can achieve.

Note: Use the 0/1 knapsack method to solve the program.

Example

Input:

3

60 10

100 20

120 30

50

Output:

220

Explanation:

By selecting the second and third items, you achieve a maximum profit of Rs. 220. The sum of their profits (100 + 120) gives you the highest total profit while not exceeding the knapsack's weight capacity of 50.

Input Format

The first line contains an integer N, representing the number of items available in the store.

Each of the next N lines contains two space-separated integers P and W, where P is the profit that can be obtained by stealing the item and W is the weight of the item.

The last line contains an integer C representing the weight capacity of your knapsack.

Output Format

The output displays an integer representing the maximum total profit that you can achieve by selecting items for your knapsack.

Refer to the sample output for the formatting specifications

Sample Test Case

Input: 3

60 10

100 20

120 30

50

Output: 220

Answer

```
#include <stdio.h>
```

```
// Utility function to get maximum of two integers
```

```
int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```

```
int main() {  
    int N, C;  
    scanf("%d", &N);
```

```
    int profit[N], weight[N];  
    for (int i = 0; i < N; i++) {  
        scanf("%d %d", &profit[i], &weight[i]);  
    }
```

```
    scanf("%d", &C);
```



```
// DP table: dp[i][w] = max profit using first i items with capacity w
int dp[N + 1][C + 1];
```

```
for (int i = 0; i <= N; i++) {
    for (int w = 0; w <= C; w++) {
        if (i == 0 || w == 0) {
            dp[i][w] = 0;
        } else if (weight[i - 1] <= w) {
            dp[i][w] = max(profit[i - 1] + dp[i - 1][w - weight[i - 1]], dp[i - 1][w]);
        } else {
            dp[i][w] = dp[i - 1][w];
        }
    }
}

printf("%d\n", dp[N][C]);
return 0;
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 4_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Meera is preparing for a science exhibition and needs to select a group of items whose total weight is exactly equal to a required target value. Each item can be selected at most once.

Given a list of item weights and a target weight K , determine whether it is possible to select a subset of items such that their sum is exactly K .

If such a subset exists, print any one valid subset. If no such subset exists, print -1.

This problem is derived from the Subset Sum problem, which is NP-Complete, and the solution must be implemented using Dynamic Programming.

Input Format

The first line contains two integers n and K

The second line contains n space-separated integers

Output Format

Print elements of one valid subset whose sum equals K

If no such subset exists, print -1

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5 9
3 34 4 12 5

Output: 5 4

Answer

```
#include <stdio.h>
```

```
int main() {  
    int n, K;  
    scanf("%d %d", &n, &K);
```

```
    int arr[n];  
    for (int i = 0; i < n; i++) {  
        scanf("%d", &arr[i]);  
    }
```

```
    // DP table: dp[i][j] = 1 if sum j is possible using first i items  
    int dp[n + 1][K + 1];
```

```
    for (int i = 0; i <= n; i++) {  
        for (int j = 0; j <= K; j++) {  
            if (j == 0)  
                dp[i][j] = 1; // sum 0 always possible  
            else if (i == 0)  
                dp[i][j] = 0; // no items, non-zero sum not possible  
            else if (arr[i - 1] <= j)  
                dp[i][j] = dp[i - 1][j] || dp[i - 1][j - arr[i - 1]];
```

```

    else
        dp[i][j] = dp[i - 1][j];
    }
}

if (!dp[n][K]) {
    printf("-1\n");
    return 0;
}

// Reconstruct subset
int subset[n], count = 0;
int i = n, j = K;
while (i > 0 && j > 0) {
    if (dp[i - 1][j]) {
        i--;
    } else {
        subset[count++] = arr[i - 1];
        j -= arr[i - 1];
        i--;
    }
}

// Print subset
for (int k = 0; k < count; k++) {
    printf("%d ", subset[k]);
}
printf("\n");

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Rohit is a travel planner who must organize a business trip covering multiple cities.

The travel cost between each pair of cities is given in a matrix.

He must:

Start from city 0
Visit every city exactly once
Return back to city 0
Your task is to compute:

The minimum possible travel cost
The sequence of cities visited
The average cost per step (excluding the final return step)

This problem is a classic Travelling Salesman Problem (TSP) solved using Dynamic Programming with Bitmasking (Held–Karp Algorithm).

Input Format

- The first line contains an integer n , the number of cities.
- The next n lines contain n space-separated integers, representing the cost matrix.

$\text{cost}[i][j]$ is the cost of traveling from city i to city j .

Output Format

Print the path:

The Path is: 0--->...--->0

Print the minimum cost:

Minimum cost is X

Print the average cost per step:

Average cost per step is Y

Rounded to 2 decimal places.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Output: The Path is: 0--->2--->3--->1--->0

Minimum cost is 80

Average cost per step is 26.67

Answer

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
int n;
```

```
int cost[15][15];
```

```
int dp[1<<11][11];
```

```
int parent[1<<11][11];
```

```
// DP + Bitmasking function
```

```
int tsp(int mask, int pos) {
```

```
    if (mask == (1<<n) - 1) {
```

```
        return cost[pos][0]; // return to start
```

```
    }
```

```
    if (dp[mask][pos] != -1) return dp[mask][pos];
```

```
    int ans = INT_MAX;
```

```
    parent[mask][pos] = -1;
```

```
    for (int city = 0; city < n; city++) {
```

```
        if ((mask & (1<<city)) == 0) {
```

```
            int newAns = cost[pos][city] + tsp(mask | (1<<city), city);
```

```
            // Tie-breaking: prefer smaller city index if costs are equal
```

```
            if (newAns < ans || (newAns == ans && (parent[mask][pos] == -1 || city < parent[mask][pos]))) {
```

```
                ans = newAns;
```

```

        parent[mask][pos] = city;
    }
}
}
return dp[mask][pos] = ans;
}

```

```

int main() {
    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    // Initialize DP and parent arrays
    for (int i = 0; i < (1<<n); i++)
        for (int j = 0; j < n; j++) {
            dp[i][j] = -1;
            parent[i][j] = -1;
        }
}

```

```

int minCost = tsp(1, 0);

```

```

// Reconstruct path
int mask = 1, pos = 0;
printf("The Path is: 0");
while (1) {
    int nextCity = parent[mask][pos];
    if (nextCity == -1) break;
    printf("--->%d", nextCity);
    mask |= (1<<nextCity);
    pos = nextCity;
}
printf("--->0\n");

```

```

printf("Minimum cost is %d\n", minCost);

```

```

// Average cost per step (excluding final return step)
double avgCost = (double)minCost / (n - 1);
printf("Average cost per step is %.2f\n", avgCost);

return 0;
}

```

Status : **Wrong**

Marks : 0/10

3. Problem Statement

Captain Aidan, a daring pirate, has discovered an ancient treasure cave filled with valuable relics. However, the cave is protected by an ancient spell that limits the weight of the treasure he can carry. Aidan has a magic knapsack with a maximum weight capacity of W . Inside the cave, he finds n different treasures, each with a specific weight and value.

Since he can only take each treasure once, Aidan must carefully choose which items to take to maximize the total value without exceeding the weight limit of his knapsack.

Help Captain Aidan pick the most valuable treasures while staying within the weight limit using branch and bound technique

Input Format

The first line contains two integers n (the number of items) and W (the capacity of the knapsack), separated by a space.

The second line contains n integers, where the i -th integer represents the weight of the i -th item.

The third line contains n integers, where the i -th integer represents the value of the i -th item

Output Format

The output prints the single integer representing the maximum value obtained by filling the knapsack with the given set of items.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 60
10 20 30 40 50

60 100 120 140 180

Output: 280

Answer

```
#include <stdio.h>
#include <stdlib.h>
```

```
// Structure to represent an item
typedef struct {
    int weight;
    int value;
    double ratio;
} Item;
```

```
// Structure to represent a node in Branch and Bound
typedef struct {
    int level;
    int profit;
    int weight;
    double bound;
} Node;
```

```
// Function to compare items by value/weight ratio
int cmp(const void *a, const void *b) {
    Item *i1 = (Item *)a;
    Item *i2 = (Item *)b;
    if (i2->ratio > i1->ratio) return 1;
    else if (i2->ratio < i1->ratio) return -1;
    return 0;
}
```

```
// Function to calculate upper bound of profit in subtree
double bound(Node u, int n, int W, Item arr[]) {
    if (u.weight >= W) return 0;

    double profit_bound = u.profit;
    int j = u.level + 1;
    int totweight = u.weight;

    while (j < n && totweight + arr[j].weight <= W) {
        totweight += arr[j].weight;
        profit_bound += arr[j].value;
    }
}
```

```

    j++;
}

if (j < n)
    profit_bound += (W - totweight) * arr[j].ratio;

return profit_bound;
}

```

```

// Branch and Bound Knapsack
int knapsack(int W, Item arr[], int n) {
    qsort(arr, n, sizeof(Item), cmp);

```

```

    Node u, v;
    u.level = -1;
    u.profit = 0;
    u.weight = 0;
    u.bound = bound(u, n, W, arr);

```

```

    int maxProfit = 0;

```

```

    // Simple queue simulation
    Node queue[10000];
    int front = 0, rear = 0;
    queue[rear++] = u;

```

```

    while (front < rear) {
        u = queue[front++];
        if (u.level == n - 1) continue;

```

```

        v.level = u.level + 1;
        v.weight = u.weight + arr[v.level].weight;
        v.profit = u.profit + arr[v.level].value;

```

```

        if (v.weight <= W && v.profit > maxProfit)
            maxProfit = v.profit;

```

```

        v.bound = bound(v, n, W, arr);
        if (v.bound > maxProfit)
            queue[rear++] = v;

```

```

        v.weight = u.weight;

```

```

        v.profit = u.profit;
        v.bound = bound(v, n, W, arr);
        if (v.bound > maxProfit)
            queue[rear++] = v;
    }

    return maxProfit;
}

int main() {
    int n, W;
    scanf("%d %d", &n, &W);

    Item arr[n];
    for (int i = 0; i < n; i++) scanf("%d", &arr[i].weight);
    for (int i = 0; i < n; i++) scanf("%d", &arr[i].value);

    for (int i = 0; i < n; i++)
        arr[i].ratio = (double)arr[i].value / arr[i].weight;

    int result = knapsack(W, arr, n);
    printf("%d\n", result);

    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Josh is learning about the Fractional Knapsack problem, where a knapsack has a fixed maximum capacity. Given a set of items, each with a value and a weight, Josh wants to maximize the total value placed into the knapsack.

Unlike the 0/1 Knapsack problem, fractions of items are allowed. Items must be selected using a Greedy strategy based on their value-to-weight ratio.

Write a program to determine the fraction of each item that should be included in the knapsack to achieve the maximum total value.

Formula

Value-to-Weight Ratio

$\text{ratio} = \text{value} / \text{weight}$

Fraction Calculation

$\text{fraction} = \text{remainingCapacity} / \text{weight}$

Notes

Items must be selected using the Greedy approach based on value-to-weight ratio. Sorting is required to determine the optimal order. Output fractions must be printed in the original item order. Use double datatype only for calculations.

Input Format

The first line contains an integer n , representing the number of items.

The second line contains n space-separated integers representing item values.

The third line contains n space-separated integers representing item weights.

The fourth line contains an integer capacity, representing knapsack capacity.

Output Format

Print n space-separated double values, representing the fraction of each item selected.

Output must be printed rounded to exactly two decimal places.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

3 5 1 2 4

40 50 20 10 30

75

Output: 0.00 0.70 0.00 1.00 1.00

Answer

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int index;
    int value;
    int weight;
    double ratio;
} Item;

// Comparator for sorting items by ratio (descending)
int cmp(const void *a, const void *b) {
    Item *i1 = (Item *)a;
    Item *i2 = (Item *)b;
    if (i2->ratio > i1->ratio) return 1;
    else if (i2->ratio < i1->ratio) return -1;
    return 0;
}

int main() {
    int n, capacity;
    scanf("%d", &n);

    int values[n], weights[n];
    for (int i = 0; i < n; i++) scanf("%d", &values[i]);
    for (int i = 0; i < n; i++) scanf("%d", &weights[i]);
    scanf("%d", &capacity);

    Item items[n];
    for (int i = 0; i < n; i++) {
        items[i].index = i;
        items[i].value = values[i];
        items[i].weight = weights[i];
        items[i].ratio = (double)values[i] / weights[i];
    }

    qsort(items, n, sizeof(Item), cmp);

    double fractions[n];
```

```

for (int i = 0; i < n; i++) fractions[i] = 0.0;

int remaining = capacity;
for (int i = 0; i < n; i++) {
    if (items[i].weight <= remaining) {
        fractions[items[i].index] = 1.0;
        remaining -= items[i].weight;
    } else {
        fractions[items[i].index] = (double)remaining / items[i].weight;
        remaining = 0;
        break;
    }
}

for (int i = 0; i < n; i++) {
    printf("%.2f", fractions[i]);
    if (i != n - 1) printf(" ");
}

printf("\n");

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Ananya is preparing for a long trek and has a bag with limited capacity. She has several items, each with a weight and value, and she wants to maximize the total value she can carry. She may take whole items or fractions of them to fit in her bag.

The maximum value is calculated using the formula:

Maximum Value = $\sum (\text{value}[i] * \text{fraction_taken}[i])$

where $\text{fraction_taken}[i] = \min(1, \text{remaining_capacity} / \text{weight}[i])$

Write a program to help Ananya determine the maximum achievable value.

Input Format

The input consists of several lines, each representing an item.

Each line (until -1) contains two integers: weight and value of an item.

The input terminates with -1.

The final line contains one integer representing the bag's capacity.

Output Format

The output prints "The maximum value of the current list is:"

On the next line, print the maximum value achievable with exactly two decimal places

Refer to the sample outputs for the formatting specifications.

Sample Test Case

Input: 10 60

20 100

30 120

-1

50

Output: The maximum value of the current list is:

240.00

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct {  
    int weight;  
    int value;  
    double ratio;  
} Item;
```

```
// Comparator for sorting items by ratio (descending)
```

```
int cmp(const void *a, const void *b) {
```

```

    Item *i1 = (Item *)a;
    Item *i2 = (Item *)b;
    if (i2->ratio > i1->ratio) return 1;
    else if (i2->ratio < i1->ratio) return -1;
    return 0;
}

```

```

int main() {
    Item items[1000];
    int n = 0;

    // Read items until -1
    while (1) {
        int w;
        if (scanf("%d", &w) != 1) return 0;
        if (w == -1) break;
        int v;
        scanf("%d", &v);
        items[n].weight = w;
        items[n].value = v;
        items[n].ratio = (double)v / w;
        n++;
    }
}

```

```

int capacity;
scanf("%d", &capacity);

// Sort items by ratio
qsort(items, n, sizeof(Item), cmp);

```

```

double maxValue = 0.0;
int remaining = capacity;

```

```

for (int i = 0; i < n; i++) {
    if (items[i].weight <= remaining) {
        remaining -= items[i].weight;
        maxValue += items[i].value;
    } else {
        maxValue += items[i].value * ((double)remaining / items[i].weight);
        break;
    }
}
}

```



```
printf("The maximum value of the current list is:\n");  
printf("%.2f\n", maxVal);  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 4_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 38.5

Section 1 : Coding

1. Problem Statement

Janu is tasked with creating a program that solves the Traveling Salesman Problem (TSP). The goal is to find the shortest possible route that visits a set of cities and returns to the starting city.

You are required to implement a program that takes input regarding the number of cities and the distance matrix between cities and outputs the optimal route and its associated minimum cost.

The diagonal values will be 0, as the distance from a city to itself is 0.

Note: Solve it using Branch and Bound approach.

Input Format

The first line of input consists of an integer n , representing the number of cities.

The next n lines contain n space-separated integers each, representing the distance matrix between the cities.

The cell (i, j) represents the distance from city i to city j .

Output Format

The first line of output displays "The Path is: " followed by the path of the route using ---> in between.

The second line displays "Minimum cost is " followed by the total minimum cost associated with the shortest route.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 4 1 3

4 0 2 1

1 2 0 5

3 1 5 0

Output: The Path is: 1 ---> 3 ---> 2 ---> 4 ---> 1

Minimum cost is 7

Answer

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
int n;
```

```
int dist[10][10];
```

```
int visited[10];
```

```
int final_path[10];
```

```
int curr_path[10];
```

```
int final_res = INT_MAX;
```

```
int calculate_cost(int path[], int path_len) {
```

```
    int cost = 0;
```

```
    for(int i = 0; i < path_len - 1; i++) {
```

```

    cost += dist[path[i]-1][path[i+1]-1];
}
cost += dist[path[path_len-1]-1][path[0]-1];
return cost;
}

```

```

void tsp(int level, int curr_cost) {
    if(level == n) {

        int total_cost = curr_cost + dist[curr_path[level-1]-1][curr_path[0]-1];
        if(total_cost < final_res) {
            final_res = total_cost;
            for(int i = 0; i < n; i++)
                final_path[i] = curr_path[i];
        }
        return;
    }
}

```

```

for(int i = 1; i <= n; i++) {
    if(!visited[i]) {
        visited[i] = 1;
        curr_path[level] = i;
        int new_cost = curr_cost;
        if(level > 0) new_cost += dist[curr_path[level-1]-1][i-1];

        if(new_cost < final_res)
            tsp(level+1, new_cost);

        visited[i] = 0;
    }
}
}
}

```

```

int main() {
    scanf("%d", &n);
    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
            scanf("%d", &dist[i][j]);

    for(int i = 0; i <= n; i++) visited[i] = 0;
    curr_path[0] = 1;
    visited[1] = 1;
}

```

```

tsp(1, 0);

printf("The Path is: ");
for(int i = 0; i < n; i++)
    printf("%d-->", final_path[i]);
printf("%d\n", final_path[0]);

printf("Minimum cost is %d\n", final_res);

return 0;
}

```

Status : Partially correct

Marks : 8.5/10

2. Problem Statement

Karan is planning a delivery route that starts from city 0, visits every other city exactly once, and returns back to city 0.

The travel costs between cities are given as a cost matrix.

Your task is to find the minimum possible travel cost and print the optimal route.

Cost Formula

For a route:

0 a b ... 0

Total Cost = cost[0][a] + cost[a][b] + ... + cost[last][0]

Input Format

The first line contains an integer n, representing the number of cities.

The next n lines contain n space-separated integers representing the cost matrix.

Output Format

The output prints multiple lines showing path exploration in the format:

The first line prints the minimum travel cost.

The second line prints the optimal path starting from city 0 and ending at city 0.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0

Output: 80
0 1 3 2 0

Answer

```
#include <stdio.h>
#include <limits.h>

int n;
int cost[5][5];
int visited[5];
int best_path[5];
int curr_path[5];
int min_cost = INT_MAX;

void tsp(int city, int level, int curr_cost) {
    if(level == n) {
        curr_cost += cost[city][0]; // Return to start
        if(curr_cost < min_cost) {
            min_cost = curr_cost;
            for(int i = 0; i < n-1; i++)
```

```

        best_path[i] = curr_path[i];
    }
    return;
}

for(int i = 1; i < n; i++) { // 0 is starting city
    if(!visited[i]) {
        visited[i] = 1;
        curr_path[level] = i;
        tsp(i, level+1, curr_cost + cost[city][i]);
        visited[i] = 0;
    }
}
}
}

int main() {
    scanf("%d", &n);
    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    for(int i = 0; i < n; i++) visited[i] = 0;

    tsp(0, 0, 0);

    printf("%d\n", min_cost);

    // Print optimal path starting and ending at 0
    printf("0 ");
    for(int i = 0; i < n-1; i++)
        printf("%d ", best_path[i]);
    printf("0\n");

    return 0;
}

```

Status : Wrong

Marks : 0/10

3. Problem Statement

You are a logistics manager responsible for planning delivery routes for a

fleet of vehicles. Each vehicle needs to visit a set of locations to deliver goods, and the cost of traveling between locations is known.

Your goal is to find the shortest route for each vehicle, ensuring that each location is visited exactly once before returning to the starting point.

Note: Solve it using Branch and Bound approach.

Input Format

The first line contains an integer n , the number of locations.

The next n lines contain n integers each, representing the cost matrix of traveling between locations.

The i th integer on the j th line represents the cost of traveling from location i to location j .

Output Format

The first line of output displays "Shortest Path: " followed by the shortest path that visits all cities.

The second line of output displays "Minimum cost is " followed by the minimum cost.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

0 10 8 9 7

10 0 10 5 6

8 10 0 8 9

9 5 8 0 6

7 6 9 6 0

Output: Shortest Path: 1 3 4 2 5 1

Minimum cost is 34

Answer

```
#include <stdio.h>
```



```
#include <limits.h>
```

```
int n;  
int cost[10][10];  
int visited[10];  
int best_path[10];  
int curr_path[10];  
int min_cost = INT_MAX;
```

```
void tsp(int city, int level, int curr_cost) {  
    if(level == n) {  
        curr_cost += cost[city][0];  
        if(curr_cost < min_cost) {  
            min_cost = curr_cost;  
            for(int i = 0; i < n; i++)  
                best_path[i] = curr_path[i];  
        }  
        return;  
    }  
}
```

```
for(int i = 1; i < n; i++) {  
    if(!visited[i]) {  
        visited[i] = 1;  
        curr_path[level] = i;  
        tsp(i, level+1, curr_cost + cost[city][i]);  
        visited[i] = 0;  
    }  
}
```

```
int main() {  
    scanf("%d", &n);  
  
    for(int i = 0; i < n; i++)  
        for(int j = 0; j < n; j++)  
            scanf("%d", &cost[i][j]);  
  
    for(int i = 0; i < n; i++) visited[i] = 0;  
  
    curr_path[0] = 0;  
    visited[0] = 1;
```

```

tsp(0, 1, 0);

printf("Shortest Path: ");
for(int i = 0; i < n; i++)
    printf("%d ", best_path[i]+1);
printf("1\n");

printf("Minimum cost is %d\n", min_cost);

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Michael has been given an important task—developing a program to solve the classic 0/1 Knapsack Problem. His goal is to help users select the most valuable combination of items while staying within a specified weight limit.

In this problem, each item has a value and a weight, and the objective is to maximize the total value without exceeding the given weight capacity.

Michael's program should determine the maximum possible value that can be obtained by selecting the optimal set of items.

Input Format

The first line of input consists of an integer n , representing the number of items available for selection.

The second line of input consists of n space-separated integers, P , which represent the values of the items.

The third line of input consists of n space-separated integers, C , which represent the weights of the corresponding items.

The fourth line of input consists of an integer W , representing the maximum weight limit of the knapsack.

Output Format

The output displays the maximum value that can be obtained using the knapsack algorithm in the following format: "Maximum value: [maximum value]"

Refer to the sample outputs for the exact format.

Sample Test Case

Input: 6

300 150 120 100 90 80

6 3 3 2 2 2

10

Output: Maximum value: 490

Answer

```
#include <stdio.h>
```

```
int n;
```

```
int P[15], C[15];
```

```
int W;
```

```
int max_value = 0;
```

```
void knapsack(int idx, int curr_weight, int curr_value) {
```

```
    if(curr_weight > W) return;
```

```
    if(idx == n) {
```

```
        if(curr_value > max_value)
```

```
            max_value = curr_value;
```

```
        return;
```

```
    }
```

```
    knapsack(idx+1, curr_weight + C[idx], curr_value + P[idx]);
```

```
    knapsack(idx+1, curr_weight, curr_value);
```

```
}
```

```
int main() {
```

```
    scanf("%d", &n);
```

```
    for(int i=0; i<n; i++)
```

```
        scanf("%d", &P[i]);
```

```
for(int i=0; i<n; i++)
    scanf("%d", &C[i]);
scanf("%d", &W);

knapsack(0, 0, 0);

printf("Maximum value: %d\n", max_value);
return 0;
}
```

Status : Correct

Marks : 10/10

5. Problem Statement

Ragul is interested in solving the knapsack problem. He has a list of n items, each with a weight and a value. Ragul wants to determine the maximum value he can obtain by selecting a combination of items to fit into his knapsack, which has a maximum weight capacity.

Help Ragul write a program that calculates the following:

Calculate and print the sum of all n values. Calculate and print the average value of all n values with exactly 2 decimal places. Calculate and print the maximum value that can be obtained by selecting a combination of items to fit into the knapsack without exceeding its capacity.

Note: Use the 0/1 knapsack method to solve the problem.

Input Format

The first line of input contains an integer n , representing the number of items.

The next line contains n space-separated integers, representing the weights of the items.

The next line contains n space-separated integers, representing the values of the items.

The last line contains an integer C , representing the maximum weight capacity of the knapsack.

Output Format

The output consists of the following in each line:

1. The sum of all n values as an integer.
2. The average value of all n values with exactly 2 decimal places is a double value.
3. The maximum value that can be obtained as an integer.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
10 15 20
45 65 70
30

Output: Sum of values: 180
Average of values: 60.00
Maximum amount: 115

Answer

```
#include <stdio.h>
```

```
int n, C;  
int weight[10], value[10];  
int max_value = 0;
```

```
void knapsack(int idx, int curr_weight, int curr_value) {  
    if(curr_weight > C) return;  
    if(idx == n) {  
        if(curr_value > max_value)  
            max_value = curr_value;  
        return;  
    }  
}
```

```
    knapsack(idx + 1, curr_weight + weight[idx], curr_value + value[idx]);
```

```
    knapsack(idx + 1, curr_weight, curr_value);  
}
```

```
int main() {
    scanf("%d", &n);
    for(int i = 0; i < n; i++)
        scanf("%d", &weight[i]);
    for(int i = 0; i < n; i++)
        scanf("%d", &value[i]);
    scanf("%d", &C);

    int sum = 0;
    for(int i = 0; i < n; i++)
        sum += value[i];
    double avg = (double)sum / n;

    knapsack(0, 0, 0);

    printf("Sum of values: %d\n", sum);
    printf("Average of values: %.2lf\n", avg);
    printf("Maximum amount: %d\n", max_value);

    return 0;
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 4_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 14

Section 1 : MCQ

1. Which of the following methods can be used to solve the Knapsack problem?

Answer

All the mentioned options

Status : Correct

Marks : 1/1

2. 0/1 knapsack is based on _____method.

Answer

Dynamic programming

Status : Correct

Marks : 1/1

3. What is the primary advantage of using the Branch and Bound approach for the Knapsack problem?

Answer

It systematically explores possible solutions while pruning non-promising ones.

Status : Correct

Marks : 1/1

4. Which data structure is commonly used to implement the Branch and Bound approach for solving the Knapsack problem?

Answer

Priority Queue

Status : Correct

Marks : 1/1

5. In the Branch and Bound approach, what does the term “bounding function” refer to?

Answer

A function that estimates the upper bound on the maximum value that can be achieved from a node.

Status : Correct

Marks : 1/1

6. Which strategy is commonly used in the Branch and Bound method to decide which node to explore next?

Answer

Best-first search

Status : Correct

Marks : 1/1

7. Which of the following methods is used to prune the search space in the Branch and Bound method for the Knapsack problem?

Answer

Ignoring nodes with a weight exceeding the capacity.

Status : Correct

Marks : 1/1

8. The travelling salesman problem can be solved using _____.

Answer

Bellman - Ford algorithm

Status : Wrong

Marks : 0/1

9. Travelling salesman problem is an example of _____.

Answer

None of the mentioned options

Status : Correct

Marks : 1/1

10. How does the practical Travelling salesman problem differ from the classical Travelling salesman problem?

Answer

In the practical travelling salesman problem, each vertex can be visited more than once.

Status : Correct

Marks : 1/1

11. The traveling salesman problem involves n cities with paths connecting the cities. The time taken for traversing through all the cities without knowing in advance the length of a minimum tour is

Answer

$O(n!)$

Status : Correct

Marks : 1/1

12. In which search problem, to find the shortest path, each city must be

visited only once?

Answer

Travelling Salesman problem

Status : Correct

Marks : 1/1

13. What is the main goal of using Branch and Bound in TSP?

Answer

To minimize search space by pruning unpromising paths

Status : Correct

Marks : 1/1

14. Which value helps in deciding whether a node should be pruned in Branch and Bound?

Answer

Bound (lower bound of cost)

Status : Correct

Marks : 1/1

15. Given items with weights [10, 20, 30] and values [60, 100, 120], what is the value-to-weight ratio for the second item?

Answer

5

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 5_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Given an array of integers, count the number of smaller elements that appear to the right of each element. Implement a solution using merge sort to count these elements efficiently. Output the result as an array of counts corresponding to each element in the original array.

Example

Input:

4

5 2 6 1

Output:

2 1 1 0

Explanation:

To the right of 5, there are 2 smaller elements (2 and 1).

To the right of 2, there is only 1 smaller element (1).

To the right of 6, there is 1 smaller element (1).

To the right of 1, there is 0 smaller element.

Input Format

The first line of input contains a single integer n , representing the length of the array.

The second line contains n space-separated integers, representing the elements of the array.

Output Format

The output prints a single line containing the counts of smaller elements for each value in the array, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

5 2 6 1

Output: 2 1 1 0

Answer

```
#include <stdio.h>
```

```
typedef struct {  
    int val;  
    int index;  
} Pair;
```

```
void merge(Pair arr[], int left, int mid, int right, int count[]) {  
    int n1 = mid - left + 1;  
    int n2 = right - mid;
```

```
Pair L[n1], R[n2];
for (int i = 0; i < n1; i++) L[i] = arr[left + i];
for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
```

```
int i = 0, j = 0, k = left;
int rightCount = 0;
```

```
while (i < n1 && j < n2) {
    if (L[i].val <= R[j].val) {
        count[L[i].index] += rightCount;
        arr[k++] = L[i++];
    } else {
        rightCount++;
        arr[k++] = R[j++];
    }
}
```

```
while (i < n1) {
    count[L[i].index] += rightCount;
    arr[k++] = L[i++];
}
while (j < n2) {
    arr[k++] = R[j++];
}
}
```

```
void mergeSort(Pair arr[], int left, int right, int count[]) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid, count);
        mergeSort(arr, mid + 1, right, count);
        merge(arr, left, mid, right, count);
    }
}
```

```
int main() {
    int n;
    scanf("%d", &n);

    Pair arr[n];
    int count[n];
```

```

for (int i = 0; i < n; i++) {
    scanf("%d", &arr[i].val);
    arr[i].index = i;
    count[i] = 0;
}

mergeSort(arr, 0, n - 1, count);

for (int i = 0; i < n; i++) {
    printf("%d ", count[i]);
}
printf("\n");

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Ravi, a mathematics teacher, is helping his students understand quadratic equations. He wants to apply the quadratic formula $[A * x^2 + B * x + C]$ to each number in a sorted array. Once the formula is applied, the modified numbers need to be sorted again. Ravi plans to use QuickSort for efficient sorting. Help Ravi by implementing a program that performs these operations.

Input Format

The first line contains an integer n , representing the size of the array.

The second line contains n space-separated integers, representing the elements of the array in ascending order.

The third line contains three integers A , B , and C , the coefficients of the quadratic formula.

Output Format

The first line of output prints: Array after applying the equation and sorting

The subsequent lines print: <sorted_value>

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

-1 0 1 2 3 4

-1 2 -1

Output: Array after applying the equation and sorting:

-9 -4 -4 -1 -1 0

Answer

```
#include <stdio.h>
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
int main() {
```

```

int n;
scanf("%d", &n);

int arr[n];
for (int i = 0; i < n; i++) {
    scanf("%d", &arr[i]);
}

int A, B, C;
scanf("%d %d %d", &A, &B, &C);
for (int i = 0; i < n; i++) {
    arr[i] = A * arr[i] * arr[i] + B * arr[i] + C;
}
quickSort(arr, 0, n - 1);
printf("Array after applying the equation and sorting:\n");
for (int i = 0; i < n; i++) {
    printf("%d ", arr[i]);
}
printf("\n");

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Amit, an e-commerce manager, is working on an inventory management system to track the top-selling products. Given the sales data of various products, he wants to quickly determine the Kth best-selling product to make informed restocking decisions.

Help Amit implement Heap Sort to efficiently find the Kth best-selling product from the given sales data.

Input Format

The first line of input consists of an integer n, representing the number of products.

The second line consists of n space-separated integers, representing the sales

data of each product.

The third line contains an integer K, representing the rank of the product to retrieve.

Output Format

The output prints the Kth best-selling product based on the given sales data.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

10 20 15 5 25

2

Output: 20

Answer

```
#include <stdio.h>
```

```
// Function to swap two integers
```

```
void swap(long long *a, long long *b) {
```

```
    long long temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
// Heapify function for max-heap
```

```
void heapify(long long arr[], int n, int i) {
```

```
    int largest = i;
```

```
    int left = 2 * i + 1;
```

```
    int right = 2 * i + 2;
```

```
    if (left < n && arr[left] > arr[largest])
```

```
        largest = left;
```

```
    if (right < n && arr[right] > arr[largest])
```

```
        largest = right;
```

```
    if (largest != i) {
```

```
        swap(&arr[i], &arr[largest]);  
        heapify(arr, n, largest);  
    }  
}
```

```
// Heap Sort function  
void heapSort(long long arr[], int n) {  
    // Build max heap  
    for (int i = n / 2 - 1; i >= 0; i--)  
        heapify(arr, n, i);  
  
    // Extract elements one by one  
    for (int i = n - 1; i > 0; i--) {  
        swap(&arr[0], &arr[i]);  
        heapify(arr, i, 0);  
    }  
}
```

```
int main() {  
    int n;  
    scanf("%d", &n);  
  
    long long arr[n];  
    for (int i = 0; i < n; i++) {  
        scanf("%lld", &arr[i]);  
    }  
  
    int K;  
    scanf("%d", &K);  
  
    // Sort using Heap Sort  
    heapSort(arr, n);  
  
    // Kth best-selling product = Kth largest element  
    // After heap sort, array is ascending, so Kth largest is arr[n-K]  
    printf("%lld\n", arr[n - K]);  
  
    return 0;  
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Tom is working in a warehouse inventory system, where the item IDs are assigned sequentially in ascending order.

He wants to develop a program using recursive binary search to efficiently determine the closest item ID that is less than or equal to a given target ID.

The program takes the total number of items and their sorted IDs as input, assisting warehouse staff in quickly identifying the closest available item less than or equal to the target ID. Help Tom in this program.

Input Format

The first line of input consists of an integer N, representing the number of items in the warehouse.

The second line consists of N space-separated integers, representing the sorted list of item IDs.

The third line consists of an integer representing the target item ID.

Output Format

The output prints "The closest item ID less than or equal to X is Y", where X is the target ID and Y is the closest item ID less than or equal to X.

If no such element exists, print "No closest item with an ID less than or equal to X exists in the warehouse", where X is the target ID.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7

17 25 38 47 51 62 79

50

Output: The closest item ID less than or equal to 50 is 47

Answer

```
#include <stdio.h>
```

```
// Recursive binary search to find closest <= target  
int closestItem(int arr[], int low, int high, int target, int *result) {  
    if (low > high) return -1;
```

```
    int mid = low + (high - low) / 2;
```

```
    if (arr[mid] == target) {  
        *result = arr[mid];  
        return mid;
```

```
    }
```

```
    else if (arr[mid] < target) {  
        *result = arr[mid]; // possible candidate  
        return closestItem(arr, mid + 1, high, target, result);
```

```
    }
```

```
    else {  
        return closestItem(arr, low, mid - 1, target, result);
```

```
    }
```

```
}
```

```
int main() {
```

```
    int n;  
    scanf("%d", &n);
```

```
    int arr[15]; // safe buffer for up to 10 items  
    for (int i = 0; i < n; i++) {  
        scanf("%d", &arr[i]);  
    }
```

```
    int target;  
    scanf("%d", &target);
```

```
    int result = -1;  
    closestItem(arr, 0, n - 1, target, &result);
```

```
    if (result == -1) {  
        printf("No closest item with an ID less than or equal to %d exists in the  
warehouse\n", target);  
    } else {  
        printf("The closest item ID less than or equal to %d is %d\n", target, result);  
    }
```

```
} return 0;
```

Status : Correct

Marks : 10/10

5. Problem Statement

Rahul owns a bookstore and maintains a sorted list of ISBN numbers of books in his inventory. When a customer asks for a specific book, Rahul wants to quickly check whether the book is available in his inventory.

Write a program that implements a non-recursive binary search to determine if a given ISBN exists in the sorted list of ISBN numbers.

Input Format

The first line contains an integer N, representing the number of ISBN numbers in the inventory.

The second line contains N space-separated integers representing the sorted list of ISBN numbers.

The third line contains a single integer X, the ISBN number to be searched.

Output Format

If the ISBN X is found in the inventory, print: "Book with ISBN X is available at index Y" where Y is the zero-based index of the ISBN in the list.

If the ISBN X is not found, print: "Book with ISBN X is not available in the inventory"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
2 3 40 50 20
3

Output: Book with ISBN 3 is available at index 1

Answer

```
#include <stdio.h>
```

```
int binarySearch(int arr[], int n, int x) {  
    int low = 0, high = n - 1;
```

```
    while (low <= high) {  
        int mid = low + (high - low) / 2;
```

```
        if (arr[mid] == x)  
            return mid;
```

```
        else if (arr[mid] < x)  
            low = mid + 1;
```

```
        else  
            high = mid - 1;
```

```
    }
```

```
    return -1;
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int arr[35]; // safe buffer for up to 30 ISBNs
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    int x;
```

```
    scanf("%d", &x);
```

```
    int index = binarySearch(arr, n, x);
```

```
    if (index != -1) {
```

```
        printf("Book with ISBN %d is available at index %d\n", x, index);
```

```
    } else {
```

```
        printf("Book with ISBN %d is not available in the inventory\n", x);
```

```
    }
```

```
    return 0;
```

}

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 5_COD

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Sam needs to sort an array of integers using the divide-and-conquer method. He wants to implement the merge sort algorithm, displaying the array after each iteration to track the sorting process.

Help him write a program that divides the array, merges it back, and prints the array.

Input Format

The first line of input consists of an integer n , denoting the array size.

The second line consists of n space-separated integers, representing the array of elements.

Output Format

The first line of output prints "Given Array" followed by a (String), indicating the start of the initial array display.

The second line of output prints the array elements (integers) separated by spaces.

After each merge iteration, the output prints "After iteration X", where X is the iteration number (starting from 1). This is followed by the (String) "After iteration X" to indicate the iteration count.

On the next line, the current state of the array after that iteration is printed, showing the array elements (integers) separated by spaces.

Finally, after all iterations are completed, the output prints "Sorted Array" followed by a (String), indicating the sorted array.

The final line prints the sorted array (integers), with each element separated by spaces.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

4 1 5 3 2 6

Output: Given Array

4 1 5 3 2 6

After iteration 1

1 4 5 3 2 6

After iteration 2

1 4 5 3 2 6

After iteration 3

1 4 5 2 3 6

After iteration 4

1 4 5 2 3 6
After iteration 5
1 2 3 4 5 6
Sorted Array
1 2 3 4 5 6

Answer

```
#include <stdio.h>
```

```
int iteration = 0;
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
void merge(int arr[], int l, int m, int r, int n) {
    int n1 = m - l + 1;
    int n2 = r - m;

    int L[n1], R[n2];
    for (int i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k++] = L[i++];
        } else {
            arr[k++] = R[j++];
        }
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
    iteration++;
    printf("After iteration %d\n", iteration);
    printArray(arr, n);
}
void mergeSort(int arr[], int l, int r, int n) {
    if (l < r) {
```

```
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m, n);
        mergeSort(arr, m + 1, r, n);
        merge(arr, l, m, r, n);
    }
}
```

```
int main() {
    int n;
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Given Array\n");
    printArray(arr, n);

    mergeSort(arr, 0, n - 1, n);

    printf("Sorted Array\n");
    printArray(arr, n);

    return 0;
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Micheal is tasked with developing a system to manage and sort student records for a university. Each student record contains the student's name, GPA, age, and major. The goal is to implement a program that allows the user to input multiple student records and then sort these records based on a chosen criterion: GPA, age, or major.

Can you guide Micheal in developing the program using a quick sort algorithm?

Input Format

The first line of input consists of an integer n , representing the number of student records.

For each student record, the following lines consist of:

- Name (string)
- GPA (float)
- Age (integer)
- Major (string)

The last line of input consists of the sorting criterion, which can be one of the following strings:

- "gpa"
- "age"
- "major"

Output Format

The first line of output will print:

Sorted Student Records:

The output should display a sorted list of student records in a tabular format based on the chosen criterion. The GPA value should be rounded off to two decimal places.

Table Header

"Name\t\tGPA\t\tAge\t\tMajor"

Table Content

For each student, the format of the output will be:

"Name\tGPA\tAge\t\tMajor"

- The values should be aligned as follows:
- Name: Left-aligned, up to 10 characters.
- GPA: Displayed with two decimal places.
- Age: Left-aligned, up to 3 digits.

- Major: Left-aligned, up to 15 characters.

Note: "\t" - It creates a horizontal tab space

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

Harish

7.8

23

ece

Trell

8

21

cse

Kamalesh

7

25

it

Maaran

5.6

20

civil

age

Output: Sorted Student Records:

Name	GPA	Age	Major
------	-----	-----	-------

Maaran	5.60	20	civil
--------	------	----	-------

Trell	8.00	21	cse
-------	------	----	-----

Harish	7.80	23	ece
--------	------	----	-----

Kamalesh	7.00	25	it
----------	------	----	----

Answer

```
#include <stdio.h>
```

```
#include <string.h>
```

```
typedef struct {  
    char name[20];  
    float gpa;  
    int age;  
    char major[20];  
} Student;
```

```
char criterion[10];  
int compare(Student a, Student b) {  
    if (strcmp(criterion, "gpa") == 0) {  
        if (a.gpa < b.gpa) return -1;  
        else if (a.gpa > b.gpa) return 1;  
        else return 0;  
    } else if (strcmp(criterion, "age") == 0) {  
        return a.age - b.age;  
    } else if (strcmp(criterion, "major") == 0) {  
        return strcmp(a.major, b.major);  
    }  
    return 0;  
}
```

```
int partition(Student arr[], int low, int high) {  
    Student pivot = arr[high];  
    int i = low - 1;  
    for (int j = low; j < high; j++) {  
        if (compare(arr[j], pivot) <= 0) {  
            i++;  
            Student temp = arr[i];  
            arr[i] = arr[j];  
            arr[j] = temp;  
        }  
    }  
    Student temp = arr[i + 1];  
    arr[i + 1] = arr[high];  
    arr[high] = temp;  
    return i + 1;  
}  
void quickSort(Student arr[], int low, int high) {  
    if (low < high) {  
        int pi = partition(arr, low, high);  
        quickSort(arr, low, pi - 1);  
        quickSort(arr, pi + 1, high);  
    }  
}
```

```

}
void printStudents(Student arr[], int n) {
    printf("Name\t\tGPA\t\tAge\t\tMajor\n");
    for (int i = 0; i < n; i++) {
        printf("%-10s\t%.2f\t%-3d\t\t%-15s\n",
            arr[i].name, arr[i].gpa, arr[i].age, arr[i].major);
    }
}

```

```

int main() {
    int n;
    scanf("%d", &n);
    Student arr[n];

    for (int i = 0; i < n; i++) {
        scanf("%s", arr[i].name);
        scanf("%f", &arr[i].gpa);
        scanf("%d", &arr[i].age);
        scanf("%s", arr[i].major);
    }

    scanf("%s", criterion);

    quickSort(arr, 0, n - 1);

    printf("Sorted Student Records:\n");
    printStudents(arr, n);

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Ragavi, a computer scientist, is working on efficient text processing for search engines. She has a list of words in random order and needs to sort them in lexicographic order using Heap Sort to improve search performance.

Help Ragavi implement Heap Sort to arrange the words in lexicographic order efficiently.

Input Format

The first line consists of an integer N denoting the number of strings.

The next line consists of N space-separated strings.

Output Format

The output displays N space-separated sorted strings in lexicographic order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 10

banana pineapple mango chikoo jack star guava lemon tomato orange

Output: banana chikoo guava jack lemon mango orange pineapple star tomato

Answer

```
#include <stdio.h>
#include <string.h>
void swap(char *a, char *b) {
    char temp[100];
    strcpy(temp, a);
    strcpy(a, b);
    strcpy(b, temp);
}
void heapify(char arr[][100], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n && strcmp(arr[left], arr[largest]) > 0)
        largest = left;

    if (right < n && strcmp(arr[right], arr[largest]) > 0)
        largest = right;
```



```

        if (largest != i) {
            swap(arr[i], arr[largest]);
            heapify(arr, n, largest);
        }
    }
}

void heapSort(char arr[][100], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--) {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    char arr[20][100];
    for (int i = 0; i < n; i++) {
        scanf("%s", arr[i]);
    }

    heapSort(arr, n);

    for (int i = 0; i < n; i++) {
        printf("%s ", arr[i]);
    }
    printf("\n");

    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Imagine you are working as a librarian in a large library. The library uses an automated system to keep track of book positions on the shelves. Each book is assigned a unique ID, but some IDs are more common than others due to the popularity of certain editions.

The system logs the positions of books in a sorted list based on their IDs. You need to find the first and last positions of a specific book ID in this sorted list using recursive binary search.

Example

Input:

10

1 2 2 2 2 3 4 8 8 8

8

Output:

7 9

Explanation:

The first and last positions of book ID 8 are [7, 9] since the index is 0-based.

Input Format

The first line of input consists of an integer n , representing the total number of book IDs.

The second line consists of a sorted list of n integers representing the book IDs.

The third line consists of an integer x , representing the specific book ID to find.

Output Format

The output displays indexes of the first and last occurrence of x , as space-separated integers.

If book ID x is not present in the shelves, print "NO OCCURRENCES".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10

1 2 2 2 2 3 4 8 8 8

8

Output: 7 9

Answer

```
#include <stdio.h>
```

```
int firstOccurrence(int arr[], int low, int high, int x) {
```

```
    if (high >= low) {
```

```
        int mid = low + (high - low) / 2;
```

```
        if ((mid == 0 || x > arr[mid - 1]) && arr[mid] == x)
```

```
            return mid;
```

```
        else if (x > arr[mid])
```

```
            return firstOccurrence(arr, mid + 1, high, x);
```

```
        else
```

```
            return firstOccurrence(arr, low, mid - 1, x);
```

```
    }
```

```
    return -1;
```

```
}
```

```
int lastOccurrence(int arr[], int low, int high, int n, int x) {
```

```
    if (high >= low) {
```

```
        int mid = low + (high - low) / 2;
```

```
        if ((mid == n - 1 || x < arr[mid + 1]) && arr[mid] == x)
```

```
            return mid;
```

```
        else if (x < arr[mid])
```

```
            return lastOccurrence(arr, low, mid - 1, n, x);
```

```
        else
```

```
            return lastOccurrence(arr, mid + 1, high, n, x);
```

```
    }
```

```
    return -1;
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int arr[25];
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
int x;  
scanf("%d", &x);  
  
int first = firstOccurrence(arr, 0, n - 1, x);  
int last = lastOccurrence(arr, 0, n - 1, n, x);  
  
if (first == -1 || last == -1) {  
    printf("NO OCCURRENCES\n");  
} else {  
    printf("%d %d\n", first, last);  
}  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 5_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 14

Section 1 : MCQ

1. In heap sort, when does the actual sorting take place?

Answer

During the swap and heapify phase

Status : Correct

Marks : 1/1

2. In heap sort, which of the following heap operations is performed to build a valid heap from an unsorted array?

Answer

Heapify

Status : Correct

Marks : 1/1

3. In heap sort, the element that is swapped with the final element of the heap is _____, in the case of a max-heap, sorted in descending order.

Answer

Largest element

Status : Correct

Marks : 1/1

4. What is the worst-case time complexity of the heap sort algorithm?

Answer

$O(n \log n)$

Status : Correct

Marks : 1/1

5. In a quick sort algorithm, where are the smaller elements placed to the pivot during the partition process, assuming we are sorting in increasing order?

Answer

To the left of the pivot

Status : Correct

Marks : 1/1

6. Which of the following is true about Quicksort?

Answer

It is an in-place sorting algorithm

Status : Correct

Marks : 1/1

7. What is the worst-case time complexity of the Quicksort algorithm?

Answer

$O(n^2)$

Status : Correct

Marks : 1/1

8. Given a sorted array of 1000 unique integers, how many maximum comparisons does Binary Search make in the worst case?

Answer

10

Status : Correct

Marks : 1/1

9. Suppose you are using Binary Search on a sorted array of even length (e.g., 8 elements). At each step, you calculate the middle as:

$\text{mid} = (\text{low} + \text{high}) // 2$

Which of the following statements is always true?

Answer

The mid index may not divide the array into equal halves in all steps.

Status : Correct

Marks : 1/1

10. You apply Binary Search on the sorted array [3, 7, 10, 15, 21, 30] to find the element 13. Which sequence of middle elements will Binary Search compare with before declaring that the element is not present?

Answer

10 21 15

Status : Correct

Marks : 1/1

11. You are using Binary Search to find the first occurrence of a value x in a sorted array that may contain duplicates. Which of the following modifications is necessary?

Answer

When $x == \text{arr}[\text{mid}]$, move high to mid - 1.

Status : Correct

Marks : 1/1

12. A student implements Binary Search and calculates the middle index as:

$\text{mid} = (\text{low} + \text{high}) // 2$

But when the input values of low and high are both large (e.g., low = 2_000_000_000, high = 2_000_000_010), the program crashes due to integer overflow. What is the best fix?

Answer

Use $\text{mid} = \text{low} + (\text{high} - \text{low}) // 2$

Status : Correct

Marks : 1/1

13. Is Merge Sort a stable sorting algorithm?

Answer

Yes, always stable.

Status : Correct

Marks : 1/1

14. Merge sort is _____.

Answer

Non-comparison sorting algorithm

Status : Wrong

Marks : 0/1

15. Which of the following methods is used for sorting in merge sort?

Answer

merging

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 6_COD

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Ankit wants to share the job among his employees so that maximum profit can be obtained. The condition is that no two jobs for a particular employee overlap.

Given N jobs, where every job is represented by the following three elements:

Start Time

Finish Time

Profit or Value Associated (≥ 0)

Find the maximum-profit subset of jobs such that no two jobs in the subset overlap.

Input Format

The first line of the input consists of the value of n.

The next n lines of the inputs consist of the start, end, and profit.

Output Format

The output prints the optimal profit.

Refer to the sample input and output for format specifications.

Sample Test Case

Input: 4

3 10 20

1 2 50

6 19 100

2 100 200

Output: The optimal profit is 250

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Job {  
    int start, finish, profit;  
};
```

```
int compare(const void *a, const void *b) {  
    struct Job *job1 = (struct Job *)a;  
    struct Job *job2 = (struct Job *)b;  
    return job1->finish - job2->finish;  
}
```

```
int latestNonConflict(struct Job jobs[], int i) {  
    for (int j = i - 1; j >= 0; j--) {  
        if (jobs[j].finish <= jobs[i].start)
```

```
        return j;
    }
    return -1;
}
```

```
int findMaxProfit(struct Job jobs[], int n) {
    qsort(jobs, n, sizeof(struct Job), compare);

    int dp[n];

    dp[0] = jobs[0].profit;

    for (int i = 1; i < n; i++) {
        int includeProfit = jobs[i].profit;

        int l = latestNonConflict(jobs, i);
        if (l != -1)
            includeProfit += dp[l];

        dp[i] = (includeProfit > dp[i - 1]) ? includeProfit : dp[i - 1];
    }

    return dp[n - 1];
}
```

```
int main() {
    int n;
    scanf("%d", &n);

    struct Job jobs[n];

    for (int i = 0; i < n; i++) {
        scanf("%d %d %d", &jobs[i].start, &jobs[i].finish, &jobs[i].profit);
    }

    int result = findMaxProfit(jobs, n);

    printf("The optimal profit is %d", result);

    return 0;
}
```

}

Status : Correct

Marks : 10/10

2. Problem Statement

In a city, there are several locations connected by roads. Each road has a certain distance. You are tasked with finding the shortest distances between all pairs of locations in the city using the Floyd-Warshall algorithm.

Write a program that calculates the shortest path between all pairs of locations in the city.

Example

Input:

4

4

0 1 3

1 2 1

2 3 1

0 2 5

Output:

0 3 4 5

3 0 1 2

4 1 0 1

5 2 1 0

Explanation:

The city has 4 locations (0 to 3).

There are 4 roads connecting the locations.

The roads and their distances are:

- 0 1 with distance 3
- 1 2 with distance 1
- 2 3 with distance 1
- 0 2 with distance 5

Initially, the distance matrix is set to infinity (INF) except for direct roads and diagonal values (0).

The Floyd-Warshall algorithm updates the matrix by checking intermediate paths.

The shortest path from 0 to 2 (via 1) is updated to 4 (0 1 2) instead of 5.

The shortest path from 0 to 3 is 5 (0 1 2 3).

The final distance matrix is printed, showing the shortest distances.

0 3 4 5

3 0 1 2

4 1 0 1

5 2 1 0

Input Format

The first line contains an integer N, representing the number of locations in the city.

The second line contains an integer M, representing the number of roads in the city.

The next M lines each contain three space-separated integers: u, v, w where:

- u and v represent two locations connected by a road.
- w represents the distance between u and v.

Output Format

The output prints a matrix, where the value at the i-th row and j-th column represents the shortest distance between location i and location j.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

4

0 1 3

1 2 1

2 3 1

0 2 5

Output: 0 3 4 5

3 0 1 2

4 1 0 1

5 2 1 0

Answer

```
#include <stdio.h>
```

```
#define INF 100000
```

```
int main() {
```

```
    int n, m;
```

```
    scanf("%d", &n);
```

```
    scanf("%d", &m);
```

```
    int dist[n][n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < n; j++) {
```

```
            if (i == j)
```

```
                dist[i][j] = 0;
```

```
            else
```

```
                dist[i][j] = INF;
```

```
        }
```

```
    }
```

```
    for (int i = 0; i < m; i++) {
```

```
        int u, v, w;
```

```
        scanf("%d %d %d", &u, &v, &w);
```

```
        dist[u][v] = w;
```

```

        dist[v][u] = w;
    }

    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("%d ", dist[i][j]);
        }
        printf("\n");
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Sharon is developing a program that analyzes the reachability of nodes in a graph. Given an adjacency matrix representing the graph and its size, her goal is to determine whether there exists a path from one vertex to another for all pairs of vertices.

Write a program to help Sharon perform this reachability analysis using Warshall's Algorithm.

Input Format

The first line of input consists of an integer N, representing the number of vertices in the graph.

The following N lines consist of N space-separated integers, forming the

adjacency matrix graph, where graph[i][j] is either 0 or 1.

Output Format

The output prints an NxN matrix representing the reachability matrix for all pairs of vertices.

Each element in the matrix should be 1 if there is a path from the corresponding row vertex to the column vertex, and 0 otherwise.

Sample Test Case

Input: 4

0 1 0 0

0 0 1 0

0 0 0 1

0 0 0 0

Output: 0 1 1 1

0 0 1 1

0 0 0 1

0 0 0 0

Answer

```
#include <stdio.h>
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int graph[n][n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < n; j++) {
```

```
            scanf("%d", &graph[i][j]);
```

```
        }
```

```
    }
```

```
    for (int k = 0; k < n; k++) {
```

```
        for (int i = 0; i < n; i++) {
```

```
            for (int j = 0; j < n; j++) {
```

```
                graph[i][j] = graph[i][j] || (graph[i][k] && graph[k][j]);
```

```
            }
```

```
        }
```

```
    }
```



```

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        printf("%d ", graph[i][j]);
    }
    printf("\n");
}

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

You and your group of adventurous friends are planning a once-in-a-lifetime outdoor adventure trip to an exotic location. To ensure that your adventure goes smoothly, you need to carefully choose what to carry with you, as you have a limited carrying capacity. Your goal is to maximize the overall value of the items you bring while staying within your weight limit.

Write a program that helps you and your friends determine the optimal selection of items to bring on your adventure trip. The program should implement the 0–1 Knapsack Problem algorithm to find the best combination of items to maximize their total value while respecting the weight constraint.

Input Format

The first line contains an integer n , representing the number of items available for selection.

The second line contains an integer limit, representing the maximum weight limit your group can carry in a bag.

The next n lines contain space-separated information for each item:

- The name of the item is a string (without spaces).
- The weight of the item.
- The value of the item.

Output Format

The first line of output displays "Total value: " followed by an integer representing the total value of the selected items.

The second line of output displays "Total weight: " followed by an integer representing the total weight of the selected items.

Refer to the sample outputs for the exact format.

Sample Test Case

Input: 2

25

Bag 10 250

Note 15 430

Output: Total value: 680

Total weight: 25

Answer

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    int n, limit;
```

```
    scanf("%d", &n);
```

```
    scanf("%d", &limit);
```

```
    char name[n][51];
```

```
    int weight[n], value[n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%s %d %d", name[i], &weight[i], &value[i]);
```

```
    }
```

```
    int dp[n + 1][limit + 1];
```

```
    for (int i = 0; i <= n; i++) {
```

```
        for (int w = 0; w <= limit; w++) {
```

```
            if (i == 0 || w == 0)
```

```

        dp[i][w] = 0;
    else if (weight[i - 1] <= w) {
        int include = value[i - 1] + dp[i - 1][w - weight[i - 1]];
        int exclude = dp[i - 1][w];
        dp[i][w] = (include > exclude) ? include : exclude;
    } else {
        dp[i][w] = dp[i - 1][w];
    }
}
}

int totalValue = dp[n][limit];

int w = limit;
int totalWeight = 0;

for (int i = n; i > 0; i--) {
    if (dp[i][w] != dp[i - 1][w]) {
        totalWeight += weight[i - 1];
        w -= weight[i - 1];
    }
}

printf("Total value: %d\n", totalValue);
printf("Total weight: %d", totalWeight);

return 0;
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 6_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Virat is given a group of individuals, and he wants to identify if there is an influential individual within the group. An influential individual is defined as someone who is followed by more people than they follow.

He is provided with a list of follow-up relationships among the individuals in the form of a matrix. Your task is to help Virat determine if there is an influential individual and, if so, find their index within the group.

Write a program to solve this problem using Warshall's Algorithm.

Input Format

The first line of input consists of an integer N, representing the number of individuals in the group.

The following N lines will contain N space-separated integers, representing the following relationship matrix for the individuals.

Output Format

If an influential individual is found, print "Influential Individual: X," where X is the index (0-based) of the influential individual.

Otherwise, print "There is no influential individual in the group".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 1 1 0

0 0 1 0

0 0 0 0

0 0 1 0

Output: Influential Individual: 2

Answer

```
#include <stdio.h>
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int graph[n][n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < n; j++) {
```

```
            scanf("%d", &graph[i][j]);
```

```
        }
```

```
    }
```

```
    for (int k = 0; k < n; k++) {
```

```
        for (int i = 0; i < n; i++) {
```

```
            for (int j = 0; j < n; j++) {
```

```
                graph[i][j] = graph[i][j] || (graph[i][k] && graph[k][j]);
```

```

    }
    }
}

int found = 0;

for (int i = 0; i < n; i++) {
    int followsAnyone = 0;
    int followedByAll = 1;

    for (int j = 0; j < n; j++) {
        if (graph[i][j] == 1) {
            followsAnyone = 1;
            break;
        }
    }

    for (int j = 0; j < n; j++) {
        if (i != j && graph[j][i] == 0) {
            followedByAll = 0;
            break;
        }
    }

    if (!followsAnyone && followedByAll) {
        printf("Influential Individual: %d", i);
        found = 1;
        break;
    }
}

if (!found) {
    printf("There is no influential individual in the group");
}

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem statement

Sarah has been tasked with developing a program to determine the shortest paths among nodes in a directed network represented as a graph containing nodes and weighted edges.

Her goal is to implement the Floyd-Warshall algorithm to compute the shortest path matrix for this directed network.

Input Format

The first line of input consists of an integer n , representing the number of nodes in the network.

The next n lines contain a space-separated integer, representing the adjacency matrix 'graph' of the network. Each integer is the weight of the edge from node i to node j .

Output Format

The output prints the shortest path matrix as ' $n \times n$ ' integers, where each element $\text{graph}[i][j]$ represents the shortest distance from node ' i ' to node ' j '.

Print each row of the matrix on a new line, with elements separated by a space, and if no path exists, 999 will be displayed.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
0 3 999 4
8 0 2 999
5 999 0 1
2 999 999 0
Output: 0 3 5 4
5 0 2 3
3 6 0 1
2 5 7 0

Answer

```
#include <stdio.h>
```

```
#define INF 999
```

```
int main() {  
    int n;  
    scanf("%d", &n);
```

```
    int dist[n][n];
```

```
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) {  
            scanf("%d", &dist[i][j]);  
        }  
    }
```

```
    for (int k = 0; k < n; k++) {  
        for (int i = 0; i < n; i++) {  
            for (int j = 0; j < n; j++) {  
                if (dist[i][k] != INF && dist[k][j] != INF) {  
                    if (dist[i][k] + dist[k][j] < dist[i][j]) {  
                        dist[i][j] = dist[i][k] + dist[k][j];  
                    }  
                }  
            }  
        }  
    }
```

```
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) {  
            printf("%d ", dist[i][j]);  
        }  
        printf("\n");  
    }
```

```
    return 0;
```

```
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

You are working as a backend engineer for a drone delivery logistics company that operates across multiple cities. Each city is represented as a node, and the drone flight paths between cities are represented as directed edges with weights indicating the time (in minutes) it takes for a drone to travel between them.

Due to varying air traffic, regulations, and geography, the direct travel time between cities may not always be the fastest. Sometimes, going through intermediate cities results in shorter delivery times.

Your task is to build a system that calculates the shortest possible delivery time between every pair of cities, considering all possible intermediate routes.

To efficiently compute this, you're required to implement the Floyd-Warshall Algorithm.

Input Format

The first line of input contains an integer N representing the total number of cities.

The next N lines contains N space-separated integers, representing the delivery time matrix:

$\text{time}[i][j]$ is the time (in minutes) taken to fly directly from City i to City j .

A value of 0 means the same city (i.e., $i == j$).

A value of 999 indicates no direct route between City i and City j .

Output Format

The output prints the shortest distance between every pair of cities.

If there is no shortest path between two cities, then print INF.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

0 5 999 10

999 0 3 999

999 999 0 1

999 999 999 0

Output: 0 5 8 9

INF 0 3 4

INF INF 0 1

INF INF INF 0

Answer

```
#include <stdio.h>
```

```
#define INF 999
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int dist[n][n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < n; j++) {
```

```
            scanf("%d", &dist[i][j]);
```

```
        }
```

```
    }
```

```
    for (int k = 0; k < n; k++) {
```

```
        for (int i = 0; i < n; i++) {
```

```
            for (int j = 0; j < n; j++) {
```

```
                if (dist[i][k] != INF && dist[k][j] != INF) {
```

```
                    if (dist[i][k] + dist[k][j] < dist[i][j]) {
```

```
                        dist[i][j] = dist[i][k] + dist[k][j];
```

```
                    }
```

```
                }
```

```
            }
```

```
        }
```

```
    }
```

```

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        if (dist[i][j] == INF)
            printf("INF ");
        else
            printf("%d ", dist[i][j]);
    }
    printf("\n");
}

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Given the weights and values of n items, put these items in a knapsack of capacity W to get the maximum total value in the knapsack. In other words, given two integer arrays, $val[0..n-1]$ and $wt[0..n-1]$ which represent values and weights associated with n items, respectively. Also given an integer W which represents knapsack capacity, find out the maximum value subset of $val[]$ such that the sum of the weights of this subset is smaller than or equal to W . You cannot break an item, either pick the complete item or don't pick it.

Solve this problem using dynamic programming.

Input Format

The first line of input represents the number of items.

The second line of input consists of the values of each item separated by spaces.

The third line of input consists of the weights of each item separated by spaces.

The fourth line of input consists of an integer, which represents the maximum capacity of the knapsack.

Output Format

The output prints the maximum value of items that can be held by the knapsack.

Sample Test Case

Input: 3
60 100 120
10 20 30
50

Output: 220

Answer

```
#include <stdio.h>
```

```
int main() {  
    int n, W;  
    scanf("%d", &n);  
  
    int val[n], wt[n];  
  
    for (int i = 0; i < n; i++) {  
        scanf("%d", &val[i]);  
    }  
  
    for (int i = 0; i < n; i++) {  
        scanf("%d", &wt[i]);  
    }  
  
    scanf("%d", &W);  
  
    int dp[n + 1][W + 1];  
  
    for (int i = 0; i <= n; i++) {  
        for (int w = 0; w <= W; w++) {  
            if (i == 0 || w == 0)  
                dp[i][w] = 0;  
            else if (wt[i - 1] <= w) {  
                int include = val[i - 1] + dp[i - 1][w - wt[i - 1]];  
                int exclude = dp[i - 1][w];  
                dp[i][w] = (include > exclude) ? include : exclude;  
            } else {  
                dp[i][w] = dp[i - 1][w];  
            }  
        }  
    }  
}
```

```
}  
    printf("%d", dp[n][W]);  
    return 0;  
}
```

Status : Correct

Marks : 10/10

5. Problem Statement

Sarah loves challenges and has recently taken up a new programming problem. She is fascinated by the classic "0/1 Knapsack Problem" and wants to create a program that can efficiently solve it.

The 0/1 Knapsack Problem involves selecting a combination of items with given weights and values to maximize the total value, while not exceeding a given weight capacity.

Write a program that takes input for the 0/1 Knapsack Problem and outputs the maximum value that can be achieved, as well as the selected weights that contribute to this maximum value.

Note: The weights are printed in reverse order of the input, meaning the algorithm starts checking from the last item and prints items that are included first.

Input Format

The first line contains an integer n , representing the number of items available.

The second line contains n integers separated by space, representing the values of the items. The i th integer corresponds to the value of the i th item.

The third line contains n integers separated by space, representing the weights of the items. The i th integer corresponds to the weight of the i th item.

The fourth line contains an integer W , representing the maximum weight capacity of the knapsack.

Output Format

The first line output displays "Maximum value: " followed by an integer, representing the maximum value that can be achieved.

The second line output displays "Selected weights: " followed by space-separated integers, representing the selected weights contributing to the maximum value.

Refer to the sample outputs for the exact format.

Sample Test Case

Input: 3

60 100 120

10 20 30

50

Output: Maximum value: 220

Selected weights: 30 20

Answer

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, W;
```

```
    scanf("%d", &n);
```

```
    int val[n], wt[n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &val[i]);
```

```
    }
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &wt[i]);
```

```
    }
```

```
    scanf("%d", &W);
```

```
    int dp[n + 1][W + 1];
```

```

for (int i = 0; i <= n; i++) {
    for (int w = 0; w <= W; w++) {
        if (i == 0 || w == 0)
            dp[i][w] = 0;
        else if (wt[i - 1] <= w) {
            int include = val[i - 1] + dp[i - 1][w - wt[i - 1]];
            int exclude = dp[i - 1][w];
            dp[i][w] = (include > exclude) ? include : exclude;
        } else {
            dp[i][w] = dp[i - 1][w];
        }
    }
}

printf("Maximum value: %d\n", dp[n][W]);

int w = W;
printf("Selected weights: ");
for (int i = n; i > 0; i--) {
    if (dp[i][w] != dp[i - 1][w]) {
        printf("%d ", wt[i - 1]);
        w -= wt[i - 1];
    }
}

return 0;
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 6_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 15

Section 1 : MCQ

1. Which of the following is/are property/properties of a dynamic programming problem?

Answer

Both optimal substructure and overlapping subproblems

Status : Correct

Marks : 1/1

2. If an optimal solution can be created for a problem by constructing optimal solutions for its subproblems, the problem possesses _____ property.

Answer

Optimal substructure

Status : Correct

Marks : 1/1

3. In dynamic programming, the technique of storing the previously calculated values is called _____.

Answer

Memoization

Status : Correct

Marks : 1/1

4. The following paradigm can be used to find the solution to the problem in minimum time:

Given a set of non-negative integers, and a value K, determine if there is a subset of the given set with a sum equal to K.

Answer

Dynamic Programming

Status : Correct

Marks : 1/1

5. Which of the following is an example of a problem that can be solved using memoization in dynamic programming?

Answer

Computing the edit distance between two strings

Status : Correct

Marks : 1/1

6. The Floyd-Warshall algorithm for all-pairs shortest paths computation is based on

Answer

Dynamic Programming paradigm

Status : Correct

Marks : 1/1

7. Floyd-Warshall algorithm utilizes _____ to solve the all-pairs shortest paths problem on a directed graph in _____ time.

Answer

Dynamic programming, $\theta(V^3)$

Status : Correct

Marks : 1/1

8. Floyd Warshall's Algorithm can be applied to _____.

Answer

Directed graphs

Status : Correct

Marks : 1/1

9. In the Floyd Warshall algorithm, how many iterations are required to find the shortest path between all pairs of vertices in a graph with N vertices?

Answer

N

Status : Correct

Marks : 1/1

10. Consider a directed weighted graph with V vertices and E edges, where each edge weight is represented as a 32-bit signed integer. What will be the maximum possible value for the edge weight to guarantee the correct shortest path calculation using the Floyd Warshall algorithm?

Answer

$2^{31} - 1$

Status : Correct

Marks : 1/1

11. 0/1 knapsack is based on _____ method.

Answer

Dynamic programming

Status : Correct

Marks : 1/1

12. Fractional knapsack problem differs from 0/1 knapsack because:

Answer

Items can be divided into fractions

Status : Correct

Marks : 1/1

13. Which of the following methods can be used to solve the Knapsack problem?

Answer

All the mentioned options

Status : Correct

Marks : 1/1

14. What is the primary advantage of using the Branch and Bound approach for the Knapsack problem?

Answer

It systematically explores possible solutions while pruning non-promising ones.

Status : Correct

Marks : 1/1

15. Which of the following problems cannot be solved optimally by greedy approach?

Answer

0/1 Knapsack

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 7_COD

Attempt : 1

Total Mark : 30

Marks Obtained : 20

Section 1 : Coding

1. Problem Statement

Prim's Algorithm for Minimum Spanning Tree (MST):

Sita is planning to lay roads to connect 5 locations in her city. The distances between locations are given as an adjacency matrix, where a value of 0 indicates no direct road between two locations (locations are numbered 0 to 4).

Help her find the Minimum Spanning Tree (MST) using Prim's Algorithm so that all locations are connected with the minimum total road length. Print each MST edge and its weight in the order they are added to the MST.

Input Format

The input consists of 5 lines, each containing 5 space-separated integers,

representing the adjacency matrix of the graph. A value of 0 indicates no direct connection between two locations.

Output Format

The first line of output prints the header: 'Edge \tWeight'

Each of the next V-1 lines prints one MST edge in the format:

'u - v\tw'

where u is the parent location, v is the child location, and w is the integer edge weight, printed in the order edges are added to the MST (MST discovery order).

Refer to the sample output for formatting specifications.

Refer to the sample output for format specifications.

Sample Test Case

Input: 0 2 0 6 0

2 0 3 8 5

0 3 0 8 7

6 8 0 0 9

0 5 7 9 0

Output: Edge Weight

0 - 1 2

1 - 2 3

1 - 4 5

0 - 3 6

Answer

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#define V 5
```

```
int minKey(int key[], int mstSet[]) {
```

```

    int min = INT_MAX, min_index;
    for (int v = 0; v < V; v++) {
        if (mstSet[v] == 0 && key[v] < min) {
            min = key[v];
            min_index = v;
        }
    }
    return min_index;
}

```

```

int main() {
    int graph[V][V];

    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }
}

```

```

int parent[V];
int key[V];
int mstSet[V];

```

```

for (int i = 0; i < V; i++) {
    key[i] = INT_MAX;
    mstSet[i] = 0;
}

```

```

key[0] = 0;
parent[0] = -1;

```

```

printf("Edge \tWeight\n");

```

```

for (int count = 0; count < V; count++) {
    int u = minKey(key, mstSet);
    mstSet[u] = 1;
}

```

```

    if (parent[u] != -1) {

```

```

    printf("%d - %d\t%d\n", parent[u], u, graph[u][parent[u]]);
}

for (int v = 0; v < V; v++) {
    if (graph[u][v] != 0 && mstSet[v] == 0 && graph[u][v] < key[v]) {
        parent[v] = u;
        key[v] = graph[u][v];
    }
}
}

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem statement

Alex is a network engineer tasked with laying cables to connect multiple offices in the city. Each possible cable connection between two offices has an associated cost. To minimize the total cabling cost while ensuring all offices are connected, Alex decides to use Kruskal's Algorithm to find the Minimum Spanning Tree (MST) of the office network.

Given a connected, undirected, weighted graph with V vertices (numbered 0 to $V-1$) and E edges, find the MST and print each edge included along with the total minimum cost.

Input Format

The first line contains an integer V , representing the number of vertices (numbered 0 to $V-1$).

The second line contains an integer E , representing the number of edges.

Each of the next E lines contains three space-separated integers u , v , and w , representing an undirected edge between vertex u and vertex v with weight w .

Output Format

The first line of output prints: 'Edges in the constructed MST:'

Each of the next V-1 lines prints one MST edge in the format:

'u -- v == weight'

where u and v are vertex numbers (integers) and weight is the edge cost (integer), printed in the order edges are added to the MST (ascending weight order).

The last line prints: 'Minimum Spanning Tree: totalCost'

where totalCost is an integer representing the total MST weight.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

3

0 1 2

1 2 2

0 2 2

Output: Edges in the constructed MST:

0 -- 1 == 2

1 -- 2 == 2

Minimum Spanning Tree: 4

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Edge {  
    int u, v, w;  
};
```

```
int parent[105];
```



```
int find(int i) {  
    if (parent[i] != i)  
        parent[i] = find(parent[i]);  
    return parent[i];  
}
```

```
void unionSet(int x, int y) {  
    int px = find(x);  
    int py = find(y);  
    parent[px] = py;  
}
```

```
int compare(const void *a, const void *b) {  
    struct Edge *e1 = (struct Edge *)a;  
    struct Edge *e2 = (struct Edge *)b;  
    return e1->w - e2->w;  
}
```

```
int main() {  
    int V, E;  
    scanf("%d", &V);  
    scanf("%d", &E);  
  
    struct Edge edges[10000];  
  
    for (int i = 0; i < E; i++) {  
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);  
    }  
  
    qsort(edges, E, sizeof(struct Edge), compare);  
  
    for (int i = 0; i < V; i++) {  
        parent[i] = i;  
    }  
  
    int count = 0;  
    int totalCost = 0;  
  
    printf("Edges in the constructed MST:\n");  
    for (int i = 0; i < E && count < V - 1; i++) {
```

```

int u = edges[i].u;
int v = edges[i].v;
int w = edges[i].w;

if (find(u) != find(v)) {
    printf("%d -- %d == %d\n", u, v, w);
    totalCost += w;
    unionSet(u, v);
    count++;
}
}

printf("Minimum Spanning Tree: %d", totalCost);

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

You are designing a data compression tool that uses Huffman Coding to encode data efficiently.

Write a program to read a set of characters along with their corresponding Huffman variable-length binary codes, and use them to generate the encoded binary string for a given input message.

It is guaranteed that every character in the input message has a corresponding Huffman code in the provided code table.

Input Format

The first line contains an integer N (integer), representing the number of characters in the code table.

Each of the next N lines contains a character C and its corresponding Huffman code (a binary string of 0s and 1s), separated by a space.

The last line contains the input message (a string of characters) that needs to be encoded. Every character in the message is guaranteed to have a corresponding

entry in the code table.

Output Format

The output prints 'Coded Data: ' followed by a binary string (a sequence of 0s and 1s) representing the encoded message, obtained by concatenating the Huffman code of each character in the input message in order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 10

D 0100

U 0000

F 0101

M 0001

A 0010

N 0011

C 1100

O 1101

E 1110

H 1111

HUFF

Output: Coded Data: 1111000001010101

Answer

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    int N;
```

```
    scanf("%d", &N);
```

```
    char codes[26][21];
```

```
    char ch;
```

```
    for (int i = 0; i < 26; i++) {
```

```
        codes[i][0] = '\0';
```

```
    }
```

```
for (int i = 0; i < N; i++) {  
    scanf(" %c %s", &ch, codes[ch - 'A']);  
}  
  
char message[105];  
scanf("%s", message);  
  
printf("Coded Data: ");  
  
for (int i = 0; message[i] != '\0'; i++) {  
    printf("%s", codes[message[i] - 'A']);  
}  
  
return 0;  
}
```

Status : Wrong

Marks : 0/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 7_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Mary is planning to build a railway network to connect cities efficiently. She needs a program that, given the number of cities and the segments between them, determines the minimum cost to connect all cities using an undirected weighted graph.

Implement a solution using Kruskal's Algorithm to find the Minimum Spanning Tree (MST) of the city network. If it is not possible to connect all cities, output an appropriate message

Example

Input

3

3

0 1 4

1 2 5

0 2 3

Output

7

Explanation

The input specifies 3 cities and 3 track segments with their corresponding weights.

The track segments are:

Track Segment 1: Connects city 0 and city 1, with a weight of 4.

Track Segment 2: Connects city 1 and city 2 with a weight of 5.

Track Segment 3: Connects city 0 and city 2 with a weight of 3.

The algorithm finds the minimum cost to build railway tracks connecting all cities. In this case, the minimum spanning tree includes track segment 3 (0-2 with weight 3) and track segment 1 (0-1 with weight 4). The total cost is $3 + 4 = 7$.

Input Format

The first line of input contains an integer `num_cities`, representing the number of cities (numbered 0 to `num_cities - 1`).

The second line contains an integer `num_segments`, representing the number of undirected segments (edges) between cities.

The next `num_segments` lines each contain three space-separated integers: source, destination, and weight, representing an undirected segment between city source and city destination with the given cost.

Output Format

The first line of output prints an integer representing the minimum cost to connect all cities.

If it is not possible to connect all the cities, the output prints 'It is not possible to connect all cities' (without a period).

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

3

0 1 4

1 2 5

0 2 3

Output: 7

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Edge {  
    int u, v, w;  
};
```

```
int parent[20];
```

```
int find(int x) {  
    if (parent[x] != x)  
        parent[x] = find(parent[x]);  
    return parent[x];  
}
```

```
void unionSet(int a, int b) {  
    int pa = find(a);  
    int pb = find(b);  
    parent[pa] = pb;  
}
```

```
int compare(const void *a, const void *b) {  
    struct Edge *e1 = (struct Edge *)a;
```

```

    struct Edge *e2 = (struct Edge *)b;
    return e1->w - e2->w;
}

int main() {
    int n, m;
    scanf("%d", &n);
    scanf("%d", &m);

    struct Edge edges[20];

    for (int i = 0; i < m; i++) {
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
    }

    qsort(edges, m, sizeof(struct Edge), compare);

    for (int i = 0; i < n; i++) {
        parent[i] = i;
    }

    int totalCost = 0;
    int edgeCount = 0;

    for (int i = 0; i < m; i++) {
        if (find(edges[i].u) != find(edges[i].v)) {
            unionSet(edges[i].u, edges[i].v);
            totalCost += edges[i].w;
            edgeCount++;
        }
    }

    if (edgeCount == n - 1) {
        printf("%d", totalCost);
    } else {
        printf("It is not possible to connect all cities");
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Maya, a project planner, must connect multiple office branches using network cables while keeping the total cost within a given budget. Each possible connection between two branches has an associated cost.

Write a program to find the Minimum Spanning Tree (MST) of the branch network using Kruskal's Algorithm on an undirected weighted graph. If the total cost of the MST is within the given budget, display the cost. Otherwise, indicate that it is not possible.

It is guaranteed that the graph is connected in all test cases

Input Format

The first line contains three space-separated integers V, E, and budget, representing the number of branches (vertices, numbered 1 to V), the number of possible connections (edges), and the total budget respectively.

Each of the next E lines contains three space-separated integers src, dest, and weight, representing an undirected connection between branch src and branch dest with cost weight.

Output Format

If the MST cost is within the budget, the output prints:

'Minimum Cost Spanning Tree within the budget: X'

where X is an integer representing the total minimum cost.

Otherwise, the output prints:

'A valid MST within the budget is not possible.'

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3 3 5
1 2 2

2 3 3
3 1 4

Output: Minimum Cost Spanning Tree within the budget: 5

Answer

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Edge {
    int u, v, w;
};
```

```
int parent[105];
```

```
int find(int x) {
    if (parent[x] != x)
        parent[x] = find(parent[x]);
    return parent[x];
}
```

```
void unionSet(int a, int b) {
    int pa = find(a);
    int pb = find(b);
    parent[pa] = pb;
}
```

```
int compare(const void *a, const void *b) {
    struct Edge *e1 = (struct Edge *)a;
    struct Edge *e2 = (struct Edge *)b;
    return e1->w - e2->w;
}
```

```
int main() {
    int V, E;
    long long budget;
    scanf("%d %d %lld", &V, &E, &budget);
```

```
    struct Edge edges[1005];
```

```
    for (int i = 0; i < E; i++) {
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
    }
```

```
qsort(edges, E, sizeof(struct Edge), compare);
```

```
for (int i = 1; i <= V; i++) {  
    parent[i] = i;  
}
```

```
long long totalCost = 0;  
int count = 0;
```

```
for (int i = 0; i < E && count < V - 1; i++) {  
    if (find(edges[i].u) != find(edges[i].v)) {  
        unionSet(edges[i].u, edges[i].v);  
        totalCost += edges[i].w;  
        count++;  
    }  
}
```

```
if (totalCost <= budget) {  
    printf("Minimum Cost Spanning Tree within the budget: %lld", totalCost);  
} else {  
    printf("A valid MST within the budget is not possible.");  
}  
return 0;  
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Sita is planning to connect multiple locations in her city with roads. Each possible road between two locations has a construction distance (in kilometres). The city is represented as an undirected weighted graph using an adjacency matrix, where a value of 0 indicates no direct road between two locations (locations are numbered 0 to numLocations - 1).

To minimise the overall construction distance, she decides to use Prim's Algorithm to construct the Minimum Spanning Tree (MST). Help her find the MST edges and the total minimum distance.

Input Format

The first line of input contains an integer numLocations, representing the number of locations (numbered 0 to numLocations - 1).

The next numLocations lines each contain numLocations space-separated integers representing the adjacency matrix, where graph[i][j] is the distance between location i and location j. A value of 0 indicates no direct road between those two locations.

Output Format

The first line of output prints: 'Location\tDistance'

Each of the next numLocations - 1 lines prints the MST edge in the format:

'parent -> child \t\t\t dist km'

where parent is the parent location number, child is the child location number, and dist is the edge weight (integer) in kilometres.

The last line of output prints: 'Minimum total distance: totalCost km'

where totalCost is an integer.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 2

0 5

5 0

Output: Location Distance

0 -> 1 5 km

Minimum total distance: 5 km

Answer

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#define MAX 10
```

```
int minKey(int key[], int mstSet[], int n) {
```

```
    int min = INT_MAX, min_index = -1;
```

```
    for (int v = 0; v < n; v++) {
```

```
        if (!mstSet[v] && key[v] < min) {
```

```
            min = key[v];
```

```
            min_index = v;
```

```
        }
```

```
    }
```

```
    return min_index;
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int graph[MAX][MAX];
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < n; j++) {
```

```
            scanf("%d", &graph[i][j]);
```

```
        }
```

```
    }
```

```
    int parent[MAX];
```

```
    int key[MAX];
```

```
    int mstSet[MAX];
```

```
    for (int i = 0; i < n; i++) {
```

```
        key[i] = INT_MAX;
```

```
        mstSet[i] = 0;
```

```

    }
    key[0] = 0;
    parent[0] = -1;

    for (int count = 0; count < n - 1; count++) {
        int u = minKey(key, mstSet, n);
        mstSet[u] = 1;

        for (int v = 0; v < n; v++) {
            if (graph[u][v] != 0 && !mstSet[v] && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }

    printf("Location\tDistance\n");

    int totalCost = 0;

    for (int i = 1; i < n; i++) {
        printf("%d -> %d\t\t%d km\n", parent[i], i, graph[i][parent[i]]);
        totalCost += graph[i][parent[i]];
    }

    printf("Minimum total distance: %d km", totalCost);

    return 0;
}

```

Status : Wrong

Marks : 0/10

4. Problem Statement

Riya works at a data tools company that needs a compact representation of frequently used characters for fast transmission. She decides to use Huffman Coding, a lossless compression technique that assigns shorter bit-strings to more frequent symbols and longer bit-strings to less frequent

ones.

The Huffman codes should be printed in the order that leaf nodes are encountered during a left-to-right traversal of the constructed Huffman tree.

Write a program that builds a Huffman tree from a list of symbols with their frequencies and prints the Huffman codes for each symbol.

Input Format

The first line contains an integer n , representing the number of distinct symbols.

Each of the next n lines contains a single printable character (the symbol) followed by a space and a positive integer (the frequency), representing the symbol and its occurrence count.

Output Format

The output prints n lines, one per symbol, in the order that leaf nodes are encountered during a left-to-right traversal of the Huffman tree (left child before right child at every internal node).

Each line prints the symbol followed by a colon, a space, and its Huffman code (a binary string of 0s and 1s):

'<symbol>: <binary_code>'

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 2

a 1

b 2

Output: a: 0

b: 1

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
#define MAX 300
```

```
struct Node {
    char ch;
    int freq;
    struct Node *left, *right;
};
```

```
struct Node* createNode(char ch, int freq) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->ch = ch;
    node->freq = freq;
    node->left = node->right = NULL;
    return node;
}
```

```
struct MinHeap {
    int size;
    struct Node* arr[MAX];
};
```

```
void swap(struct Node** a, struct Node** b) {
    struct Node* t = *a;
    *a = *b;
    *b = t;
}
```

```
void heapify(struct MinHeap* heap, int i) {
    int smallest = i;
    int l = 2*i + 1;
    int r = 2*i + 2;

    if (l < heap->size && heap->arr[l]->freq < heap->arr[smallest]->freq)
        smallest = l;
```



```

if (r < heap->size && heap->arr[r]->freq < heap->arr[smallest]->freq)
    smallest = r;

if (smallest != i) {
    swap(&heap->arr[i], &heap->arr[smallest]);
    heapify(heap, smallest);
}
}

```

```

struct Node* extractMin(struct MinHeap* heap) {
    struct Node* temp = heap->arr[0];
    heap->arr[0] = heap->arr[--heap->size];
    heapify(heap, 0);
    return temp;
}

```

```

void insertHeap(struct MinHeap* heap, struct Node* node) {
    int i = heap->size++;
    heap->arr[i] = node;

    while (i && heap->arr[(i-1)/2]->freq > heap->arr[i]->freq) {
        swap(&heap->arr[i], &heap->arr[(i-1)/2]);
        i = (i-1)/2;
    }
}

```

```

struct Node* buildTree(struct MinHeap* heap) {
    while (heap->size > 1) {
        struct Node* left = extractMin(heap);
        struct Node* right = extractMin(heap);

        struct Node* newNode = createNode('$', left->freq + right->freq);
        newNode->left = left;
        newNode->right = right;

        insertHeap(heap, newNode);
    }
    return extractMin(heap);
}

```

```
void printCodes(struct Node* root, char code[], int top) {  
    if (!root) return;
```

```
    if (root->left) {  
        code[top] = '0';  
        printCodes(root->left, code, top + 1);  
    }
```

```
    if (root->right) {  
        code[top] = '1';  
        printCodes(root->right, code, top + 1);  
    }
```

```
    if (!root->left && !root->right) {  
        code[top] = '\0';  
        printf("%c: %s\n", root->ch, code);  
    }  
}
```

```
int main() {  
    int n;  
    scanf("%d", &n);
```

```
    struct MinHeap heap;  
    heap.size = 0;
```

```
    for (int i = 0; i < n; i++) {  
        char ch;  
        int freq;  
        scanf(" %c %d", &ch, &freq);  
        heap.arr[heap.size++] = createNode(ch, freq);  
    }
```

```
    for (int i = (heap.size - 1) / 2; i >= 0; i--) {  
        heapify(&heap, i);  
    }
```

```
    struct Node* root = buildTree(&heap);  
    char code[100];
```

```
    printCodes(root, code, 0);  
    return 0;  
}
```

Status : Correct

Marks : 10/10

5. Problem Statement

In a data science class, Professor Huffman demonstrates how encoded messages can be decoded using a pre-built Huffman tree.

Given a Huffman code table (mapping each character to its binary code) and an encoded binary message, decode the message back to its original text by traversing the Huffman tree.

The code table is guaranteed to be a valid prefix-free Huffman code.

The encoded message consists only of '0' and '1' characters, and every bit maps to exactly one symbol in the Huffman tree

The tree is defined as follows:

Internal nodes are represented by "*". The left edge (0) leads to the symbol "A". The right edge (1) leads to the symbol "B".

This means:

"0" A "1" B

Your task is to decode a given encoded message (a string of 0s and 1s) back into its original form consisting of "A" and "B".

Input Format

The first line contains an integer n , representing the number of symbols in the code table.

Each of the next n lines contains a character and its Huffman code (binary string), separated by a space.

The last line contains the encoded binary message (a string of 0s and 1s) to be decoded

Output Format

The output prints the decoded message as a sequence of characters ('A' or 'B'), followed by a newline.

Refer to the sample output for formatting specifications

Sample Test Case

Input: 11100011100

Output: BBBAABBBA

Answer

```
#include <stdio.h>
#include <string.h>

int main() {
    char encoded[105];

    scanf("%s", encoded);

    for (int i = 0; encoded[i] != '\0'; i++) {
        if (encoded[i] == '0')
            printf("A");
        else
            printf("B");
    }

    return 0;
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 7_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 14

Section 1 : MCQ

1. The frequencies of letters in a file are:

$s = 17, k = 24, y = 37$

A Huffman tree is constructed, and we assign 0 to the left branch and 1 to the right branch at each node.

Which of the following is the Huffman code for the letter 's'?

Answer

10

Status : Correct

Marks : 1/1

2. In Huffman coding, does data in a tree always occur?

Answer

Leaves

Status : Correct

Marks : 1/1

3. Which of the following algorithms is the best approach for solving Huffman codes?

Answer

Greedy algorithm

Status : Correct

Marks : 1/1

4. From the following given tree, what is the computed codeword for 'c'?

Answer

110

Status : Correct

Marks : 1/1

5. From the following given tree, what is the code word for the character 'a'?

Answer

011

Status : Correct

Marks : 1/1

6. Kruskal's algorithm is used to _____.

Answer

Find minimum spanning tree

Status : Correct

Marks : 1/1

7. Consider the following graph:

Which edges would be included in the minimum spanning tree using Prim's algorithm starting from vertex A?

Answer

AC, CD, DE, EB, BF

Status : Wrong

Marks : 0/1

8. Which of the following is true?

Answer

Prim's algorithm initializes with a vertex

Status : Correct

Marks : 1/1

9. Prim's algorithm can be efficiently implemented using _____ for graphs with greater density.

Answer

d-ary heap

Status : Correct

Marks : 1/1

10. Which of the following is false about Prim's algorithm?

Answer

It constructs MST by selecting edges in increasing order of their weights

Status : Correct

Marks : 1/1

11. What is the worst-case time complexity of Prim's algorithm if the adjacency matrix is used?

Answer

$O(V^2)$

Status : Correct

Marks : 1/1

12. Consider a graph $G = (V, E)$, where $V = \{v_1, v_2, \dots, v_{100}\}$, $E = \{(v_i, v_j) \mid 1 \leq i < j \leq 100\}$ and the weight of the edge (v_i, v_j) is $i - j$. The weight of the minimum spanning tree of G is _____.

Answer

99

Status : Correct

Marks : 1/1

13. An undirected graph $G(V, E)$ contains n ($n > 2$) nodes named $v_1, v_2, \dots, v_n$. Two nodes v_i, v_j are connected if and only if $0 < |i - j| \leq 2$. Each edge (v_i, v_j) is assigned a weight $i + j$. A sample graph with $n = 4$ is shown below. What will be the cost of the minimum spanning tree (MST) of such a graph with n nodes?

Answer

$n^2 - n + 1$

Status : Correct

Marks : 1/1

14. An undirected graph G has n nodes. Its adjacency matrix is given by an $n \times n$ square matrix whose (i) diagonal elements are 0's and (ii) non-diagonal elements are 1's. which one of the following is TRUE?

Answer

Graph G has multiple distinct MSTs, each of cost $n-1$

Status : Correct

Marks : 1/1

15. Consider a weighted complete graph G on the vertex set $\{v_1, v_2, \dots, v_n\}$ such that the weight of the edge (v_i, v_j) is $2|i-j|$. The weight of a minimum spanning tree of G is:

Answer

$2n-2$

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 8_COD

Attempt : 1

Total Mark : 40

Marks Obtained : 0

Section 1 : Coding

1. Problem Statement

Ram is tasked with finding the shortest distances from a source router to all other routers in a network. The network is represented as a graph of N routers connected by M bidirectional weighted links.

The program uses Dijkstra's Algorithm: starting from the source router, it repeatedly selects the unvisited router with the smallest known distance, then updates distances to its neighbors if a shorter path is found through it. This continues until all routers have been visited.

Given the network details and a source router, find and print the shortest distance from the source router to every router in the network.

Input Format

The first line of input consists of an integer N, representing the number of routers.

The second line of input consists of an integer M, representing the number of links.

The next M lines each consist of three space-separated integers: r1, r2, and w, representing a bidirectional link between router r1 and router r2 with weight w.

The last line of input consists of an integer s, representing the source router.

Output Format

The output consists of N lines printed in ascending order of router index (from 0 to N-1).

Each line prints two space-separated integers: the router index and its shortest distance from the source router.

The source router itself is printed with distance 0.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

5

0 1 1

0 2 3

1 2 1

1 3 4

2 3 1

0

Output: 0 0

1 1

2 2

3 3

Answer

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
if (!visited[v] && graph[u][v] != 0 && dist[u] != INT_MAX &&
```

```

    dist[u] + graph[u][v] < dist[v])
{
    dist[v] = dist[u] + graph[u][v];
}

int main() {
    int numRouters, numLinks;
    scanf("%d", &numRouters);
    scanf("%d", &numLinks);
    int graph[MAX_ROUTERS][MAX_ROUTERS] = {0};
    int adj[MAX_ROUTERS][MAX_ROUTERS] = {0};
    for (int i = 0; i < numLinks; i++) {
        int router1, router2, weight;
        scanf("%d %d %d", &router1, &router2, &weight);
        graph[router1][router2] = weight;
        graph[router2][router1] = weight;
        adj[router1][router2] = 1;
        adj[router2][router1] = 1;
    }
    int source;
    scanf("%d", &source);
    dijkstra(graph, adj, source, numRouters);
    return 0;
}

```

Status : Wrong

Marks : 0/10

2. Problem Statement

Alice is working on a delivery system for an e-commerce platform. The system uses a network of delivery hubs connected by roads, each with a certain travel time in minutes.

Alice needs to find the fastest delivery time from a source hub to every other hub in the network. Given the network details and a source hub, the program uses Dijkstra's Algorithm to find and print the shortest delivery time from the source hub to all hubs in the network.

Dijkstra's Algorithm works by starting from the source hub with distance 0, then repeatedly selecting the unvisited hub with the smallest known time and updating the times to its neighbors if a shorter path is found through

it. This continues until all hubs have been visited.

Input Format

The first line of input consists of an integer N, representing the number of delivery hubs.

The second line of input consists of an integer M, representing the number of roads between hubs.

The next M lines each consist of three space-separated integers: r1, r2, and t, representing a bidirectional road between hub r1 and hub r2 with a delivery time of t minutes.

The last line of input consists of an integer s, representing the source delivery hub.

Output Format

The output prints N lines representing the shortest delivery times from the source hub to all hubs (including the source) in ascending order of hub index.

Each line prints two space-separated integers: the hub index and its shortest delivery time from the source hub.

The delivery time of the source hub itself is 0.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

3

0 1 2

0 2 4

1 2 1

0

Output: 0 0

1 2

2 3

Answer

```
#include <stdio.h>
#include <limits.h>
```

```
#define MAX_NODES 10
```

```
int main() {
    int n, m;
    scanf("%d", &n);
    scanf("%d", &m);

    int graph[MAX][MAX];

    // initialize graph
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            graph[i][j] = 0;
        }
    }

    // read edges (undirected)
    for (int i = 0; i < m; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        graph[u][v] = w;
        graph[v][u] = w;
    }

    int src;
    scanf("%d", &src);

    int dist[MAX];
    int visited[MAX] = {0};

    for (int i = 0; i < n; i++) {
        dist[i] = INF;
    }

    dist[src] = 0;
```

```
for (int count = 0; count < n - 1; count++) {
```

```
    int min = INF, u = -1;
```

```
    for (int i = 0; i < n; i++) {  
        if (!visited[i] && dist[i] < min) {  
            min = dist[i];  
            u = i;  
        }  
    }  
}
```

```
visited[u] = 1;
```

```
for (int v = 0; v < n; v++) {  
    if (!visited[v] && graph[u][v] != 0 &&  
        dist[u] != INF &&  
        dist[u] + graph[u][v] < dist[v]) {  
  
        dist[v] = dist[u] + graph[u][v];  
    }  
}
```

```
for (int i = 0; i < n; i++) {  
    printf("%d %d\n", i, dist[i]);  
}
```

```
return 0;
```

```
int main() {
```

```
    int N, M, s;
```

```
    scanf("%d", &N);
```

```
    scanf("%d", &M);
```

```
    for (int i = 0; i < N; i++) {  
        for (int j = 0; j < N; j++) {  
            graph[i][j] = -1;  
        }  
    }
```

```
    for (int i = 0; i < M; i++) {  
        int r1, r2, w;  
        scanf("%d %d %d", &r1, &r2, &w);  
        graph[r1][r2] = w;
```

```

    graph[r2][r1] = w;
}
scanf("%d", &s);
dijkstra(N, s);
for (int i = 0; i < N; i++) {
    printf("%d %d\n", i, dist[i]);
}
return 0;
}

```

Status : Wrong

Marks : 0/10

3. Problem statement

Create a program to calculate and display the topological order of a Directed Acyclic Graph (DAG) with n vertices. The graph is represented as a lower triangular adjacency matrix, where for every vertex i and vertex j with $j < i$, the edge from i to j exists (value 1), and no edge exists from j to i (value 0). All diagonal and upper triangular values are 0.

The program should:

Construct and display the adjacency matrix of the graph. Perform a Depth First Search (DFS) traversal on the graph. Display the topological order of the vertices in reverse order of their DFS end times.

Input Format

The first line of input consists of an integer n , representing the number of vertices in the graph. No further input is required; the adjacency matrix is constructed internally by the program.

Output Format

The first line of output prints: Adjacency Matrix of the graph

The next n lines print the adjacency matrix row by row, where each value is preceded by a tab character (`\t`).

A blank line is printed after the matrix.

The next line prints: Topological order:

The final line prints the topological order of the vertices in the format: -->v1-->v2-->...-->vn, where vertices appear in reverse order of their DFS end times.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

Output: Adjacency Matrix of the graph

```
0 0 0
1 0 0
1 1 0
```

Topological order:

-->2-->1-->0

Answer

```
#include<stdio.h>
```

```
int s[100], res[100], top;
```

```
int adj[MAX][MAX];
int visited[MAX];
int stack[MAX];
int top = -1;
```

```
void dfs(int v, int n) {
    visited[v] = 1;

    for (int i = 0; i < n; i++) {
        if (adj[v][i] && !visited[i]) {
            dfs(i, n);
        }
    }
}
```

```
    stack[++top] = v;
}
```

```

// Print adjacency matrix
printf("Adjacency Matrix of the graph\n");
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        printf("\t%d", adj[i][j]);
    }
    printf("\n");
}

```

```

printf("\n");

```

```

// DFS topo sort
for (int i = 0; i < n; i++) {
    visited[i] = 0;
}

```

```

for (int i = 0; i < n; i++) {
    if (!visited[i]) {
        dfs(i, n);
    }
}

```

```

printf("Topological order:\n");

```

```

for (int i = top; i >= 0; i--) {
    printf("-->%d", stack[i]);
}

```

```

int main() {
    int a[100][100], n, i, j;
    scanf("%d", &n);
    buildMatrix(a, n);
    printf("Adjacency Matrix of the graph\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++)
            printf("\t%d", a[i][j]);
        printf("\n");
    }
    printf("\nTopological order:\n");
}

```

```

    topological_order(n, a);
    for (i = top - 1; i >= 0; i--)
        printf("-->%d", res[i]);
    printf("\n");
    return 0;
}

```

Status : Wrong

Marks : 0/10

4. Problem Statement

Write a program to implement and print the topological order of a directed acyclic graph (DAG). The program should take the number of vertices and the adjacency matrix of the graph as input and then output the topological order.

When multiple vertices have zero indegree simultaneously, process them in ascending order of their vertex number (1-indexed). The output vertices are numbered starting from 1

Input Format

The first line of input consists of an integer N, representing the number of vertices in the graph.

The following N lines contain the adjacency matrix of the graph. Each line contains N space-separated integers (0 or 1), where 1 indicates a directed edge from the corresponding row vertex to the column vertex.

Output Format

The output prints the topological order of the vertices as space-separated 1-indexed integers on a single line.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

```
1 0 0
0 1 0
1 0 0
```

Output: 3 1 2

Answer

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int adj[MAX][MAX];
```

```
    int indegree[MAX] = {0};
```

```
    int visited[MAX] = {0};
```

```
    // input matrix + compute indegree
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < n; j++) {
```

```
            scanf("%d", &adj[i][j]);
```

```
            if (adj[i][j] == 1) {
```

```
                indegree[j]++;
```

```
            }
```

```
        }
```

```
    }
```

```
    for (int count = 0; count < n; count++) {
```

```
        int v = -1;
```

```
        // pick smallest vertex with indegree 0
```

```
        for (int i = 0; i < n; i++) {
```

```
            if (!visited[i] && indegree[i] == 0) {
```

```
                v = i;
```

```
                break;
```

```
            }
```

```
        }
```

```
        visited[v] = 1;
```

```
        printf("%d ", v + 1); // convert to 1-indexed
```

```
// remove edges
for (int j = 0; j < n; j++) {
    if (adj[v][j] == 1) {
        indegree[j]--;
    }
}

return 0;
}
```

Status : Wrong

Marks : 0/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 8_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 31

Section 1 : Coding

1. Problem Statement

You are tasked with finding the shortest communication paths from a source router to all other routers in a network using Dijkstra's algorithm.

The network is represented as a graph, where routers are nodes and communication links between them are edges with weights representing the time or cost of communication. A value of 0 in the adjacency matrix indicates no direct link between two routers.

Example

Explanation:

The minimum distance from 0 to 1 = 4. Path: 0 1

The minimum distance from 0 to 2 = 12. Path: 0 1 2

The minimum distance from 0 to 3 = 19. Path: 0 1 2 3

The minimum distance from 0 to 4 = 21. Path: 0 7 6 5 4

The minimum distance from 0 to 5 = 11. Path: 0 7 6 5

The minimum distance from 0 to 6 = 9. Path: 0 7 6

The minimum distance from 0 to 7 = 8. Path: 0 7

The minimum distance from 0 to 8 = 14. Path: 0 1 2 8"

Input Format

The first line contains an integer V, representing the number of vertices in the graph.

The next V lines each contain V space-separated integers, representing the adjacency matrix of the graph. A value of 0 indicates no direct edge between the vertices.

The last line contains an integer src, representing the source vertex.

Output Format

The first line of output prints: Vertex followed by a tab character followed by Distance from Source

The next V lines each print the vertex number followed by three tab characters, a space, and the shortest distance from the source. Vertices are printed in ascending order from 0 to V-1.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

0 1 3 1

1 0 4 2

3 4 0 5

1 2 5 0

1

Output: Vertex Distance from Source

0 1

1 0

2 4

3 2

Answer

```
// You are using GCC#include <stdio.h>
```

```
#define INF 1000000
```

```
int main() {
```

```
    int V;
```

```
    scanf("%d", &V);
```

```
    int graph[10][10];
```

```
    for (int i = 0; i < V; i++) {
```

```
        for (int j = 0; j < V; j++) {
```

```
            scanf("%d", &graph[i][j]);
```

```
        }
```

```
    }
```

```
    int src;
```

```
    scanf("%d", &src);
```

```
    int dist[10];
```

```
    int visited[10] = {0};
```

```
    for (int i = 0; i < V; i++) {
```

```
        dist[i] = INF;
```

```
    }
```

```
    dist[src] = 0;
```

```
    for (int count = 0; count < V - 1; count++) {
```

```
        int min = INF, u = -1;
```

```
        for (int i = 0; i < V; i++) {
```

```
            if (!visited[i] && dist[i] < min) {
```



```

        min = dist[i];
        u = i;
    }
}

visited[u] = 1;

for (int v = 0; v < V; v++) {
    if (!visited[v] && graph[u][v] != 0 &&
        dist[u] != INF &&
        dist[u] + graph[u][v] < dist[v]) {

        dist[v] = dist[u] + graph[u][v];
    }
}

printf("Vertex \tDistance from Source\n");

for (int i = 0; i < V; i++) {
    printf("%d\t\t\t %d\n", i, dist[i]);
}

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Given a network of cities represented as an unweighted graph, find the shortest path between a source city and a destination city in terms of the minimum number of roads to be traversed. Implement a program that utilizes the Dijkstra algorithm to solve this problem.

The program should take the city network, source city, and destination city as inputs, and provide the shortest path length and the sequence of cities to be visited along the path as outputs.

If no path exists between the source and destination, print 'No path found'.

Note: Since the graph is unweighted, all edges have an implicit weight of 1.

Input Format

The first line contains an integer n , representing the number of cities in the network.

The second line contains an integer m , representing the number of roads in the city network.

The following m lines contain pairs of integers u and v , representing a road between cities u and v .

The next line contains an integer s , representing the source city.

The last line contains an integer d , representing the destination city.

Output Format

The first line prints Shortest path length: followed by the number of edges in the shortest path.

The second line prints Path: followed by the city indices along the shortest path, each followed by a space.

If no path exists, print No path found.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 8

10

0 1

0 3

1 2

3 4

3 7

4 5

4 6

4 7

5 6
6 7
0
7

Output: Shortest path length: 2
Path: 0 3 7

Answer

```
#include <stdio.h>
```

```
#define MAX 100
```

```
#define INF 1000000
```

```
int queue[MAX];
```

```
int front = 0, rear = 0;
```

```
void push(int x) {  
    queue[rear++] = x;  
}
```

```
int pop() {  
    return queue[front++];  
}
```

```
int empty() {  
    return front == rear;  
}
```

```
int main() {  
    int n, m;  
    scanf("%d", &n);  
    scanf("%d", &m);
```

```
    int adj[MAX][MAX] = {0};
```

```
    for (int i = 0; i < m; i++) {  
        int u, v;  
        scanf("%d %d", &u, &v);  
        adj[u][v] = 1;  
        adj[v][u] = 1;  
    }
```

```

int s, d;
scanf("%d %d", &s, &d);

int dist[MAX];
int parent[MAX];
int visited[MAX] = {0};

for (int i = 0; i < n; i++) {
    dist[i] = INF;
    parent[i] = -1;
}

// BFS
dist[s] = 0;
visited[s] = 1;
push(s);

while (!empty()) {
    int u = pop();

    for (int v = 0; v < n; v++) {
        if (adj[u][v] == 1 && !visited[v]) {
            visited[v] = 1;
            dist[v] = dist[u] + 1;
            parent[v] = u;
            push(v);
        }
    }
}

if (dist[d] == INF) {
    printf("No path found");
    return 0;
}

// reconstruct path
int path[MAX], len = 0;
int cur = d;

while (cur != -1) {
    path[len++] = cur;
    cur = parent[cur];
}

```

```

    }

    printf("Shortest path length: %d\n", dist[d]);
    printf("Path: ");

    for (int i = len - 1; i >= 0; i--) {
        printf("%d ", path[i]);
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Liam, a compiler designer, is optimizing task scheduling in a dependency-based system. He is working with a Directed Acyclic Graph (DAG), where each node represents a task, and a directed edge from one node to another signifies a dependency.

To schedule the tasks optimally, Liam must perform a topological sort using Depth-First Search (DFS), sorting the tasks based on their departure times in descending order.

Your task is to help Liam implement this topological sorting algorithm.

Input Format

The first line contains an integer N , representing the number of vertices (tasks) in the DAG.

The second line contains an integer E , representing the number of directed edges (dependencies) in the graph.

The next E lines each contain two space-separated integers:

- src The starting vertex of a directed edge.
- dest The ending vertex of a directed edge.

Output Format

The output prints N space-separated integers in a single line, representing the 0-indexed vertices in descending order of their DFS departure times, each followed by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

6

5 2

5 0

4 0

4 1

2 3

3 1

Output: 5 4 2 3 1 0

Answer

```
#include <stdio.h>
```

```
#define MAX 100
```

```
int adj[MAX][MAX];
```

```
int visited[MAX];
```

```
int stack[MAX];
```

```
int top = -1;
```

```
void dfs(int v, int n) {
```

```
    visited[v] = 1;
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (adj[v][i] && !visited[i]) {
```

```
            dfs(i, n);
```

```
        }
```

```
    }
```

```
    stack[++top] = v;
```

```
}
```

```

int main() {
    int n, e;
    scanf("%d", &n);
    scanf("%d", &e);

    for (int i = 0; i < e; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
    }

    for (int i = 0; i < n; i++) {
        visited[i] = 0;
    }

    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            dfs(i, n);
        }
    }

    for (int i = top; i >= 0; i--) {
        printf("%d ", stack[i]);
    }

    return 0;
}

```

Status : Partially correct

Marks : 1/10

4. Problem Statement:

You are given a directed acyclic graph (DAG) G with N vertices and M edges. From G , a new graph G^A is constructed using the same vertex set and the following rules:

If there exists an edge from vertex u to vertex v in G , then there is no edge from v to u in G^A . If there exists an edge from u to v in G , and from v to w in G , then there exists an edge from u to w in G^A . In G^A , find the maximum possible size of a set S of vertices such that for every pair of vertices (u, v) in S , there exists a directed edge between them (either $u \rightarrow v$ or $v \rightarrow u$).

Example:

Input: N=3, M=2, edges: 1 2, 2 3

G^* contains edges: 1 2, 2 3, and 1 3 (by rule 2).

Set $S = \{1, 2, 3\}$: edge exists between every pair (1 2, 2 3, 1 3). This is valid with size 3.

Output: 3

Input Format

The first line contains two space-separated integers N and M, representing the number of vertices and edges in graph G.

The next M lines each contain two space-separated integers u and v ($1 \leq u, v \leq N$), representing a directed edge from vertex u to vertex v.

Output Format

Print a single integer representing the maximum possible size of set S.

Refer to the sample output for formatting specifications

Sample Test Case

Input: 3 2

1 2

2 3

Output: 3

Answer

```
#include <stdio.h>
```

```
#define MAX 105
```

```
int adj[MAX][MAX];
```

```
int visited[MAX];
```

```
int stack[MAX];
```

```
int top = -1;
```



```

void dfs(int v, int n) {
    visited[v] = 1;

    for (int i = 1; i <= n; i++) {
        if (adj[v][i] && !visited[i]) {
            dfs(i, n);
        }
    }
}

```

```

    stack[++top] = v;
}

```

```

int main() {
    int n, m;
    scanf("%d %d", &n, &m);

    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
    }
}

```

```

for (int i = 1; i <= n; i++) visited[i] = 0;

```

```

for (int i = 1; i <= n; i++) {
    if (!visited[i]) dfs(i, n);
}

```

```

int dp[MAX] = {0};

```

```

for (int i = top; i >= 0; i--) {
    int u = stack[i];
    if (dp[u] == 0) dp[u] = 1;
}

```

```

for (int v = 1; v <= n; v++) {
    if (adj[u][v]) {
        if (dp[v] < dp[u] + 1) {
            dp[v] = dp[u] + 1;
        }
    }
}

```

```

}
}
}

```

```
int ans = 0;
for (int i = 1; i <= n; i++) {
    if (dp[i] > ans) ans = dp[i];
}

printf("%d", ans);

return 0;
}
```

Status : Correct

Marks : 10/10

5. Problem Statement

You are given a directed acyclic graph (DAG) represented as an adjacency list.

Your task is to perform a topological sort on the graph and print the vertices in topological order. The input graph is guaranteed to be a directed acyclic graph.

Input Format

The first line of input consists of an integer N, representing the number of vertices in the directed acyclic graph.

The second line consists of an integer E, representing the number of directed edges in the graph.

The following E lines consist of two space-separated integers, src and dest, representing a directed edge from vertex src to vertex dest.

Output Format

The output consists of the vertices in topological order separated by spaces.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6

6

5 2

5 0

4 0

4 1

2 3

3 1

Output: 4 5 0 2 3 1

Answer

```
#include <stdio.h>
```

```
#define MAX 105
```

```
int adj[MAX][MAX];  
int indegree[MAX];
```

```
int queue[MAX];  
int front = 0, rear = 0;
```

```
void push(int x) {  
    queue[rear++] = x;  
}
```

```
int pop() {  
    return queue[front++];  
}
```

```
int empty() {  
    return front == rear;  
}
```

```
int main() {  
    int n, e;  
    scanf("%d", &n);  
    scanf("%d", &e);
```

```
    for (int i = 0; i < e; i++) {  
        int u, v;
```

```

scanf("%d %d", &u, &v);
adj[u][v] = 1;
indegree[v]++;
}

for (int i = 0; i < n; i++) {
    if (indegree[i] == 0) {
        push(i);
    }
}

while (!empty()) {
    int u = pop();
    printf("%d ", u);

    for (int v = 0; v < n; v++) {
        if (adj[u][v]) {
            indegree[v]--;
            if (indegree[v] == 0) {
                push(v);
            }
        }
    }
}

return 0;
}

```

Status : Wrong

Marks : 0/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 8_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 11

Section 1 : MCQ

1. Which type of graphs can be topologically sorted?

Answer

Directed graphs

Status : Correct

Marks : 1/1

2. Topological sort can be used to detect which of the following?

Answer

Cycles in a directed graph

Status : Correct

Marks : 1/1

3. In topological sort, vertices are visited in which order?

Answer

In topological order

Status : Correct

Marks : 1/1

4. Which of the following data structures can be used to implement topological sort?

Answer

All of the listed options

Status : Wrong

Marks : 0/1

5. Which of the following algorithms can be used to perform topological sort?

Answer

Depth First Search (DFS)
Breadth First Search (BFS)

Status : Correct

Marks : 1/1

6. Topological sort is equivalent to which of the traversals in trees?

Answer

Pre-order traversal

Status : Correct

Marks : 1/1

7. When is the topological sort of a graph unique?

Answer

When there exists a hamiltonian path in the graph

Status : Correct

Marks : 1/1

8. In Dijkstra's algorithm, if two vertices u and v have the same tentative distance from the source, which one is chosen first?

Answer

It can be arbitrary

Status : Correct

Marks : 1/1

9. In a graph with non-negative weights, why is Dijkstra's algorithm guaranteed to find the shortest path?

Answer

Because once a vertex's distance is finalized, it cannot be improved

Status : Correct

Marks : 1/1

10. How can Dijkstra's algorithm be adapted to return not only distances but also the actual paths?

Answer

Maintain a parent array for each vertex

Status : Correct

Marks : 1/1

11. In Dijkstra's algorithm, what is the purpose of the relaxation step when processing an edge (u, v) ?

Answer

To update the distance to vertex v if a shorter path through u is found

Status : Correct

Marks : 1/1

12. In a graph with multiple edges between the same pair of vertices, how should Dijkstra's algorithm handle them?

Answer

Consider only the edge with the maximum weight

Status : Wrong

Marks : 0/1

13. If a graph has an edge weight of zero, will Dijkstra's algorithm still work correctly?

Answer

No, never

Status : Wrong

Marks : 0/1

14. Which property of Dijkstra's algorithm guarantees that the shortest path to a vertex is correct when the vertex is added to the finalized set?

Answer

Greedy choice property

Status : Wrong

Marks : 0/1

15. Why can Dijkstra's algorithm fail on a graph with a negative weight edge even if there are no negative cycles?

Answer

Because a previously finalized vertex may get a shorter path later

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 2_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

In a kingdom far away, two types of soup are prepared daily — Soup A and Soup B. The royal chef always starts with n milliliters of each soup. Each time a servant is served, one of the following four serving operations is chosen at random (each with equal probability of 0.25):

Serve 100 ml of Soup A and 0 ml of Soup B

Serve 75 ml of Soup A and 25 ml of Soup B

Serve 50 ml of Soup A and 50 ml of Soup B

Serve 25 ml of Soup A and 75 ml of Soup B

The serving continues until either or both soups become empty. If the required soup amount for an operation exceeds the available quantity,

serve as much as possible (up to what remains).

You are to compute the probability that Soup A becomes empty first, plus half the probability that both soups become empty at the same time.

Special Note:

Since all operations serve soup in multiples of 25 ml, you must first scale n down to units of 25 ml before recursive processing:

Example

Sample Input 1

$n = 50$

Sample Output 2

0.625

Explanation

If we choose the first two operations, A will become empty first.

For the third operation, A and B will become empty at the same time.

For the fourth operation, B will become empty first.

So the total probability of A becoming empty first plus half the probability that A and B become empty at the same time, is $0.25 * (1 + 1 + 0.5 + 0) = 0.625$.

Input Format

The input consists of an integer n , representing the total volume of the soup.

Output Format

The output prints the probability that soup A will be empty first, plus half the probability that A and B become empty at the same time.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 50

Output: 0.625

Answer

```
#include <stdio.h>
```

```
#include <math.h>
```

```
double dp[101][101];
```

```
double solve(int a, int b) {
```

```
    if (a <= 0 && b <= 0) return 0.5;
```

```
    if (a <= 0) return 1.0;
```

```
    if (b <= 0) return 0.0;
```

```
    if (dp[a][b] != -1.0)
```

```
        return dp[a][b];
```

```
    dp[a][b] = 0.25 * (
```

```
        solve(a - 4, b) +
```

```
        solve(a - 3, b - 1) +
```

```
        solve(a - 2, b - 2) +
```

```
        solve(a - 1, b - 3)
```

```
    );
```

```
    return dp[a][b];
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int m = ceil(n / 25.0);
```

```
    for (int i = 0; i <= 100; i++) {
```

```

    for (int j = 0; j <= 100; j++) {
        dp[i][j] = -1.0;
    }
}

double result = solve(m, m);

printf("%.3f", result);

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

In a unique team-building exercise, a company has decided to distribute a certain number of resources among its departments based on mathematical powers. The company has x resources that need to be distributed such that each department receives a different power of an integer. Specifically, each department i receives n resources, where n is a given power.

George, the resource manager, needs to find all possible ways to distribute exactly x resources using the powers of integers starting from 1. Each integer's power can only be used once.

Write a program to help George determine the number of ways to distribute x resources in this manner using a recursive manner.

Example 1

Input:

10

2

Output:

1

Explanation:

$$x = 10$$

$$n = 2$$

$10 = 1^2 + 3^2$, hence we have only 1 possibility.

Example 2

Input:

100

2

Output:

3

Explanation:

$$x = 100$$

$$n = 2$$

$100 = 10^2$ OR $6^2 + 8^2$ OR $1^2 + 3^2 + 4^2 + 5^2 + 7^2$, hence total 3 possibilities.

Input Format

The first line of input consists of the integer x , representing the total number of resources.

The second line consists of the integer n , representing the power to which the integers are raised.

Output Format

The output should be a single integer representing the number of ways to distribute exactly x resources.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 100

2

Output: 3

Answer

```
#include <stdio.h>
```

```
#include <math.h>
```

```
int countWays(int x, int n, int num) {  
    int power = pow(num, n);
```

```
    if (x == 0)  
        return 1;
```

```
    if (power > x)  
        return 0;
```

```
    return countWays(x - power, n, num + 1) +  
           countWays(x, n, num + 1);  
}
```

```
int main() {  
    int x, n;
```

```
    scanf("%d", &x);  
    scanf("%d", &n);
```

```
    printf("%d", countWays(x, n, 1));
```

```
    return 0;  
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

In a small research lab, a programmer named Arin is analyzing binary data

to detect system flags stored as set bits in an integer. One day Arin receives an integer and must determine how many bits are set to 1 in its binary representation.

Write a program to ensure that this task is done so Arin can continue validating the system's binary logs. The objective is to count all set bits in the given number using any valid approach.

Function specification: countSetBits(int n)

Input Format

The input consists of a single positive integer n.

Output Format

The output prints a single integer representing the count of set bits (1s) in the binary representation of n.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 21

Output: 3

Answer

```
#include <stdio.h>
```

```
int countSetBits(int n) {  
    int count = 0;  
  
    while (n > 0) {  
        count += (n & 1);  
        n = n >> 1;  
    }  
  
    return count;  
}
```

```
int main() {  
    int n;  
    scanf("%d", &n);  
  
    printf("%d", countSetBits(n));  
  
    return 0;  
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Given a square matrix of size $n \times n$, rotate the elements along each layer (or boundary) of the matrix by one position in a clockwise direction. The rotation should be performed layer by layer, starting from the outermost layer and proceeding inward.

Examples:

Input 2:

```
3 3  
1 2 3  
4 5 6  
7 8 9
```

Output 2:

```
4 1 2  
7 5 3  
8 9 6
```

Explanation:

For a clockwise boundary rotation by one step:

We start with the outermost layer (there's only one layer in a 3×3 matrix)

We save the top-left element (1)

Move left edge upward: 4 1, 7 4

Move bottom edge leftward: 8 7, 9 8

Move right edge downward: 6 9, 3 6

Move top edge rightward: 2 3

Place the saved element (1) in the second position of the top row, so 1 2

Input Format

The first line contains two integers R and C — the number of rows and columns in the matrix.

The next R lines each contain C integers, representing the elements of the matrix.

Output Format

The output displays the rotated matrix after performing the clockwise boundary rotation.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 4 4

1 2 3 4

5 6 7 8

9 10 11 12

13 14 15 16

Output: 5 1 2 3

9 10 6 4

13 11 7 8

14 15 16 12

Answer

```
#include <stdio.h>
```

```
int main() {  
    int n;
```

```
scanf("%d %d", &n, &n);
```

```
int matrix[n][n];  
for (int i = 0; i < n; i++)  
    for (int j = 0; j < n; j++)  
        scanf("%d", &matrix[i][j]);
```

```
int layers = n / 2;
```

```
for (int layer = 0; layer < layers; layer++) {  
    int first = layer;  
    int last = n - layer - 1;
```

```
    int temp = matrix[first][first];
```

```
    for (int i = first; i < last; i++)  
        matrix[i][first] = matrix[i + 1][first];
```

```
    for (int j = first; j < last; j++)  
        matrix[last][j] = matrix[last][j + 1];
```

```
    for (int i = last; i > first; i--)  
        matrix[i][last] = matrix[i - 1][last];
```

```
    for (int j = last; j > first + 1; j--)  
        matrix[first][j] = matrix[first][j - 1];
```

```
    matrix[first][first + 1] = temp;  
}
```

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++)  
        printf("%d ", matrix[i][j]);  
    printf("\n");  
}
```

```
    return 0;  
}
```

Status : Correct

Marks : 10/10

5. Problem Statement

Given a 2D matrix of size ROW $\times$ COL, print all elements of the matrix in diagonal order.

Diagonal Order Definition:

Start from the first element at position (0,0). Traverse all diagonals starting from the leftmost column going downward, then continue with diagonals starting from the bottom row going rightward. Within each diagonal, print elements from the top-right corner to bottom-left corner.

Example:

Input matrix

```
5 4  
1 2 3 4  
5 6 7 8  
9 10 11 12  
13 14 15 16  
17 18 19 20
```

Diagonal printing of the above matrix is

```
1  
5 2  
9 6 3  
13 10 7 4  
17 14 11 8  
18 15 12
```

19 16
20

Input Format

The first line of input contains two integers, ROW and COL.

The next ROW lines each contain COL space-separated integers

Output Format

The output consists of multiple lines, one for each diagonal. Each line contains the matrix elements of that diagonal in the order from top-right to bottom-left, separated by spaces.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 5 4
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
17 18 19 20
Output: 1
5 2
9 6 3
13 10 7 4
17 14 11 8
18 15 12
19 16
20

Answer

```
#include <stdio.h>
```

```
int main() {  
    int ROW, COL;  
    scanf("%d %d", &ROW, &COL);
```

```
int matrix[ROW][COL];
```

```
for (int i = 0; i < ROW; i++)  
    for (int j = 0; j < COL; j++)  
        scanf("%d", &matrix[i][j]);
```

```
for (int startRow = 0; startRow < ROW; startRow++) {  
    int r = startRow, c = 0;  
    while (r >= 0 && c < COL) {  
        printf("%d ", matrix[r][c]);  
        r--;  
        c++;  
    }  
    printf("\n");  
}
```

```
for (int startCol = 1; startCol < COL; startCol++) {  
    int r = ROW - 1, c = startCol;  
    while (r >= 0 && c < COL) {  
        printf("%d ", matrix[r][c]);  
        r--;  
        c++;  
    }  
    printf("\n");  
}
```

```
return 0;  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 9_COD

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Bob has a task to find a Hamiltonian cycle in a given undirected simple graph (no self-loops) represented as an adjacency matrix, where a value of 1 indicates an edge exists between two vertices and a value of 0 indicates no edge. A Hamiltonian cycle is a cycle that visits every vertex exactly once and returns to the starting vertex. Bob needs to write a program that will determine if such a cycle exists and, if so, print the cycle starting and ending at vertex 0. If no cycle is found, the program should output "No solution".

Your task is to help Bob implement this.

Input Format

The first line contains an integer V, representing the number of vertices.

The following V lines each contain V space-separated integers (0 or 1), representing the adjacency matrix. The matrix is symmetric (undirected graph) and has zeros on the diagonal (no self-loops)

Output Format

If a Hamiltonian cycle is found, the first line prints "Solution found" and the second line prints the cycle as a series of space-separated integers, representing the vertices starting and ending at vertex 0.

If no Hamiltonian cycle is found, print "No solution".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 8

0 1 0 1 1 0 0 0

1 0 1 0 0 1 0 0

0 1 0 1 0 0 1 0

1 0 1 0 0 0 0 1

1 0 0 0 0 1 0 1

0 1 0 0 1 0 1 0

0 0 1 0 0 1 0 1

0 0 0 1 1 0 1 0

Output: Solution found

0 1 2 3 7 6 5 4 0

Answer

```
#include <stdio.h>
```

```
#define MAX 8
```

```
int graph[MAX][MAX];
```

```
int path[MAX];
```

```
int visited[MAX];
```

```
int V;
```

```
int isSafe(int v, int pos) {
```

```
    if (graph[path[pos - 1]][v] == 0)
```

```

        return 0;
    if (visited[v])
        return 0;

    return 1;
}

int solve(int pos) {
    if (pos == V) {
        if (graph[path[pos - 1]][path[0]] == 1)
            return 1;
        else
            return 0;
    }

    for (int v = 1; v < V; v++) {
        if (isSafe(v, pos)) {
            path[pos] = v;
            visited[v] = 1;

            if (solve(pos + 1))
                return 1;

            visited[v] = 0;
        }
    }

    return 0;
}

```

```

int main() {
    scanf("%d", &V);

    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    for (int i = 0; i < V; i++)
        visited[i] = 0;
}

```



```
path[0] = 0;
visited[0] = 1;
```

```
if (solve(1)) {
    printf("Solution found\n");
    for (int i = 0; i < V; i++) {
        printf("%d ", path[i]);
    }
    printf("0");
} else {
    printf("No solution");
}
```

```
return 0;
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Given an undirected complete graph with N vertices ($N \geq 3$), find the number of distinct Hamiltonian cycles in the graph using backtracking. A Hamiltonian cycle is a cycle that visits each vertex exactly once and returns to the starting vertex.

Two Hamiltonian cycles are considered identical if one can be obtained from the other by rotation (changing the starting vertex) or reversal (traversing in the opposite direction).

Example:

For $N=4$, the vertices are 0, 1, 2, 3. The 3 distinct Hamiltonian cycles are:

0 1 2 3 00 1 3 2 00 2 1 3 0

Output: 3

Input Format

The input consists of a single integer N , representing the number of vertices in the complete graph.

Output Format

Print a single integer on its own line representing the number of distinct Hamiltonian cycles in the complete graph with N vertices.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

Output: 60

Answer

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int n;  
int visited[MAX];  
int path[MAX];  
long long count = 0;
```

```
void dfs(int pos) {  
    if (pos == n) {  
        // valid cycle found (last connects to 0 automatically in complete graph)  
        count++;  
        return;  
    }
```

```
    for (int i = 1; i < n; i++) {  
        if (!visited[i]) {  
            visited[i] = 1;  
            path[pos] = i;  
  
            dfs(pos + 1);  
  
            visited[i] = 0;
```

```
        }  
    }  
}
```

```

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    visited[0] = 1;
    path[0] = 0;

    dfs(1);

    // divide by 2 for reverse duplicates
    printf("%lld", count / 2);

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

A college is organizing an event and has collected donations from N sponsors. Each sponsor contributes a fixed amount. The event coordinator wants to know all possible groups of sponsors whose donation sum matches exactly the required event budget S.

Write a program using Backtracking to find all subsets of sponsor contributions whose sum is exactly equal to S.

A subset may contain any number of sponsors, but each sponsor amount can be used only once.

Formula:

A subset is valid if: $a_1 + a_2 + \dots + a_k = S$

Input Format

The first line contains an integer N, representing the number of sponsors.

The second line contains N space-separated integers in ascending order,

representing the donation amounts.

The third line contains an integer S, representing the required budget.

Output Format

Print all subsets whose sum is exactly S, one subset per line. Within each subset, print the elements in the order they appear in the input, each followed by a space. Subsets are printed in the order they are discovered during backtracking (subsets starting with smaller elements appear first).

If no subset exists, print: No subset found

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

10 20 30 40

50

Output: 10 40

20 30

Answer

```
#include <stdio.h>
```

```
int N, S;  
int arr[20];  
int path[20];  
int found = 0;
```

```
void printSubset(int len) {  
    for (int i = 0; i < len; i++) {  
        printf("%d ", path[i]);  
    }  
    printf("\n");  
}
```

```

void solve(int idx, int sum, int len) {
    if (sum == S) {
        found = 1;
        printSubset(len);
        return;
    }

    if (idx == N || sum > S)
        return;

    // Include current element
    path[len] = arr[idx];
    solve(idx + 1, sum + arr[idx], len + 1);

    // Exclude current element
    solve(idx + 1, sum, len);
}

int main() {
    scanf("%d", &N);

    for (int i = 0; i < N; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &S);

    solve(0, 0, 0);

    if (!found) {
        printf("No subset found");
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Emily is working on arranging queens on a chessboard where one queen

must already be fixed at a given position. She wants to find all possible configurations of placing N queens such that no two queens attack each other, with the condition that the queen at the specified position cannot be moved.

A queen can attack any cell in its row, column, or diagonal. Row and column indices are 0-based.

Write a program to ensure that the chessboard solutions are displayed correctly, or state "No solution" if none exist.

Input Format

The first line contains an integer N representing the size of the chessboard.

The second line contains an integer i, representing the row index of the fixed queen.

The third line contains an integer j, representing the column index of the fixed queen.

Output Format

The first line of output consists of a single floating-point number representing the maximum total value, formatted to two decimal places.

The next n lines each consist of a single floating-point number representing the fraction x_i of each item selected, formatted to two decimal places. x_i represents the fraction of the i-th item taken (1.00 means the item is fully taken, a value between 0.00 and 1.00 means a fraction is taken). The fractions should be printed in the same order as the input items.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

0

1

Output: . Q . .

. . . Q

Q . .
. Q .

Answer

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int N;  
int board[MAX][MAX];  
int col[MAX];  
int diag1[MAX * 2];  
int diag2[MAX * 2];  
int found = 0;
```

```
void printBoard() {  
    for (int i = 0; i < N; i++) {  
        for (int j = 0; j < N; j++) {  
            if (board[i][j])  
                printf("Q ");  
            else  
                printf(". ");  
        }  
        printf("\n");  
    }  
    printf("\n");  
}
```

```
void solve(int row) {  
    if (row == N) {  
        found = 1;  
        printBoard();  
        return;  
    }  
}
```

```
// Skip fixed queen row  
int fixedRow = -1, fixedCol = -1;
```

```
// find fixed queen position  
for (int i = 0; i < N; i++) {  
    for (int j = 0; j < N; j++) {
```

```

        if (board[i][j] == 2) { // fixed queen marker
            fixedRow = i;
            fixedCol = j;
        }
    }
}

if (row == fixedRow) {
    solve(row + 1);
    return;
}

for (int c = 0; c < N; c++) {
    if (col[c] || diag1[row - c + N] || diag2[row + c])
        continue;

    board[row][c] = 1;
    col[c] = diag1[row - c + N] = diag2[row + c] = 1;

    solve(row + 1);

    board[row][c] = 0;
    col[c] = diag1[row - c + N] = diag2[row + c] = 0;
}
}

int main() {
    int i, j;
    scanf("%d", &N);
    scanf("%d", &i);
    scanf("%d", &j);

    // initialize
    for (int a = 0; a < N; a++)
        for (int b = 0; b < N; b++)
            board[a][b] = 0;

    board[i][j] = 2; // fixed queen marker

    col[j] = 1;
    diag1[i - j + N] = 1;

```



```
    diag2[i + j] = 1;  
    solve(0);  
    if (!found)  
        printf("No solution");  
  
    return 0;  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 9_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Given an adjacency matrix $adj[][]$ of an undirected graph consisting of N vertices, find a Hamiltonian Path in the graph if one exists. If found, print the path as a sequence of vertex indices separated by $\rightarrow$. Otherwise, print No Hamiltonian Path found.

A Hamiltonian path visits each and every vertex of the graph exactly once. The adjacency matrix is symmetric with zeros on the diagonal (no self-loops).

Write a program using Backtracking to find the Hamiltonian Path.

Examples:

Input: $adj[][] = \{\{0, 1, 1, 1, 0\}, \{1, 0, 1, 0, 1\}, \{1, 1, 0, 1, 1\}, \{1, 0, 1, 0, 0\}\}$

Output: Yes

Explanation:

There exists a Hamiltonian Path for the given graph, as shown in the image below:

Input: $\text{adj}[][] = \{\{0, 1, 0, 0\}, \{1, 0, 1, 1\}, \{0, 1, 0, 0\}, \{0, 1, 0, 0\}\}$

Output: No

Input Format

The first line contains an integer N, representing the number of vertices in the graph.

The next N lines each contain N space-separated integers (0 or 1), representing the adjacency matrix of the graph.

Output Format

If a Hamiltonian path exists, print the vertices of the path separated by -> on a single line.

If no Hamiltonian path exists, print: No Hamiltonian Path found

Refer to the sample output for formatting specifications.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

0 1 1 1 0

1 0 1 0 1

1 1 0 1 1

1 0 1 0 0

0 1 1 0 0

Output: 0 -> 1 -> 4 -> 2 -> 3

Answer

```
#include <stdio.h>
```

```
int N;  
int adj[10][10];  
int path[10];  
int visited[10];
```

```
int found = 0;
```

```
int isSafe(int v, int pos) {  
    if (adj[path[pos - 1]][v] == 0)  
        return 0;  
    if (visited[v])  
        return 0;  
    return 1;  
}
```

```
void hamPath(int pos) {  
    if (found) return;
```

```
    if (pos == N) {  
        found = 1;  
        for (int i = 0; i < N; i++) {  
            printf("%d", path[i]);  
            if (i != N - 1) printf(" -> ");  
        }  
        return;  
    }
```

```
    for (int v = 0; v < N; v++) {  
        if (isSafe(v, pos)) {  
            path[pos] = v;  
            visited[v] = 1;
```

```
            hamPath(pos + 1);
```

```
            visited[v] = 0;
```

```
            if (found) return;
```

```
        }  
    }
```

```
    }
```

```

int main() {
    scanf("%d", &N);

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            scanf("%d", &adj[i][j]);
        }
    }

    for (int start = 0; start < N; start++) {
        for (int i = 0; i < N; i++) visited[i] = 0;

        path[0] = start;
        visited[start] = 1;

        hamPath(1);

        if (found) break;
    }

    if (!found) {
        printf("No Hamiltonian Path found");
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

You are given an undirected graph with N vertices. Implement a function `longest_hamiltonian_cycle` to find the longest Hamiltonian cycle in the graph. A Hamiltonian cycle is a cycle that visits every vertex exactly once, except for the starting and ending vertices, which are the same.

Input Format

The first line of input consists of an integer N representing the number of vertices in the graph.

The next N lines each contain N space-separated integers, representing the adjacency matrix of the graph. The value at `graph[i][j]` is 1 if there is an edge between vertex i and vertex j, and 0 otherwise.

Output Format

If a Hamiltonian cycle exists, print "Longest Hamiltonian Cycle:" followed by the vertices of the longest cycle found in order within square bracket separated by a comma.

If no Hamiltonian cycle exists, print "No Hamiltonian Cycle found".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 1 1 0

1 0 1 1

1 1 0 1

0 1 1 0

Output: Longest Hamiltonian Cycle: [0, 1, 3, 2]

Answer

```
#include <stdio.h>
```

```
int N;
```

```
int graph[10][10];
```

```
int path[10], bestPath[10];
```

```
int visited[10];
```

```
int bestLen = 0;
```

```
int isSafe(int v, int pos) {  
    if (!graph[path[pos - 1]][v]) return 0;  
    if (visited[v]) return 0;  
    return 1;  
}
```

```
void copyPath(int len) {
```

```

    for (int i = 0; i < len; i++) {
        bestPath[i] = path[i];
    }
}

```

```

void backtrack(int pos, int start) {
    if (pos == N) {
        if (graph[path[pos - 1]][start]) {
            if (N > bestLen) {
                bestLen = N;
                copyPath(N);
            }
        }
    }
    return;
}

```

```

for (int v = 0; v < N; v++) {
    if (isSafe(v, pos)) {
        path[pos] = v;
        visited[v] = 1;

        backtrack(pos + 1, start);

        visited[v] = 0;
    }
}
}

```

```

int main() {
    scanf("%d", &N);

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            scanf("%d", &graph[i][j]);
        }
    }
}

```

```

for (int start = 0; start < N; start++) {
    for (int i = 0; i < N; i++) visited[i] = 0;

    path[0] = start;
    visited[start] = 1;
}

```

```

    backtrack(1, start);
}

if (bestLen == 0) {
    printf("No Hamiltonian Cycle found");
} else {
    printf("Longest Hamiltonian Cycle: [");
    for (int i = 0; i < bestLen; i++) {
        printf("%d", bestPath[i]);
        if (i != bestLen - 1) printf(", ");
    }
    printf("]");
}

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Nira is studying the N-Queens problem and needs to write a program to solve it. The program should find a valid arrangement of N queens on an N×N chessboard, ensuring that no two queens can attack each other.

Two queens attack each other if they are in the same row, the same column, or the same diagonal. Once a solution is found, display the board with queens placed correctly. A solution is guaranteed to exist for all valid values of N.

Write a program using Backtracking to solve the N-Queens problem.

Example

Input:

4 // Value of N

Output:


```
0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0
```

Explanation: The output represents a valid arrangement of queens on the 4x4 chessboard, where 1 indicates the position of a queen and 0 indicates an empty cell.

In the first row, a queen is placed in the third column. In the second row, a queen is placed in the first column. In the third row, a queen is placed in the fourth column. In the fourth row, a queen is placed in the second column. This arrangement of queens on the board ensures that no two queens threaten each other.

Input Format

The input consists of a single integer N, representing the size of the chessboard (N×N).

Output Format

Print the N×N board as N rows of N space-separated integers. Each integer is followed by a space (including the last integer on each row). Print 1 where a queen is placed and 0 for an empty cell. Print any one valid solution.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

Output: 0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0

Answer

```
#include <stdio.h>
```

```
int N;  
int board[25][25];  
  
int isSafe(int row, int col) {  
    int i, j;  
  
    for (i = 0; i < col; i++)  
        if (board[row][i]) return 0;  
  
    for (i = row, j = col; i >= 0 && j >= 0; i--, j--)  
        if (board[i][j]) return 0;  
  
    for (i = row, j = col; i < N && j >= 0; i++, j--)  
        if (board[i][j]) return 0;  
  
    return 1;  
}
```

```
int solve(int col) {  
    if (col == N) return 1;  
  
    for (int i = 0; i < N; i++) {  
        if (isSafe(i, col)) {  
            board[i][col] = 1;  
  
            if (solve(col + 1)) return 1;  
  
            board[i][col] = 0;  
        }  
    }  
    return 0;  
}
```

```
int main() {  
    scanf("%d", &N);  
  
    for (int i = 0; i < N; i++)  
        for (int j = 0; j < N; j++)  
            board[i][j] = 0;  
  
    solve(0);  
}
```

```
for (int i = 0; i < N; i++) {  
    for (int j = 0; j < N; j++) {  
        printf("%d ", board[i][j]);  
    }  
    printf("\n");  
}  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

In chess, a knight moves in an "L" shape: two squares in one direction and one square perpendicular to it. From position (0, 0) on an infinite chessboard, a knight can move to one of the following eight positions in a single step:

(1, 2), (-1, 2), (1, -2), (-1, -2), (2, 1), (-2, 1), (2, -1), (-2, -1).

Given a target position (x, y) where both x and y are positive integers, find the minimum number of steps needed for the knight to travel from (0, 0) to (x, y) using BFS (Breadth-First Search).

Beginning from location (0,0), what is the minimum number of steps needed for a knight to get to some other arbitrary location (x,y)?

Input Format

The first line contains two space-separated integers x and y, representing the target position of the knight on the chessboard.

Output Format

Print a single integer representing the minimum number of steps needed for the knight to move from (0, 0) to (x, y).

Refer to the sample output for formatting specifications

Sample Test Case

Input: 1 2

Output: 1

Answer

```
#include <stdio.h>
```

```
#define MAX 1000
```

```
int visited[15][15];
```

```
typedef struct {  
    int x, y, d;  
} Node;
```

```
int dx[8] = {1, -1, 1, -1, 2, -2, 2, -2};  
int dy[8] = {2, 2, -2, -2, 1, 1, -1, -1};
```

```
int bfs(int tx, int ty) {  
    Node q[MAX];  
    int front = 0, rear = 0;
```

```
    q[rear++] = (Node){0, 0, 0};  
    visited[0][0] = 1;
```

```
    while (front < rear) {  
        Node cur = q[front++];
```

```
        if (cur.x == tx && cur.y == ty)  
            return cur.d;
```

```
        for (int i = 0; i < 8; i++) {  
            int nx = cur.x + dx[i];  
            int ny = cur.y + dy[i];
```

```
            if (nx >= 0 && ny >= 0 && nx <= 14 && ny <= 14 && !visited[nx][ny]) {  
                visited[nx][ny] = 1;  
                q[rear++] = (Node){nx, ny, cur.d + 1};
```

```
            }  
        }
```

```

    }
    return -1;
}

int main() {
    int x, y;
    scanf("%d %d", &x, &y);

    printf("%d", bfs(x, y));

    return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

A customer is shopping in a supermarket and has selected N items, each with a fixed price.

The customer has a discount coupon that can be applied only if the total price of some selected items is exactly equal to a target amount S.

Write a program using Backtracking to find all distinct subsets of item prices whose sum is exactly equal to S.

Each item price can be used only once. If the input contains duplicate prices, the output must not contain repeated subsets.

Formula: A subset is valid if: $p_1 + p_2 + \dots + p_k = S$

Print each valid subset with its elements in ascending order. Print subsets in the order they are discovered by the backtracking algorithm (which processes the sorted array from left to right).

Input Format

The first line contains an integer N representing the number of items.

The second line contains N space-separated integers representing item prices. The prices may include duplicate values.

The third line contains an integer S representing the target bill amount.

Output Format

Print each distinct valid subset on a separate line. Within each subset, print elements in ascending order, each followed by a space (including the last element).

Print subsets in the order they are found during backtracking on the sorted input.

If no valid subset exists, print: No subset found

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
5 5 10 15 20
25
Output: 5 5 15
5 20
10 15

Answer

```
#include <stdio.h>
#include <stdlib.h>
```

```
int arr[25];
int n, target;
int found = 0;
```

```
void printSubset(int temp[], int len) {
    for (int i = 0; i < len; i++) {
        printf("%d ", temp[i]);
    }
    printf("\n");
}
```

```
void backtrack(int idx, int temp[], int len, int sum) {
    if (sum == target) {
```

```
    printSubset(temp, len);  
    found = 1;  
    return;  
}
```

```
if (sum > target) return;
```

```
for (int i = idx; i < n; i++) {  
    if (i > idx && arr[i] == arr[i - 1]) continue;
```

```
    temp[len] = arr[i];  
    backtrack(i + 1, temp, len + 1, sum + arr[i]);
```

```
}
```

```
int main() {  
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++) {  
        scanf("%d", &arr[i]);  
    }
```

```
    scanf("%d", &target);
```

```
    // sort
```

```
    for (int i = 0; i < n - 1; i++) {  
        for (int j = i + 1; j < n; j++) {  
            if (arr[i] > arr[j]) {  
                int t = arr[i];  
                arr[i] = arr[j];  
                arr[j] = t;  
            }  
        }  
    }  
}
```

```
int temp[25];  
backtrack(0, temp, 0, 0);
```

```
if (!found) {  
    printf("No subset found");  
}
```

```
} return 0;
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 9_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 14

Section 1 : MCQ

1. The problem of finding a path in a graph that visits every vertex exactly once is called?

Answer

Hamiltonian path problem

Status : Correct

Marks : 1/1

2. In the Hamiltonian Cycle problem, if a graph contains a Hamiltonian cycle, it is called a

Answer

Hamiltonian graph

Status : Correct

Marks : 1/1

3. What is the primary difficulty in finding a Hamiltonian Path in a general graph?

Answer

It is an NP-complete problem.

Status : Correct

Marks : 1/1

4. Which of the following statements is true regarding the backtracking algorithm for the Hamiltonian Cycle problem?

Answer

It explores different paths in the graph to find a Hamiltonian cycle.

Status : Correct

Marks : 1/1

5. Which algorithmic paradigm is commonly used to solve the Hamiltonian Cycle problem?

Answer

Backtracking

Status : Correct

Marks : 1/1

6. The following paradigm can be used to find the solution to the problem in minimum time: Given a set of non-negative integers, and a value K, determine if there is a subset of the given set with a sum equal to K:

Answer

Dynamic Programming

Status : Correct

Marks : 1/1

7. What is a subset sum problem?

Answer

Finding a subset of a set that has the sum of elements equal to a given number

Status : Wrong

Marks : 0/1

8. Which of the following is not true about the subset sum problem?

Answer

the dynamic programming solution has a time complexity of $O(n \log n)$

Status : Correct

Marks : 1/1

9. What is the worst case time complexity of dynamic programming solution of the subset sum problem(sum=given subset sum)?

Answer

$O(\text{sum} * n)$

Status : Correct

Marks : 1/1

10. Placing N-queens so that no two queens attack each other is called _____.

Answer

N-queen's problem

Status : Correct

Marks : 1/1

11. Where is the n-queens problem implemented?

Answer

Chess

Status : Correct

Marks : 1/1

12. Which of the following methods can be used to solve N-Queen's problem?

Answer

Backtracking

Status : Correct

Marks : 1/1

13. Which one of the following is an application of the backtracking algorithm?

Answer

Crossword

Status : Correct

Marks : 1/1

14. For N Queens problem, the time complexity is

Answer

$O(N!)$

Status : Correct

Marks : 1/1

15. How many unique solutions are there to the 4 Queens problem, where solutions that are reflections or rotations of each other are considered the same?

Answer

2

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 10_COD

Attempt : 1

Total Mark : 30

Marks Obtained : 21

Section 1 : Coding

1. Problem Statement

Riya, the manager of a software development company, needs to assign three developers to three different projects.

Each developer has a different cost (in terms of time and resource usage) for completing each project. Riya wants to assign exactly one developer to each project such that the total development cost is minimized.

To solve this efficiently, she decides to use the Branch and Bound technique for the Assignment Problem.

Your task is to help Riya determine the minimum total cost of assigning developers to projects.

Input Format

The input consists of three lines with three space-separated integers each, representing a 3×3 cost matrix.

Each cell (i, j) represents the cost of assigning the ith developer to the jth project.

Output Format

The output should display the minimum total cost of the assignment in the following exact format: "Minimum total cost: X" , Where X is the minimum total cost obtained after assigning each worker to exactly one task.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10 12 19

10 5 7

15 14 13

Output: Minimum total cost: 28

Answer

```
#include <stdio.h>
```

```
#define N 3
```

```
#define INF 1000000
```

```
int cost[N][N];
```

```
int assigned[N];
```

```
int best = INF;
```

```
void backtrack(int worker, int currentCost) {
```

```
    if (currentCost >= best) return;
```

```
    if (worker == N) {
```

```
        if (currentCost < best)
```

```
            best = currentCost;
```

```
        return;
```

```
    }
```

```
    for (int job = 0; job < N; job++) {
```

```
        if (!assigned[job]) {
```

```

        assigned[job] = 1;
        backtrack(worker + 1, currentCost + cost[worker][job]);
        assigned[job] = 0;
    }
}
}

```

```

int main() {
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            scanf("%d", &cost[i][j]);

    for (int i = 0; i < N; i++)
        assigned[i] = 0;

    backtrack(0, 0);

    printf("Minimum total cost: %d", best);

    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Arjun is the manager of a disaster relief center responsible for sending essential supplies to a flood-affected area using a single rescue truck. The truck has a limited carrying capacity, so Arjun must carefully select which supply packages to load.

Each package has a specific weight and an importance score representing how critical it is for the affected people. Arjun wants to maximize the total importance score of the packages loaded onto the truck without exceeding its weight capacity.

To solve this efficiently, Arjun uses the Branch and Bound technique, which systematically explores possible combinations of packages while pruning choices that cannot produce better results.

Your task is to help Arjun determine the maximum total importance score that can be transported.

Input Format

The first line contains an integer N, representing the number of available packages.

The next N lines each contain two integers: weight[i] value[i]

- weight[i] – weight of the package
- value[i] – importance score of the package

The last line contains an integer W, representing the maximum capacity of the rescue truck.

Output Format

The output prints a single integer representing the maximum total importance score that can be carried without exceeding the truck's weight capacity.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 5

2 40

3 50

1 100

5 95

3 30

10

Output: 245

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct {  
    int w, v;
```



```
    double r;  
} Item;
```

```
Item a[1005];  
int n, W;
```

```
int cmp(const void *x, const void *y) {  
    Item *i = (Item *)x;  
    Item *j = (Item *)y;  
    if (j->r > i->r) return 1;  
    if (j->r < i->r) return -1;  
    return 0;  
}
```

```
double bound(int idx, int curW, int curV) {  
    double result = curV;  
    int totalW = curW;
```

```
    for (int i = idx; i < n; i++) {  
        if (totalW + a[i].w <= W) {  
            totalW += a[i].w;  
            result += a[i].v;  
        } else {  
            int remain = W - totalW;  
            result += a[i].r * remain;  
            break;  
        }  
    }
```

```
    return result;  
}
```

```
int best = 0;
```

```
void knap(int idx, int curW, int curV) {  
    if (curW > W) return;  
    if (curV > best) best = curV;  
    if (idx == n) return;
```

```
    double ub = bound(idx, curW, curV);  
    if (ub <= best) return;
```

```
    knap(idx + 1, curW + a[idx].w, curV + a[idx].v);
```

```

    knap(idx + 1, curW, curV);
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d %d", &a[i].w, &a[i].v);
        a[i].r = (double)a[i].v / a[i].w;
    }

    scanf("%d", &W);

    qsort(a, n, sizeof(Item), cmp);

    knap(0, 0, 0);

    printf("%d", best);

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

You are an art collector who wants to visit a set of art galleries in a city. Each gallery is at a different location, and the cost of traveling between galleries is known.

Your goal is to find the shortest route that allows you to visit each gallery exactly once and return to your starting point, using the traveling salesman problem (TSP). Write a program to calculate the minimum cost of this route.

Note: Solve it using Branch and Bound technique.

Input Format

The first line contains an integer n, the number of galleries.

The next n lines contain n integers each, representing the adjacency matrix.

The ith integer on the jth line represents the cost of traveling from gallery i to gallery j.

Output Format

The output displays "Minimum Cost is: " followed by the minimum cost of the route that visits each gallery exactly once and returns to the starting gallery.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Output: Minimum Cost is: 80

Answer

```
#include <stdio.h>
```

```
#define N 10
```

```
#define INF 1000000
```

```
int n;
```

```
int cost[N][N];
```

```
int visited[N];
```

```
int bestCost = INF;
```

```
int firstMin(int i) {
```

```
    int min = INF;
```

```
    for (int j = 0; j < n; j++) {
```

```
        if (i != j && cost[i][j] < min)
```

```
            min = cost[i][j];
```

```
    }
```

```
    return min;
```

```
}
```

```

int secondMin(int i) {
    int first = INF, second = INF;

    for (int j = 0; j < n; j++) {
        if (i == j) continue;

        if (cost[i][j] <= first) {
            second = first;
            first = cost[i][j];
        } else if (cost[i][j] <= second) {
            second = cost[i][j];
        }
    }
    return second;
}

```

```

void tsp(int level, int currCost, int currBound, int pathCost, int currNode) {
    if (level == n) {
        if (cost[currNode][0] != 0) {
            int totalCost = pathCost + cost[currNode][0];
            if (totalCost < bestCost)
                bestCost = totalCost;
        }
        return;
    }
}

```

```

for (int i = 0; i < n; i++) {
    if (cost[currNode][i] != 0 && !visited[i]) {
        int tempBound = currBound;

        pathCost += cost[currNode][i];

        if (level == 1)
            currBound -= (firstMin(currNode) + firstMin(i)) / 2;
        else
            currBound -= (secondMin(currNode) + firstMin(i)) / 2;

        if (currBound + pathCost < bestCost) {
            visited[i] = 1;
            tsp(level + 1, currCost, currBound, pathCost, i);
        }
    }
}

```

```
        pathCost -= cost[currNode][i];
    }
}
```

```
int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        visited[i] = 0;
        for (int j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
        }
    }
}
```

```
int currBound = 0;
```

```
for (int i = 0; i < n; i++)
    currBound += (firstMin(i) + secondMin(i));
```

```
currBound = (currBound % 2 == 0) ? currBound / 2 : currBound / 2 + 1;
```

```
visited[0] = 1;
tsp(1, 0, currBound, 0, 0);
```

```
printf("Minimum Cost is: %d", bestCost);
```

```
return 0;
```

```
}
```

Status : Partially correct

Marks : 1/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 10_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 42

Section 1 : Coding

1. Problem Statement

Meera, the director of a national disaster response team, must deploy three specialized rescue units to three critical disaster sites. Each rescue unit has a different response time (in minutes) to reach each site due to distance, terrain, and accessibility constraints.

However, Meera has an additional operational rule:

Rescue Unit 1 cannot be assigned to Site 3 due to equipment limitations. Rescue Unit 2 must not be assigned to Site 1 because of route restrictions.

Meera decides to use the Branch and Bound technique to explore only valid and efficient assignments while respecting all constraints.

Your task is to determine the total response time of the best valid

deployment plan that satisfies all given restrictions.

Input Format

The input consists of three lines with three space-separated integers each, representing a 3×3 response time matrix.

Each cell (i, j) represents the response time (in minutes) if the ith rescue unit is deployed to the jth disaster site.

Output Format

The output should display the total response time in the following exact format: "Total Response Time: X", where X is the total response time of the best valid deployment plan.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 12 25 40
30 14 28
16 18 11

Output: Total Response Time: 37

Answer

```
#include <stdio.h>
#include <limits.h>
```

```
int cost[3][3];
int used[3];
int minCost = INT_MAX;
```

```
void solve(int row, int currCost) {
    if (row == 3) {
        if (currCost < minCost)
            minCost = currCost;
        return;
    }
```

```
    for (int col = 0; col < 3; col++) {
```

```

// constraints:
// Unit 1 (row 0) cannot go to Site 3 (col 2)
// Unit 2 (row 1) cannot go to Site 1 (col 0)

if (!used[col]) {
    if (row == 0 && col == 2) continue;
    if (row == 1 && col == 0) continue;

    used[col] = 1;
    solve(row + 1, currCost + cost[row][col]);
    used[col] = 0;
}
}
}

int main() {
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            scanf("%d", &cost[i][j]);

    for (int i = 0; i < 3; i++)
        used[i] = 0;

    solve(0, 0);

    printf("Total Response Time: %d", minCost);

    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Captain Aidan, a daring pirate, has discovered an ancient treasure cave filled with valuable relics. However, the cave is protected by an ancient spell that limits the weight of the treasure he can carry. Aidan has a magic knapsack with a maximum weight capacity of W . Inside the cave, he finds n different treasures, each with a specific weight and value.

Since he can only take each treasure once, Aidan must carefully choose which items to take to maximize the total value without exceeding the weight limit of his knapsack.

Help Captain Aidan pick the most valuable treasures while staying within the weight limit using branch and bound technique

Input Format

The first line contains two integers n (the number of items) and W (the capacity of the knapsack), separated by a space.

The second line contains n integers, where the i -th integer represents the weight of the i -th item.

The third line contains n integers, where the i -th integer represents the value of the i -th item

Output Format

The output prints the single integer representing the maximum value obtained by filling the knapsack with the given set of items.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 60
10 20 30 40 50
60 100 120 140 180
Output: 280

Answer

```
#include <stdio.h>
```

```
int n, W;  
int w[105], v[105];
```

```
int best = 0;
```

```

void dfs(int i, int currW, int currV) {
    if (currW > W) return;

    if (i == n) {
        if (currV > best)
            best = currV;
        return;
    }

    // take item
    dfs(i + 1, currW + w[i], currV + v[i]);

    // skip item
    dfs(i + 1, currW, currV);
}

int main() {
    scanf("%d %d", &n, &W);

    for (int i = 0; i < n; i++)
        scanf("%d", &w[i]);

    for (int i = 0; i < n; i++)
        scanf("%d", &v[i]);

    dfs(0, 0, 0);

    printf("%d", best);

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Commander Vikram is preparing a spacecraft for an interplanetary research mission. The spacecraft has a limited cargo capacity, so he must carefully decide which scientific equipment modules to load.

Each module has a scientific importance value and a specific weight. Commander Vikram wants to load the spacecraft in such a way that the total scientific value of the selected modules is maximized without exceeding the cargo weight limit.

To solve this efficiently, he uses the Branch and Bound technique applied to the 0/1 Knapsack problem, which explores possible combinations of equipment and eliminates options that cannot lead to better solutions.

Your task is to help Commander Vikram determine the maximum achievable value and the selected module weights that should be loaded into the spacecraft.

Note: The weights must be printed in reverse order of the input, meaning the algorithm starts evaluating items from the last item and prints the selected weights first.

Input Format

The first line contains an integer n , representing the number of equipment modules available.

The second line contains n space-separated integers $val[i]$, representing the scientific value of each module.

The third line contains n space-separated integers $wt[i]$, representing the weight of each module.

The fourth line contains an integer W , representing the maximum cargo capacity of the spacecraft.

Output Format

The first line output displays "Maximum value: " followed by an integer, representing the maximum value that can be achieved.

The second line output displays "Selected weights: " followed by space-separated integers, printed in reverse order of the input.

Refer to the sample outputs for the exact format.

Sample Test Case

Input: 3
60 100 120
10 20 30
50

Output: Maximum value: 220
Selected weights: 30 20

Answer

```
#include <stdio.h>
```

```
int n, W;  
int val[15], wt[15];
```

```
int bestValue = 0;  
int bestTaken[15];  
int taken[15];
```

```
void dfs(int i, int currW, int currV) {  
    if (currW > W) return;
```

```
    if (i == n) {  
        if (currV > bestValue) {  
            bestValue = currV;  
            for (int j = 0; j < n; j++)  
                bestTaken[j] = taken[j];  
        }  
        return;  
    }
```

```
    // take item  
    taken[i] = 1;  
    dfs(i + 1, currW + wt[i], currV + val[i]);
```

```
    // not take item  
    taken[i] = 0;  
    dfs(i + 1, currW, currV);  
}
```

```
int main() {
```

```

scanf("%d", &n);

for (int i = 0; i < n; i++)
    scanf("%d", &val[i]);

for (int i = 0; i < n; i++)
    scanf("%d", &wt[i]);

scanf("%d", &W);

dfs(0, 0, 0);

printf("Maximum value: %d\n", bestValue);
printf("Selected weights: ");

// reverse order printing
for (int i = n - 1; i >= 0; i--) {
    if (bestTaken[i])
        printf("%d ", wt[i]);
}

return 0;
}

```

Status : Partially correct

Marks : 2/10

4. Problem Statement

You are tasked with solving the Travelling Salesman Problem (TSP) for a given set of cities.

Given the distances between pairs of cities, your goal is to find the shortest route that visits each city exactly once and returns to the starting city.

Note: Solve it using Branch and Bound approach.

Input Format

The first line of input contains an integer n , representing the number of cities.

The following n lines each contain n space-separated integers, representing the

distances between pairs of cities.

The distance between city i and city j is given at the i th row and j th column of the matrix.

Output Format

The first line displays "The Path is: " followed by the sequence of cities visited in the shortest path,

separated by "--->".

The second line displays "Minimum cost is X", where X is the minimum cost of the shortest path found.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 4 1 3

4 0 2 1

1 2 0 5

3 1 5 0

Output: The Path is: 1--->3--->2--->4--->1

Minimum cost is 7

Answer

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
int n;
```

```
int cost[15][15];
```

```
int bestCost = INT_MAX;
```

```
int visited[15];
```

```
int path[15], bestPath[15];
```

```
void copyPath() {
```

```
    for (int i = 0; i <= n; i++)
```

```

    bestPath[i] = path[i];
}

void tsp(int level, int currCost) {
    if (level == n) {
        if (cost[path[level - 1]][path[0]] != 0) {
            int total = currCost + cost[path[level - 1]][path[0]];

            if (total < bestCost) {
                bestCost = total;
                path[n] = path[0];
                copyPath();
            }
        }
    }
    return;
}

```

```

for (int i = 1; i < n; i++) {
    if (!visited[i] && cost[path[level - 1]][i] != 0) {
        visited[i] = 1;
        path[level] = i;

        tsp(level + 1, currCost + cost[path[level - 1]][i]);

        visited[i] = 0;
    }
}

```

```

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    path[0] = 0;
    visited[0] = 1;
}

```

```

tsp(1, 0);

printf("The Path is: ");
for (int i = 0; i <= n; i++) {
    printf("%d", bestPath[i] + 1);
    if (i < n) printf("---->");
}

printf("\nMinimum cost is %d", bestCost);

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

You are a logistics manager responsible for planning delivery routes for a fleet of vehicles. Each vehicle needs to visit a set of locations to deliver goods, and the cost of traveling between locations is known.

Your goal is to find the shortest route for each vehicle, ensuring that each location is visited exactly once before returning to the starting point.

Note: Solve it using Branch and Bound approach.

Input Format

The first line contains an integer n , the number of locations.

The next n lines contain n integers each, representing the cost matrix of traveling between locations.

The i th integer on the j th line represents the cost of traveling from location i to location j .

Output Format

The first line of output displays "Shortest Path: " followed by the shortest path that visits all cities.

The second line of output displays "Minimum cost is " followed by the minimum cost.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

0 10 8 9 7

10 0 10 5 6

8 10 0 8 9

9 5 8 0 6

7 6 9 6 0

Output: Shortest Path: 1 3 4 2 5 1

Minimum cost is 34

Answer

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#define MAX 10
```

```
int n;
```

```
int cost[MAX][MAX];
```

```
int visited[MAX];
```

```
int path[MAX];
```

```
int final_path[MAX];
```

```
int final_res = INT_MAX;
```

```
int first_min(int i) {
```

```
    int min = INT_MAX;
```

```
    for (int j = 0; j < n; j++)
```

```
        if (i != j && cost[i][j] < min)
```

```
            min = cost[i][j];
```

```
    return min;
```

```
}
```

```
int second_min(int i) {
```

```
    int first = INT_MAX, second = INT_MAX;
```

```
    for (int j = 0; j < n; j++) {
```

```
        if (i == j) continue;
```

```

        if (cost[i][j] <= first) {
            second = first;
            first = cost[i][j];
        } else if (cost[i][j] <= second && cost[i][j] != first) {
            second = cost[i][j];
        }
    }
    return second;
}

```

```

void copy_to_final(int curr_path[]) {
    for (int i = 0; i < n; i++)
        final_path[i] = curr_path[i];
    final_path[n] = curr_path[0];
}

```

```

void tsp_rec(int curr_bound, int curr_weight, int level, int curr_path[]) {
    if (level == n) {
        if (cost[curr_path[level - 1]][curr_path[0]] != 0) {
            int curr_res = curr_weight + cost[curr_path[level - 1]][curr_path[0]];
            if (curr_res < final_res) {
                copy_to_final(curr_path);
                final_res = curr_res;
            }
        }
    }
    return;
}

```

```

for (int i = 0; i < n; i++) {
    if (cost[curr_path[level - 1]][i] != 0 && visited[i] == 0) {
        int temp = curr_bound;
        curr_weight += cost[curr_path[level - 1]][i];

        if (level == 1)
            curr_bound -= (first_min(curr_path[level - 1]) + first_min(i)) / 2;
        else
            curr_bound -= (second_min(curr_path[level - 1]) + first_min(i)) / 2;

        if (curr_bound + curr_weight < final_res) {
            curr_path[level] = i;
            visited[i] = 1;
        }
    }
}

```

```

        tsp_rec(curr_bound, curr_weight, level + 1, curr_path);
    }

    curr_weight -= cost[curr_path[level - 1]][i];
    curr_bound = temp;

    for (int j = 0; j < n; j++) {
        if (visited[j]) {
            curr_path[level] = j;
            break;
        }
    }

    visited[i] = 0;
}
}

void tsp() {
    int curr_path[MAX];
    int curr_bound = 0;

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    for (int i = 0; i < n; i++)
        curr_bound += (first_min(i) + second_min(i));

    curr_bound = (curr_bound & 1) ? curr_bound / 2 + 1 : curr_bound / 2;

    visited[0] = 1;
    curr_path[0] = 0;

    tsp_rec(curr_bound, 0, 1, curr_path);

    printf("Shortest Path: ");
    for (int i = 0; i <= n; i++) {
        printf("%d", final_path[i] + 1);
        if (i != n) printf(" ");
    }
    printf("\nMinimum cost is %d", final_res);
}

```

```
}
```

```
int main() {
```

```
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++)
```

```
        for (int j = 0; j < n; j++)
```

```
            scanf("%d", &cost[i][j]);
```

```
    tsp();
```

```
    return 0;
```

```
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 10_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 14

Section 1 : MCQ

1. Which of the following methods is commonly used to solve assignment problems?

Answer

Hungarian method

Status : Correct

Marks : 1/1

2. The application of assignment problems is to obtain _____

Answer

Minimum cost or maximum profit

Status : Correct

Marks : 1/1

3. The assignment problem is said to be balanced if _____.

Answer

the number of rows is equal to the number of columns

Status : Correct

Marks : 1/1

4. The minimum number of lines covering all zeros in a reduced cost matrix of order n can be _____.

Answer

at the most n

Status : Correct

Marks : 1/1

5. The assignment problem _____.

Answer

All of the mentioned options

Status : Correct

Marks : 1/1

6. You are given a knapsack that can carry a maximum weight of 60. There are 4 items with weights of {20, 30, 40, and 70} and values {70, 80, 90, and 200}.

What is the maximum value of the items you can carry in the knapsack?

Answer

160

Status : Correct

Marks : 1/1

7. What is the objective of the knapsack problem?

Answer

To Get Maximum Total Value In The Knapsack

Status : Correct

Marks : 1/1

8. Which of the following methods is used to prune the search space in the Branch and Bound method for the Knapsack problem?

Answer

Ignoring nodes with a weight exceeding the capacity.

Status : Correct

Marks : 1/1

9. What is the key difference between Backtracking and Branch and Bound in solving the Knapsack problem?

Answer

Backtracking guarantees an optimal solution, while Branch and Bound does not.

Status : Wrong

Marks : 0/1

10. What is the primary advantage of using the Branch and Bound approach for the Knapsack problem?

Answer

It systematically explores possible solutions while pruning non-promising ones.

Status : Correct

Marks : 1/1

11. The travelling salesman problem can be solved using _____.

Answer

a minimum spanning tree

Status : Correct

Marks : 1/1

12. In a traveling salesman problem, the elements of diagonal from left-top to right-bottom are _____.

Answer

all infinity

Status : Correct

Marks : 1/1

13. Which type of graph is used to model the TSP problem?

Answer

Undirected Weighted Graph

Status : Correct

Marks : 1/1

14. What is the main goal of using Branch and Bound in TSP?

Answer

To minimize search space by pruning unpromising paths

Status : Correct

Marks : 1/1

15. Which value helps in deciding whether a node should be pruned in Branch and Bound?

Answer

Bound (lower bound of cost)

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 11_COD

Attempt : 1

Total Mark : 30

Marks Obtained : 30

Section 1 : Coding

1. Problem Statement

Alice needs to assign works to employees in a company. Each employee can be assigned at most one work, and each work can be assigned to at most one employee. Her goal is to maximize the number of employees who are assigned works, ensuring that each work is assigned to exactly one employee and each employee is assigned at most one work.

Help Alice to complete the task.

Input Format

The first line consists of an integer N, representing the number of connections between employees and works.

The next N lines consists of two space-separated integers X and Y, representing a connection where X is the index of an employee (0-based indexing), and Y is

the index of a work (0-based indexing), indicating that employee X can perform work Y.

Output Format

The first line of the output displays "Maximum number of employees that could get works are: a", where a is an integer representing the maximum number of employees that could be assigned works according to the given conditions.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 9

0 0
0 1
0 2
1 0
2 1
3 4
4 3
4 4
5 5

Output: Maximum number of employees that could get works are: 6

Answer

```
#include <stdio.h>
#include <string.h>
```

```
#define MAX 50
```

```
int adj[MAX][MAX];
int matchR[MAX];
int vis[MAX];
int n;
int maxNode = 0;
```

```
int dfs(int u) {
    for (int v = 0; v <= maxNode; v++) {
        if (adj[u][v] && !vis[v]) {
            vis[v] = 1;
```

```

        if (matchR[v] == -1 || dfs(matchR[v])) {
            matchR[v] = u;
            return 1;
        }
    }
}
return 0;
}

```

```

int main() {
    scanf("%d", &n);

    memset(adj, 0, sizeof(adj));
    memset(matchR, -1, sizeof(matchR));

    int x, y;

    for (int i = 0; i < n; i++) {
        scanf("%d %d", &x, &y);
        adj[x][y] = 1;

        if (x > maxNode) maxNode = x;
        if (y > maxNode) maxNode = y;
    }

    int result = 0;

    for (int i = 0; i <= maxNode; i++) {
        memset(vis, 0, sizeof(vis));
        if (dfs(i)) result++;
    }

    printf("Maximum number of employees that could get works are: %d", result);

    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Given an undirected bipartite graph with X vertices on the left side and Y vertices on the right side, the graph may contain multiple edges between the same pair of vertices.

Your task is to determine the minimum vertex cover for this graph.

A vertex cover of a graph is a set of vertices such that every edge in the graph is incident to at least one vertex in the set.

Input Format

The first line contains two space-separated integers X and Y , representing the number of vertices in the left set and right set, respectively.

The second line contains an integer e , representing the number of edges in the bipartite graph.

The next e lines each contain two space-separated integers u and v , representing an edge between: vertex u in the left set and vertex v in the right set

Output Format

The output prints the vertices in the minimum vertex cover, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4 4

4

0 1

0 2

1 3

2 3

Output: 0 3

Answer

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <limits.h>
```

```
#define MAX 105
```

```
int X, Y;  
int adj[MAX][MAX];
```

```
int pairU[MAX], pairV[MAX], dist[MAX];
```

```
int bfs() {  
    int queue[MAX], front = 0, rear = 0;
```

```
    for (int u = 0; u < X; u++) {  
        if (pairU[u] == -1) {  
            dist[u] = 0;  
            queue[rear++] = u;  
        } else {  
            dist[u] = INT_MAX;  
        }  
    }  
}
```

```
dist[-1] = INT_MAX;
```

```
while (front < rear) {  
    int u = queue[front++];
```

```
    if (dist[u] < dist[-1]) {  
        for (int v = 0; v < Y; v++) {  
            if (adj[u][v]) {  
                if (pairV[v] == -1) {  
                    dist[-1] = dist[u] + 1;  
                } else if (dist[pairV[v]] == INT_MAX) {  
                    dist[pairV[v]] = dist[u] + 1;  
                    queue[rear++] = pairV[v];  
                }  
            }  
        }  
    }  
}
```

```
return dist[-1] != INT_MAX;
```

```
}
```

```

int dfs(int u) {
    if (u != -1) {
        for (int v = 0; v < Y; v++) {
            if (adj[u][v]) {
                if (pairV[v] == -1 ||
                    (dist[pairV[v]] == dist[u] + 1 && dfs(pairV[v]))) {
                    pairU[u] = v;
                    pairV[v] = u;
                    return 1;
                }
            }
        }
        dist[u] = INT_MAX;
        return 0;
    }
    return 1;
}

```

```

void hopcroftKarp() {
    for (int i = 0; i < X; i++) pairU[i] = -1;
    for (int i = 0; i < Y; i++) pairV[i] = -1;

    while (bfs()) {
        for (int u = 0; u < X; u++) {
            if (pairU[u] == -1)
                dfs(u);
        }
    }
}

```

```

void printVertexCover() {
    int visU[MAX] = {0}, visV[MAX] = {0};
    int queue[MAX], front = 0, rear = 0;

    // free left vertices
    for (int u = 0; u < X; u++) {
        if (pairU[u] == -1) {
            queue[rear++] = u;
            visU[u] = 1;
        }
    }
}

```

```

// alternating BFS
while (front < rear) {
    int u = queue[front++];

    for (int v = 0; v < Y; v++) {
        if (adj[u][v] && !visV[v]) {
            visV[v] = 1;

            if (pairU[u] != v && pairV[v] != -1 && !visU[pairV[v]]) {
                visU[pairV[v]] = 1;
                queue[rear++] = pairV[v];
            }
        }
    }
}

```

```

// vertex cover = (Left not visited) U (Right visited)
for (int u = 0; u < X; u++) {
    if (!visU[u])
        printf("%d ", u);
}

```

```

for (int v = 0; v < Y; v++) {
    if (visV[v])
        printf("%d ", v);
}

```

```

int main() {
    scanf("%d %d", &X, &Y);

    int e;
    scanf("%d", &e);

    memset(adj, 0, sizeof(adj));

    for (int i = 0; i < e; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
    }
}

```

```
hopcroftKarp();  
printVertexCover();  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

You are given an undirected graph with 'num_vertices' vertices and 'num_edges' edges. Your task is to determine whether the given graph is bipartite or not. A graph is bipartite if it can be divided into two disjoint sets of vertices such that each edge connects a vertex from one set to a vertex from the other set.

Input Format

The first line contains two integers separated by a space: 'num_vertices' and 'num_edges'. These represent the number of vertices and edges in the graph, respectively.

The next 'num_edges' lines each contain two integers 'u' and 'v' separated by a space, representing an edge between vertex 'u' and vertex 'v'.

Output Format

The output displays a single line indicating whether the given graph is bipartite or not.

If the graph is bipartite, print "The graph is bipartite."

If the graph is not bipartite, print "The graph is not bipartite."

Sample Test Case

Input: 4 4

0 1

1 2

2 3

3 0

Output: The graph is bipartite.

Answer

```
#include <stdio.h>
```

```
#define MAX 105
```

```
int adj[MAX][MAX];
```

```
int color[MAX];
```

```
int queue[MAX];
```

```
int isBipartite(int n) {
```

```
    for (int i = 0; i < n; i++)
```

```
        color[i] = -1;
```

```
    for (int start = 0; start < n; start++) {
```

```
        if (color[start] != -1) continue;
```

```
        int front = 0, rear = 0;
```

```
        queue[rear++] = start;
```

```
        color[start] = 0;
```

```
        while (front < rear) {
```

```
            int u = queue[front++];
```

```
            for (int v = 0; v < n; v++) {
```

```
                if (adj[u][v]) {
```

```
                    if (color[v] == -1) {
```

```
                        color[v] = 1 - color[u];
```

```
                        queue[rear++] = v;
```

```
                    }
```

```
                    else if (color[v] == color[u]) {
```

```
                        return 0;
```

```
                    }
```

```
                }
```

```
            }
```

```
        }
```

```
    }
```

```
    return 1;
```

```
}
```

```
int main() {
```

```
int n, m;
scanf("%d %d", &n, &m);

for (int i = 0; i < m; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    adj[u][v] = 1;
    adj[v][u] = 1;
}

if (isBipartite(n))
    printf("The graph is bipartite.");
else
    printf("The graph is not bipartite.");
return 0;
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 11_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 14

Section 1 : Coding

1. _____ is a matching with the largest number of edges.

Answer

Maximum bipartite matching

Status : Correct

Marks : 1/1

2. How many colors are used in a bipartite graph?

Answer

2

Status : Correct

Marks : 1/1

3. What is the simplest method to prove that a graph is bipartite?

Answer

It does not have a cycle of an odd length

Status : Correct

Marks : 1/1

4. A matching that matches all the vertices of a graph is called?

Answer

Perfect matching

Status : Correct

Marks : 1/1

5. In a bipartite graph $G=(V,U,E)$, the matching of a free vertex in V to a free vertex in U called?

Answer

Augmenting

Status : Correct

Marks : 1/1

6. There are four students in a class namely A, B, C and D. A tells that a triangle is a bipartite graph. B tells pentagon is a bipartite graph. C tells square is a bipartite graph. D tells heptagon is a bipartite graph. Who among the following is correct?

Answer

C

Status : Correct

Marks : 1/1

7. Which of the following is not a property of the bipartite graph?

Answer

Asymmetric spectrum

Status : Correct

Marks : 1/1

8. The maximum number of edges in a bipartite graph with 14 vertices is _____.

Answer

49

Status : Correct

Marks : 1/1

9. What does the Halting Problem attempt to determine?

Answer

Whether a program will eventually stop running or continue forever

Status : Correct

Marks : 1/1

10. The Halting Problem was introduced by which computer scientist?

Answer

Alan Turing

Status : Correct

Marks : 1/1

11. Why is the Halting Problem considered unsolvable?

Answer

Because no algorithm can determine for every program whether it halts or runs forever

Status : Correct

Marks : 1/1

12. In computation theory, what does an undecidable problem mean?

Answer

A problem that cannot be implemented in programming languages

Status : Wrong

Marks : 0/1

13. In the context of the Halting Problem, what does the term halting refer to?

Answer

A program stopping its execution after completing its task

Status : Correct

Marks : 1/1

14. What is the main implication of the Halting Problem for computer science?

Answer

Some computational problems cannot be solved by any algorithm

Status : Correct

Marks : 1/1

15. The Halting Problem generally takes which of the following as input?

Answer

A program and its input data

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 12_COD

Attempt : 1

Total Mark : 30

Marks Obtained : 29

Section 1 : Coding

1. Problem Statement

Raj is developing financial software that must identify all non-empty subsets of a given list of expenses whose total sum equals a specified target value.

Each expense can either be included or excluded from a subset. The same expense cannot be used more than once in a subset.

Your task is to print all such valid subsets. If no valid non-empty subset exists whose sum equals the target value, print:

No subset matches the target sum

This problem demonstrates how the number of possible subsets increases exponentially when using brute-force approaches.

Formula

Subset Sum = sum of selected elements in the subset

A subset is considered valid if:

$\text{sum}(\text{subset elements}) = \text{target}$

Input Format

The first line contains an integer n , representing the number of expenses.

The second line contains n space-separated integers representing the expense values.

The third line contains an integer t , representing the target sum.

Output Format

Print each valid non-empty subset whose elements sum to t , one subset per line.

Elements of a subset must be printed as space-separated integers.

If no such subset exists, print:

No subset matches the target sum

Refer to the sample output for formatting.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

1 2 3

3

Output: 1 2

3

Answer

```
#include <stdio.h>
```

```
int n, target;  
int arr[25];  
int subset[25];  
int found = 0;
```

```
void printSubset(int size) {  
    for (int i = 0; i < size; i++) {  
        printf("%d ", subset[i]);  
    }  
    printf("\n");  
}
```

```
void backtrack(int idx, int sum, int size) {  
    if (sum == target && size > 0) {  
        printSubset(size);  
        found = 1;  
        return;  
    }
```

```
    if (idx == n) return;
```

```
    // Include current element  
    subset[size] = arr[idx];  
    backtrack(idx + 1, sum + arr[idx], size + 1);
```

```
    // Exclude current element  
    backtrack(idx + 1, sum, size);  
}
```

```
int main() {  
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++)  
        scanf("%d", &arr[i]);
```

```
    scanf("%d", &target);
```

```
    backtrack(0, 0, 0);
```

```
    if (!found) {  
        printf("No subset matches the target sum");  
    }  
  
    return 0;  
}
```

Status : Partially correct

Marks : 9/10

2. Problem Statement

Sanjana is a cybersecurity analyst responsible for securing communication in a corporate network. The network consists of multiple computers (nodes) connected by communication links (edges).

To ensure secure communication, Sanjana wants to select a group of computers such that every pair of selected computers is directly connected to each other. This ensures a fully trusted group where all members can communicate securely without intermediaries. Such a group is called a clique.

Sanjana is given a list of connections in the network, and she needs to determine whether there exists a clique of size k .

Input Format

The first line contains two integers n and m , representing the number of computers (nodes) and connections (edges).

The next m lines each contain two integers u and v , representing a connection between nodes u and v .

The last line contains an integer k , representing the required clique size.

Output Format

If a clique of size k exists, the output prints: "Clique Found: $v_1 v_2 v_3 \dots$ "

Otherwise, the output prints: "No clique of given size exists"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 6

0 1

0 2

1 2

1 3

2 3

3 4

3

Output: Clique Found: 0 1 2

Answer

```
#include <stdio.h>
```

```
#define MAX 25
```

```
int n, m, k;  
int graph[MAX][MAX];  
int path[MAX];  
int found = 0;
```

```
int isValid(int v, int pos) {  
    for (int i = 0; i < pos; i++) {  
        if (!graph[path[i]][v])  
            return 0;  
    }  
    return 1;  
}
```

```
void printClique(int size) {  
    printf("Clique Found: ");  
    for (int i = 0; i < size; i++) {  
        printf("%d ", path[i]);  
    }  
    printf("\n");  
}
```

```
void backtrack(int start, int depth) {
```

```

    if (found) return;

    if (depth == k) {
        printClique(k);
        found = 1;
        return;
    }

    for (int v = start; v < n; v++) {
        if (isValid(v, depth)) {
            path[depth] = v;
            backtrack(v + 1, depth + 1);
        }
    }
}

int main() {
    scanf("%d %d", &n, &m);

    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        graph[u][v] = 1;
        graph[v][u] = 1;
    }

    scanf("%d", &k);

    backtrack(0, 0);

    if (!found) {
        printf("No clique of given size exists");
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Rehan, a logistics manager, is packing valuable items into two separate containers. Each item has a specific value, and he wants both containers to carry the same total value to ensure balance and fairness. He needs to decide if it's possible to divide the items this way and print one such valid division if it exists.

This problem is based on a classic NP-complete problem known as the Partition Problem, where finding a solution may require checking all combinations in the worst case.

Help him write a program to solve this problem.

Input Format

The first line of input consists of an integer n , representing the number of items.

The second line contains n space-separated integers, where each integer represents the value of an item.

Output Format

If a valid equal partition exists, print:

- Subset 1: x_1 x_2 x_3 ...
- Subset 2: y_1 y_2 y_3 ...

If no such partition exists, it prints "No valid equal partition exists."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

1 5 11 5

Output: Subset 1: 1 5 5

Subset 2: 11

Answer

```
#include <stdio.h>
```

```
#define MAX 20
```

```
int n;  
int arr[MAX];  
int subset[MAX];  
int used[MAX];  
int total = 0;  
int found = 0;
```

```
void printResult() {  
    printf("Subset 1: ");  
    for (int i = 0; i < n; i++) {  
        if (used[i])  
            printf("%d ", arr[i]);  
    }  
    printf("\nSubset 2: ");  
    for (int i = 0; i < n; i++) {  
        if (!used[i])  
            printf("%d ", arr[i]);  
    }  
    printf("\n");  
}
```

```
void backtrack(int idx, int sum) {  
    if (found) return;  
  
    if (sum == total / 2) {  
        found = 1;  
        printResult();  
        return;  
    }  
  
    if (idx == n) return;
```

```
    // Include current element in subset 1  
    used[idx] = 1;  
    backtrack(idx + 1, sum + arr[idx]);
```

```
    // Backtrack  
    used[idx] = 0;
```

```
    // Exclude current element
```

```
    backtrack(idx + 1, sum);
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        total += arr[i];
    }

    // If total is odd, no partition possible
    if (total % 2 != 0) {
        printf("No valid equal partition exists.");
        return 0;
    }

    backtrack(0, 0);

    if (!found) {
        printf("No valid equal partition exists.");
    }

    return 0;
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 12_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 14

Section 1 : MCQ

1. How many conditions have to be met if an NP-complete problem is polynomially reducible?

Answer

2

Status : Correct

Marks : 1/1

2. _____ is the class of decision problems that can be solved by non-deterministic polynomial algorithms.

Answer

NP

Status : Correct

Marks : 1/1

3. To which of the following classes does a CNF-satisfiability problem belong?

Answer

NP complete

Status : Correct

Marks : 1/1

4. For problems X and Y, Y is NP-complete and X reduces to Y in polynomial time. Which of the following is TRUE?

Answer

X is in NP, but not necessarily NP-complete

Status : Correct

Marks : 1/1

5. A problem in NP is NP-complete if

Answer

It can be reduced to any other problem in NP in polynomial time

Status : Wrong

Marks : 0/1

6. Which class contains problems that can be solved in polynomial time using a deterministic machine?

Answer

P

Status : Correct

Marks : 1/1

7. What is the key property of problems in NP?

Answer

Their solutions can be verified in polynomial time

Status : Correct

Marks : 1/1

8. What does it mean if a problem is in both NP and NP-Complete?

Answer

It is the hardest among NP problems

Status : Correct

Marks : 1/1

9. If $P = NP$ is ever proven, what would it imply?

Answer

All problems in NP can be solved in polynomial time

Status : Correct

Marks : 1/1

10. What is the class NP named after?

Answer

Nondeterministic Polynomial time

Status : Correct

Marks : 1/1

11. Which complexity class includes problems that are at least as hard as the hardest problems in NP?

Answer

NP-hard

Status : Correct

Marks : 1/1

12. What is the defining property of NP-hard problems?

Answer

They are at least as hard as the hardest problems in NP.

Status : Correct

Marks : 1/1

13. In computational complexity theory, what does "NP" stand for?

Answer

Non-Deterministic Polynomial Time

Status : Correct

Marks : 1/1

14. What is the main characteristic that distinguishes NP-complete problems from NP-hard problems?

Answer

NP-complete problems are the hardest problems in NP.

Status : Correct

Marks : 1/1

15. The concept of "polynomial-time reduction" is commonly used to establish which of the following relationships between problems?

Answer

NP-hardness

Status : Correct

Marks : 1/1